

İstatistik, Olasılık ve Makine Öğrenmesi

Python ile Kapsamlı Ders Dokümanı

Gerçek Veri Setleriyle, Teoriden Uçtan Uca Projelere

Bu belge üç konuyu tek bir zincir olarak anlatır: **olasılık** belirsizliğin dilidir, **istatistik** veriden çıkarım yapmanın yöntemidir, **makine öğrenmesi** ise bu çıkarımı otomatikleştiren mühendisliktir. Her kavram önce sezgisel, sonra matematiksel olarak verilir; ardından **gerçek veri setleri** üzerinde Python'la uygulanır — çıktılarıyla birlikte.

Hiçbir formül açıklamasız bırakılmamıştır. Her denklemin ardından *o denklemin nedir, niçin böyle kurulmuştur, hangi soruyu cevaplar ve gerçek hayatta nerede kullanılır* sorularını yanıtlayan bir açıklama kutusu ve konuyu somutlaştıran gerçek hayat örnekleri gelir.

Kısım 1 — Araç Kutusu: NumPy/pandas • gerçek veri setlerini yükleme • keşifsel veri analizi • eksik veri • veri tipleri ve ölçek düzeyleri.

Kısım 2 — Betimsel İstatistik: merkezi eğilim • yayılım • çeyrekler & aykırı değerler • çarpıklık/basıklık • korelasyon • görselleştirme • Simpson paradoksu.

Kısım 3 — Olasılık: aksiyomlar • koşullu olasılık • Bayes teoremi • kombinatorik • rastgele değişkenler • beklenen değer & varyans • eşbileşik dağılımlar.

Kısım 4 — Dağılımlar: Bernoulli, binom, Poisson, geometrik, hipergeometrik • normal, üstel, gamma, beta, t , χ^2 , F • BSY & Merkezi Limit Teoremi • Monte Carlo.

Kısım 5 — Çıkarım: örnekleme dağılımları • MLE & momentler yöntemi • yanlışlık-varyans • güven aralıkları • bootstrap.

Kısım 6 — Hipotez Testleri: p -değeri • t -testleri • ANOVA • χ^2 • parametrik olmayan testler • permütasyon • güç analizi • çoklu karşılaştırma • Bayesçi çıkarım.

Kısım 7 — Regresyon: OLS türetimi • statsmodels çıktısını okuma • varsayım tanımları • VIF • lojistik & Poisson regresyon • Ridge/Lasso.

Kısım 8 — ML Temelleri: eğitim/test • çapraz doğrulama • veri sızıntısı • yanlışlık-varyans • Pipeline • ön işleme • metrikler.

Kısım 9 — Denetimli Öğrenme: k -NN • Naive Bayes • SVM • karar ağaçları • Random Forest • Gradient Boosting • stacking • hiperparametre ayarı.

Kısım 10 — Denetimsiz Öğrenme: K-Means • hiyerarşik kümeleme • DBSCAN • GMM • PCA • t-SNE • anomali tespiti.

Kısım 11 — İleri Konular: dengesiz veri • model yorumlama (permütasyon önemi, SHAP, PDP) • kalibrasyon • zaman serileri • sinir ağları.

Kısım 12 — Projeler: Titanic • California Housing • Breast Cancer • müşteri segmentasyonu • A/B testi.

Kısım 13 — Referans: dağılım tablosu • test seçme akışı • scikit-learn sözlüğü • sık yapılan hatalar.

Hazırlanma Tarihi: 9 Eylül 2026

Anlatım Türkçe • Kod ve terimler İngilizce • Tüm çıktılar gerçek çalıştırmalardan

Contents

1 Bu Rehber Nasıl Okunur?	5
1.1 Renkli Kutuları Tanıym	5
2 Kurulum ve Kütüphaneler	6
2.1 Rehber Boyunca Kullanılan Ortak Başlık	6
3 Gerçek Veri Setlerini Yükleme	7
4 İlk Bakış: Keşifsel Veri Analizi (EDA)	8
4.1 1–3. Adım: Yapı	8
4.2 4. Adım: Eksik Veri	9
4.3 5. Adım: Sayısal Özet	9
4.4 6. Adım: Gruplara Ayırmak	10
4.5 Ölçek Düzeyleri: Hangi İstatistik Anlamlı?	10
5 Merkezî Eğilim Ölçüleri	12
6 Yayılım Ölçüleri	14
7 Dağılımın Şekli: Çarpıklık ve Basıklık	17
8 Aykırı Değerler	18
9 İki Değişken Arasındaki İlişki: Korelasyon	20
9.1 Simpson Paradoksu: Gerçek Bir Örnek	22
9.2 Anscombe Dörtlüsü: Neden Grafik Çizmelişin?	22
10 Temel Kavramlar ve Aksiyomlar	24
11 Koşullu Olasılık: Titanic Üzerinde	25
11.1 Bayes Teoremi	26
11.2 Klasik Tuzak: Tıbbi Tarama Testi	28
11.3 Kontenjans Tablosunda Bağımsızlık	28
11.4 Koşullamaya Devam: Üçüncü Değişken	28
12 Kombinatorik ve Simülasyon	29
12.1 Doğum Günü Problemi	30
12.2 Monty Hall: Simülasyonla İnanmak	30
13 Rastgele Değişkenler	31
13.1 Eşbileşik, Marjinal ve Koşullu Dağılımlar	33
14 scipy.stats Arayüzü	36
15 Kesikli Dağılımlar	37
16 Sürekli Dağılımlar	40
16.1 Gerçek Veriye Dağılım Uydurmak	41
16.2 Çarpık Veri: Log-normal Dönüşüm	43
17 Büyük Sayılar Yasası ve Merkezi Limit Teoremi	43

18 Monte Carlo ve Örneklem Yöntemleri	46
18.1 Ters Dönüşüm ve Kabul-Ret Yöntemleri	47
19 Örneklem Dağılımı ve Standart Hata	48
20 Nokta Tahmini: MLE ve Momentler Yöntemi	49
20.1 İyi Bir Tahmin Edici Nedir?	51
21 Güven Aralıkları	52
21.1 Oran için Güven Aralığı	53
22 Bootstrap: Formülsüz Çıkarım	54
23 Örneklem Büyüklüğü Belirleme	56
24 Mantık: H_0, p-değeri ve Hatalar	58
25 t-Testleri	59
25.1 Tek Örneklem t -Testi	60
25.2 Bağımsız İki Örneklem t -Testi	61
25.3 Etki Büyüklüğü: p -değerinden Daha Önemli	62
25.4 Eşleştirilmiş t -Testi	63
26 ANOVA: İkidenden Fazla Grup	64
26.1 Post-hoc: Hangi Gruplar Farklı?	65
26.2 İki Yönlü ANOVA ve Etkileşim	65
27 Kategorik Veri: Ki-Kare Testleri	66
28 Parametrik Olmayan Testler ve Permütasyon	68
28.1 Permütasyon Testi: Varsayımsız Çıkarım	69
29 Güç Analizi ve Çoklu Karşılaştırma	70
29.1 Çoklu Karşılaştırma Düzeltmesi	72
30 Bayesçi Çıkarıma Giriş	73
31 Basit Doğrusal Regresyon	76
31.1 <code>summary()</code> Çıktısını Okumak	78
32 Çoklu Regresyon	78
32.1 Kukla Değişkenler, Etkileşim ve Polinom	80
33 Varsayımlar ve Tanı Testleri	80
33.1 Çoklu Doğrusal Bağlantı (Multicollinearity)	81
33.2 Sağlam (Robust) Standart Hatalar	82
34 Lojistik Regresyon	83
35 Genelleştirilmiş Doğrusal Modeller (GLM)	85
36 Problem Tipleri ve İş Akışı	88
37 Eğitim/Test Ayrımı ve Çapraz Doğrulama	88
37.1 Çapraz Doğrulama	89

38 Veri Sızıntısı: En Pahalı Hata	91
39 Yanlılık–Varyans Dengesi	92
39.1 Öğrenme Eğrileri: Daha Fazla Veri İşe Yarar mı?	94
40 Ön İşleme ve Pipeline	95
41 Değerlendirme Metrikleri	95
41.1 Sınıflandırma	96
41.2 Regresyon	97
42 Algoritma Haritası	100
42.1 Hepsini Aynı Veride Karşılaştırmak	100
43 k-En Yakın Komşu	101
44 Karar Ağaçları	103
45 Destek Vektör Makineleri (SVM)	104
46 Naive Bayes ve Metin Sınıflandırma	106
47 Öznitelik Seçimi	108
48 Topluluk Yöntemleri (Ensembles)	109
48.1 Voting ve Stacking	111
49 Hiperparametre Ayarı	112
49.1 Boosting Kütüphaneleri	113
50 K-Means Kümeleme	114
51 Diğer Kümeleme Yöntemleri	116
52 Boyut İndirgeme: PCA	117
52.1 t-SNE ve UMAP: Görselleştirme İçin	119
53 Anomali Tespiti	120
54 Dengesiz Veri	121
54.1 Eşik Ayarı: Bedava Kazanç	122
55 Model Yorumlama	123
55.1 Permütasyon Önemi	123
55.2 Kısmi Bağımlılık ve SHAP	125
56 Olasılık Kalibrasyonu	125
57 Zaman Serisi Verisi	126
58 Sinir Ağlarına Giriş	128

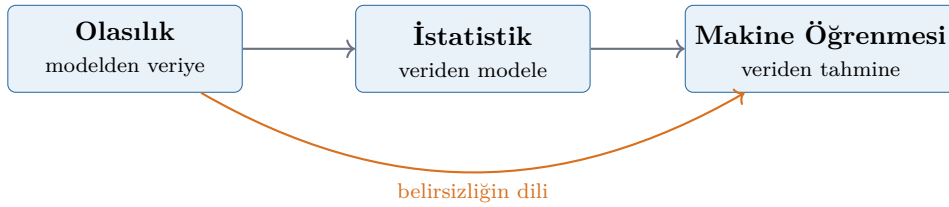
59 Proje 1 — Titanic: Sınıflandırma Hattı	131
59.1 Adım 1: Keşif	131
59.2 Adım 2: Öznitelik Mühendisliği ve Pipeline	131
59.3 Adım 3: Model Karşılaştırma	132
59.4 Adım 4: Ayar ve Nihai Test	132
60 Proje 2 — California Housing: Regresyon	133
61 Proje 3 — Meme Kanseri: Eşik ve Maliyet Analizi	135
62 Proje 4 — Bölge Segmentasyonu (Kümeleme)	136
63 Proje 5 — A/B Testi: Tasarımdan Karara	137
63.1 Adım 1: Tasarım — Örneklem Büyüklüğü	137
63.2 Adım 2: Analiz	137
63.3 Adım 3: Bayeşçi Bakış — Karar İçin Daha Kullanışlı	137
64 Dağılım Tablosu	139
65 Hangi Testi Seçmeliyim?	140
66 scikit-learn Sözlüğü	140
67 Hızlı Tarif Kitabı	141
68 Sık Yapılan Hatalar	142
69 Genel Bakış: Üç Alan Nasıl Bağlanıyor?	142

KISIM 1

Araç Kutusu ve Gerçek Veri Setleri

1. Bu Rehber Nasıl Okunur?

Üç konu birbirinin devamıdır ve bu sırayla öğrenilmelidir:



Alan	Soru	Tipik cevap
Olasılık	Model biliniyorsa veri nasıl davranır?	"Hilesiz zarla 10 atışta 3 kez altı gelme olasılığı 0,155."
İstatistik	Veri biliniyorsa model ne olabilir?	"Zar hileli görünüyor; $p = 0,03$, H_0 reddedilir."
Makine öğrenmesi	Yeni veri için ne tahmin edeyim?	"Bu müşteri %78 olasılıkla ayrılacak."

Dikkat

Makine öğrenmesini istatistik bilmeden öğrenmek, bir aracı direksiyon dışındaki hiçbir parçasını bilmeden kullanmaya benzer: düz yolda gider ama ilk virajda ne olduğunu anlamazsın. `model.fit()` çağırmak kolaydır; sonucun *anlamlı* olup olmadığını anlamak istatistiktir.

1.1 Renkli Kutuları Tanıyın

Bu rehberde hiçbir formül açıklamasız bırakılmamıştır. Her önemli denklemin ardından, *o denklemin ne olduğunu, niçin öyle kurulduğunu ve gerçek hayatta nerede karşınıza çıkacağını* anlatan kutular gelir. Renklere alışsanız belgeyi çok daha hızlı tarayabilirsiniz:

Kutu	İçeriği
Formül / Tanım / Teorem	Matematiksel ifadenin kendisi.
Nedir? Niçin? Nerede?	Formülün sezgisel açıklaması, niçin o biçimde kurulduğu ve hangi alanlarda kullanıldığı. Bir denklemini anlamadıysanız önce bu kutuyu okuyun.
Gerçek Hayattan	Konunun somut bir olayla, kararlar veya vakayla pekiştirilmesi.
Bilgi	Çıktının yorumu, ek bağlam, ileriye yapılan bağlantı.
İpucu	Pratik öneri, kısayol, varsayılan tercih.
Dikkat	Sık yapılan hata, tuzak, yanlış yorum.
Soru	Kendi kendine çözmek için problem (çözümü hemen altında).

İpucu

Önerilen okuma sırası: önce *Nedir? Niçin? Nerede?* kutusunu oku, sonra formüle dön — denklemin çoğu zaman ikinci okuyuşta kendiliğinden anlaşılır hâle gelir. Kodu ise en son çalıştır; kod, anladığın şeyi *doğrulamak* içindir, öğrenmek için değil.

2. Kurulum ve Kütüphaneler

```
$ python3 -m venv ds && source ds/bin/activate
$ pip install numpy pandas scipy matplotlib seaborn
$ pip install scikit-learn statsmodels
$ pip install jupyterlab # interaktif calisma
# Istege bagli, ileri konular icin:
$ pip install xgboost lightgbm shap imbalanced-learn optuna
```

Kütüphane	Rolü
numpy	Sayısal diziler, rastgele sayı üretimi, doğrusal cebir.
pandas	Etiketli tablo verisi; okuma, temizleme, gruplama.
scipy.stats	Olasılık dağılımları ve klasik hipotez testleri.
statsmodels	İstatistiksel modelleme: OLS, GLM, ANOVA, zaman serileri, tam testleri.
scikit-learn	Makine öğrenmesi: ön işleme, modeller, model seçimi, metrikler.
matplotlib / seaborn	Görselleştirme; seaborn istatistiksel grafiklerde pratiktir.

Bilgi

scipy.stats ile statsmodels sık karıştırılır. Kaba ayırım: *tek bir test* yapacaksan scipy.stats (ttest_ind, chi2_contingency); *model kurup katsayı, standart hata, güven aralığı ve tanı istatistikleri* istiyorsan statsmodels. Tahmin performansı peşindeysen scikit-learn.

2.1 Rehber Boyunca Kullanılan Ortak Başlık

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from scipy import stats as st
6 import statsmodels.api as sm
7 import statsmodels.formula.api as smf
8
9 # Tekrarlanabilirlik: TUM rastgelelik tek bir tohumdan
10 RNG = np.random.default_rng(42)
11 SEED = 42
12
13 pd.set_option('display.width', 100)
14 pd.set_option('display.max_columns', 20)
15 sns.set_theme(style='whitegrid', palette='deep')

```

Dikkat

`np.random.seed(42)` eski (legacy) arayüzdür ve küresel durumu değiştirir; kütüphaneler arasında sızıntı yapar. Modern yol `rng = np.random.default_rng(42)` ile *yerel* bir üreteç kullanmaktır. `scikit-learn`'de ayrıca her fonksiyona `random_state=42` vermelisin — yoksa sonuçların her çalıştırmada değişir ve karşılaştırmaların anlamsızlaşır.

3. Gerçek Veri Setlerini Yükleme

Bu rehberdeki her örnek **gerçek** bir veri setiyle çalışır. Üç kaynak kullanacağız; hepsi tek satırla gelir.

Veri seti	Satır	Sütun	Ne için kullanacağız?
penguins	344	7	Betimsel istatistik, gruplar arası karşılaştırma, <i>t</i> -testi.
titanic	891	15	Eksik veri, ki-kare, lojistik regresyon, sınıflandırma.
tips	244	7	Regresyon, ANOVA, etkileşim etkileri.
iris	150	4	Çok sınıflı sınıflandırma, kümeleme, PCA.
breast_cancer	569	30	Tıbbi tanı, ROC, eşik ve maliyet analizi.
california_housing	20640	8	Regresyon, ağaç toplulukları, öğrenme eğrileri.
wine	178	13	Kümeleme, boyut indirgeme.
digits	1797	64	Sinir ağları, t-SNE, çok sınıflı metrikler.
diabetes	442	10	Ridge/Lasso, düzenleme.

```

1 # 1) seaborn: internetten indirir, bir kez indirince onbelleğe alınır
2 import seaborn as sns
3 peng = sns.load_dataset('penguins') # Palmer Penguins (2020)
4 titanic = sns.load_dataset('titanic') # Titanic yolcu listesi
5 tips = sns.load_dataset('tips') # Restoran bahsis kayıtları
6
7 # 2) scikit-learn: paketle birlikte gelir, internet gerekmez
8 from sklearn.datasets import (load_iris, load_breast_cancer, load_wine,
9                               load_digits, load_diabetes)

```

```

10 iris = load_iris(as_frame=True)                # as_frame=True -> pandas DataFrame
11 X, y = iris.data, iris.target
12
13 # 3) Büyük veri setleri: ilk çağrıda indirilir (~2 MB)
14 from sklearn.datasets import fetch_california_housing
15 cal = fetch_california_housing(as_frame=True)
16
17 # 4) OpenML'den herhangi bir veri seti
18 from sklearn.datasets import fetch_openml
19 adult = fetch_openml('adult', version=2, as_frame=True, parser='auto')

```

Palmer Penguins Hakkında

2007–2009 yılları arasında Antarktika'daki Palmer İstasyonu'nda üç penguen türünden (*Adelie*, *Gentoo*, *Chinstrap*) 344 bireyin gaga uzunluğu, gaga derinliği, kanat uzunluğu ve vücut kütlesi ölçülmüştür. Iris veri setinin modern ve daha ilginç alternatifi olarak kabul edilir: gerçek eksik değerler, gerçek gruplar arası farklar ve gerçek bir Simpson paradoksu örneği içerir.

4. İlk Bakış: Keşifsel Veri Analizi (EDA)

Herhangi bir model kurmadan önce veriyi *tanımak* zorundasın. Aşağıdaki altı adım, her yeni veri setinde ilk yapılacak iştir.

```

1 import seaborn as sns
2 peng = sns.load_dataset('penguins')
3
4 peng.head()           # 1) İlk satırlar: sütunlar ne anlama geliyor?
5 peng.shape            # 2) Boyut
6 peng.dtypes           # 3) Veri tipleri
7 peng.isna().sum()     # 4) Eksik değerler
8 peng.describe()       # 5) Sayısal özet
9 peng['species'].value_counts() # 6) Kategorik dağılım

```

4.1 1–3. Adım: Yapı

```

1 print(peng.head())
2 print(peng.shape)
3 print(peng.dtypes)

```

```

  species  island  bill_length_mm  ...  flipper_length_mm  body_mass_g  sex
0  Adelie  Torgersen           39.1  ...             181.0       3750.0  Male
1  Adelie  Torgersen           39.5  ...             186.0       3800.0  Female
2  Adelie  Torgersen           40.3  ...             195.0       3250.0  Female
3  Adelie  Torgersen           NaN  ...              NaN          NaN    NaN
4  Adelie  Torgersen           36.7  ...             193.0       3450.0  Female

[5 rows x 7 columns]

(344, 7)

species          str
island            str
bill_length_mm    float64
bill_depth_mm     float64
flipper_length_mm float64

```

```
body_mass_g    float64
sex            str
dtype: object
```

Daha ilk satırlarda (indeks 3) bir **eksik değer** satırı görüyoruz. Bu tesadüf değil: gerçek veri her zaman kirlidir.

4.2 4. Adım: Eksik Veri

```
1 print(peng.isna().sum())
2 print(f"\nEksik içeren satir orani: {peng.isna().any(axis=1).mean():.1%}")
```

```
species        0
island         0
bill_length_mm 2
bill_depth_mm  2
flipper_length_mm 2
body_mass_g     2
sex            11
dtype: int64

Eksik içeren satir orani: 3.2%
```

Eksiklik türü Anlamı ve ne yapmalı?

MCAR	<i>Tamamen rastgele:</i> eksiklik hiçbir şeye bağlı değil. Satırı silmek yanlışlık yaratmaz (ama veri kaybettirir).
MAR	<i>Gözlenene bağlı rastgele:</i> eksiklik diğer sütunlarla açıklanabilir. Model tabanlı doldurma (imputation) uygundur.
MNAR	<i>Rastgele değil:</i> eksikliğin nedeni eksik değer kendisidir (örn. yüksek gelirli gelirini söylemez). En tehlikelisi; eksiklik göstergesi ekle ve dikkatli yorumla.

```
1 # Eksik veriyle bas etme secenekleri
2 peng.dropna() # sil (dikkatli kullan)
3 peng.fillna(peng.median(numeric_only=True)) # medyanla doldur
4 peng['sex'].fillna(peng['sex'].mode()[0]) # kategorikte mod
5
6 # Gruba gore doldurma: cok daha akillica
7 peng['body_mass_g'] = peng.groupby('species')['body_mass_g'] \
8     .transform(lambda s: s.fillna(s.median()))
9
10 # Model tabanlı doldurma (scikit-learn)
11 from sklearn.impute import SimpleImputer, KNNImputer
12 sayisal = peng.select_dtypes('number')
13 dolu = KNNImputer(n_neighbors=5).fit_transform(sayisal)
```

Dikkat

Doldurmayı (imputation) **eğitim/test ayrımından önce** yapmak *veri sızıntısıdır*: test setinin bilgisi eğitim istatistiklerine karışır ve model olduğundan iyi görünür. Doğru yol, doldurmayı Pipeline içine koymaktır (Kısım 8).

4.3 5. Adım: Sayısal Özet

```
1 print(peng.describe().round(2))
```

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
count	342.00	342.00	342.00	342.00
mean	43.92	17.15	200.92	4201.75
std	5.46	1.97	14.06	801.95
min	32.10	13.10	172.00	2700.00
25%	39.22	15.60	190.00	3550.00
50%	44.45	17.30	197.00	4050.00
75%	48.50	18.70	213.00	4750.00
max	59.60	21.50	231.00	6300.00

Ortalama vücut kütlesi 4201,75 g, standart sapma 801,95 g. Ama bu sayılar *yanaltıcıdır*: veri üç türden oluşuyor ve türler birbirinden çok farklı.

4.4 6. Adım: Gruplara Ayırmak

```
1 print(peng['species'].value_counts())
2 print(peng.groupby('species')['body_mass_g'].agg(['count', 'mean', 'std']).round(1))
```

```
species
Adelie      152
Gentoo      124
Chinstrap   68
Name: count, dtype: int64

      count    mean    std
species
Adelie    151  3700.7  458.6
Chinstrap   68  3733.1  384.3
Gentoo     123  5076.0  504.1
```

Bilgi

Genel ortalama (4202 g) hiçbir pengueni temsil etmiyor: Adelie ve Chinstrap ~ 3700 g, Gentoo ise ~ 5076 g. Ayrıca grup içi standart sapmalar (~ 400 – 500 g), genel standart sapmadan (802 g) çok daha küçük — yani değişkenliğin büyük kısmı *gruplar arasındadır*. Bu gözlem, Kısım 6'daki ANOVA'nın tam olarak ölçtüğü şeydir.

4.5 Ölçek Düzeyleri: Hangi İstatistik Anlamlı?

Bir sütuna hangi işlemi uygulayabileceğini *ölçek düzeyi* belirler.

Ölçek	Örnek	Anlamlı olan
Sınıflayıcı (nominal)	species, island	Mod, frekans, ki-kare.
Sıralayıcı (ordinal)	Eğitim düzeyi, memnuniyet (1–5)	Medyan, çeyrekler, sıra temelli testler.
Aralık (interval)	Santigrat sıcaklık, IQ	Ortalama, standart sapma; oran anlamsız (20°C, 10°C'nin iki katı değil).
Oran (ratio)	body_mass_g, gelir, süre	Hepsi; gerçek sıfır vardır, oran anlamlıdır.

Dikkat

En sık yapılan hata, sınıflayıcı bir değişkeni sayı olarak kodlayıp ortalamasını almaktır. `island` sütununu $\{0, 1, 2\}$ ile kodlarsan “ortalama ada 1,3” gibi anlamsız bir sonuç çıkar — ve daha kötüsü, model bu sahte sıralamayı öğrenir. Doğrusu *one-hot encoding*’dir (Kısım 8).

KISIM 2

Betimsel İstatistik

5. Merkezî Eğilim Ölçüleri

Hangi Soruyu Cevaplıyor?

Elinde 333 penguinin ağırlığı var — 333 sayı. Birine “bu penguinler ne kadar ağır?” diye tek cümleyle cevap vermen gerekse ne söylersin? İşte merkezî eğilim ölçüleri tam bu işi yapar: **bir veri yığını tek bir temsilci sayıya indirger**. Bu sayı, dağılımın “nerede durduğunu” söyler. **Niçin birden fazla ölçü var?** Çünkü “temsilci” kelimesinin tek bir doğru anlamı yok. Ortalama *adil paylaşırma* sorusunu cevaplar (pastayı eşit bölsek herkese ne düşer?), medyan *tipik birey* sorusunu cevaplar (sıraya dizsek ortadaki kim?), mod ise *en sık karşılaşılan* sorusunu. Aynı veride bu üç cevap birbirinden çok farklı çıkabilir — ve hangisini raporladığın, karar vericinin ne düşüneceğini doğrudan belirler.

Aritmetik Ortalama

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Verinin “ağırlık merkezi”dir: $\sum(x_i - \bar{x}) = 0$. Her gözlemi kullandığı için bilgiyi en verimli özetler, ama tam bu yüzden **aykırı değerlere duyarlıdır**.

Nedir? Niçin? Nerede?

Ne yapıyor? Formül aslında iki adımdan ibarettir: bütün değerleri topla ($\sum x_i$), sonra kişi sayısına böl ($/n$). Yani “*hepsini bir havuzda toplayıp eşit paylaştırsak kişi başına ne düşer?*” sorusunun cevabı. Fizikteki karşılığı **ağırlık merkezidir**: veri noktalarını bir cetvele koyduğun ağırlıklar gibi düşün; cetveli $\bar{x}$ noktasından parmağınla desteklersen tam dengede durur. Bu yüzden $\sum(x_i - \bar{x}) = 0$ ’dır — soldaki eksiler sağdaki artıları tam götürür.

Niçin bu kadar merkezi bir rol oynuyor? Üç sebeple: (1) Her gözlemi hesaba katar, hiçbir bilgiyi çöpe atmaz. (2) Matematiksel olarak uysaldır — toplamların ortalaması ortalamaların toplamıdır, türevi alınabilir, Merkezî Limit Teoremi doğrudan onun hakkındadır. (3) Karesel hatayı en küçükleyen sabittir: $\min_c \sum(x_i - c)^2$ probleminin çözümü tam olarak $c = \bar{x}$ ’tir. Bu son madde, ilerideki *doğrusal regresyonun* niçin ortalamalar etrafında kurulduğunu açıklar.

Nerede karşına çıkar? Fabrikada bir vardiyanın ortalama üretimi; sunucunun ortalama yanıt süresi; öğrencinin not ortalaması; A/B testinde iki grubun ortalama tıklama oranı; makine öğrenmesinde StandardScaler’ın çıkardığı merkez; sinir ağı eğitiminde minimize edilen ortalama kayıp (*mean loss*). Kısacası: veriden *tek bir sayı* çıkaran hemen her yerde.

Ölçü	Formül	Ne zaman?
Ortalama	$\bar{x} = \frac{1}{n} \sum x_i$	Simetrik dağılım, aykırı değer yok.
Medyan	Q_2	Çarpık dağılım, aykırı değer var (gelir, fiyat).
Mod	$\arg \max f(x)$	Kategorik veri; çok tepeli dağılımlar.
Budanmış ortalama	uç %10 atılır	Aykırı değere dayanıklı ama ortalamaya yakın.
Geometrik ortalama	$(\prod x_i)^{1/n}$	Oranlar, büyüme hızları, getiriler.
Harmonik ortalama	$n / \sum (1/x_i)$	Hızlar, oranların ortalaması (örn. F1 skoru).

```

1 import seaborn as sns
2 from scipy import stats as st
3
4 peng = sns.load_dataset('penguins').dropna()
5 x = peng['body_mass_g']
6
7 print(f"mean          = {x.mean():.1f}")
8 print(f"median        = {x.median():.1f}")
9 print(f"mode           = {x.mode().iloc[0]:.1f}")
10 print(f"trimmed(10%)    = {st.trim_mean(x, 0.1):.1f}")
11 print(f"geometric       = {st.gmean(x):.1f}")
12 print(f"harmonic        = {st.hmean(x):.1f}")

```

```

mean          = 4207.1
median        = 4050.0
mode          = 3800.0
trimmed(10%)  = 4159.5
geometric     = 4132.6
harmonic      = 4061.1

```

Geometrik ve Harmonik Ortalama Niçin Var?

Bu ikisi “egzotik” değil, *yanlış aritmetik ortalama alanların düştüğü tuzağın* çözümüdür.

Geometrik ortalama — çarpımsal büyümeler için. Bir yatırım fonu üç yılda %+100, %−50, %+100 getirdi. Aritmetik ortalama $(100 - 50 + 100)/3 = +\%50$ der — “yılda ortalama %50 kazandın!” Oysa 100 TL şöyle yol alır: $100 \rightarrow 200 \rightarrow 100 \rightarrow 200$. Üç yılda toplam kazanç %100, yani yıllık gerçek getiri $\sqrt[3]{2} - 1 \approx \%26$. Geometrik ortalama $\sqrt[3]{2,0 \times 0,5 \times 2,0} = 1,26$ tam bunu verir. Kural: **değerler birbirine çarpılarak birikiyorsa** (getiri, enflasyon, nüfus artışı, bakteri çoğalması, bileşik faiz) geometrik ortalama kullanılır.

Harmonik ortalama — oranların ortalaması için. Ankara’ya 100 km’yi 100 km/sa, dönüşü 50 km/sa hızla gittin. Ortalama hızın $(100 + 50)/2 = 75$ değildir! Gidiş 1 saat, dönüş 2 saat, toplam 200 km / 3 saat = 66,7 km/sa. Harmonik ortalama $2/(1/100 + 1/50) = 66,7$ tam bunu verir. Kural: **birim “bir şey başına bir şey” ise** (km/sa, TL/adet, iş/saat) ve paydalar sabitse harmonik ortalama doğrudur.

Makine öğrenmesindeki en ünlü kullanımı: F1 skoru, precision ve recall’ın *harmonik* ortalamasıdır. Niçin aritmetik değil? Çünkü harmonik ortalama küçük olan değere yapışır: precision = 1,0, recall = 0,02 olan bir model aritmetikte 0,51 alır (“yarı yarıya iyi” görünür), harmonikte ise 0,039 alır. Yani F1, “bir tarafı berbatsa model berbattır” demenin matematiksel yoludur.

Aynı Veri, Üç Farklı Manşet

2023'te bir ülkede maaş verisi açıklandı. Aynı veri setinden üç farklı haber çıktı ve üçü de yalan söylemiyordu:

- “Ortalama maaş 52.000 TL” — *aritmetik ortalama*. Birkaç üst düzey yönetici maaşı bu sayıyı tek başına yukarı çekiyor.
- “Çalışanların yarısı 31.000 TL'nin altında kazanıyor” — *medyan*. Tipik çalışanın gerçek durumu budur.
- “En yaygın maaş asgari ücret” — *mod*. En kalabalık grubun nerede yığıldığını söyler.

Hangi ölçüyü seçtiğin bir teknik detay değil, **bir anlatı tercihidir**. Bu yüzden gelir, kira, sağlık harcaması, müşteri yaşam boyu değeri gibi çarpık büyüklüklerde ciddi raporlar *her üçünü birden* verir.

Bilgi

Sıralama her zaman $harmonik \leq geometrik \leq aritmetik$ ortalamadır (eşitlik ancak tüm değerler aynıysa). Ayrıca ortalama $>$ medyan olması dağılımın **sağa çarpık** olduğunun işaretidir — burada tam olarak bu oluyor, çünkü ağır Gentoo penguenleri sağ kuyruğu çekiyor.

Neden Medyan?

Bir mahallede 10 kişinin yıllık geliri ortalama 400 bin TL olsun. Aralarına geliri 100 milyon TL olan biri taşınırsa ortalama $\approx 9,5$ milyona fırlar ama *kimsenin* durumu değişmez. Medyan ise neredeyse aynı kalır. Bu yüzden gelir, ev fiyatı, tepki süresi gibi çarpık büyüklüklerde daima medyan raporlanır.

6. Yayılım Ölçüleri

Hangi Soruyu Cevaplıyor?

İki restoranın da servis süresi ortalama 20 dakika. Birincisinde her sipariş 19–21 dakika arasında geliyor; ikincisinde kimi 5 dakikada, kimi 45 dakikada. Ortalama ikisini de aynı gösteriyor — oysa müşteri deneyimi taban tabana zıt. Aradaki farkı **yayılım ölçüleri** yakalar: veri merkezin etrafında ne kadar dağınık?

Yayılım, uygulamada çoğu zaman ortalamadan *daha* önemlidir; çünkü **yayılım = belirsizlik = risk**. Finansta “volatilité” dediğimiz şey getirilerin standart sapmasıdır. Üretimde kalite kontrolün tamamı yayılımı küçültmek üzerine kuruludur. İstatistiksel çıkarımın tamamı (güven aralığı, p -değeri, hipotez testi) “gözlediğim fark, doğal yayılımın içinde mi yoksa dışında mı?” sorusunun cevabıdır — yani yayılımı bilmeden hiçbir istatistiksel karar veremezsin.

Varyans ve Standart Sapma

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2, \quad s = \sqrt{s^2}$$

Paydadaki $n-1$ (*Bessel düzeltmesi*), örneklem varyansının anakütle varyansı σ^2 için **yansız** tahmin edici olmasını sağlar: $\mathbb{E}[s^2] = \sigma^2$.

Nedir? Niçin? Nerede?

Formülü kelimelerle okuyalım. Her gözlemin ortalamadan sapmasını al $(x_i - \bar{x})$, *karesini* al, hepsini topla, gözlem sayısına böl. Yani varyans “**ortalama karesel sapma**”dır.

Niçin karesi alınıyor? İki sebep: (1) Sapmaların kendisini toptasaydık sonuç *her zaman sıfır* çıkardı (ortalamanın ağırlık merkezi olma özelliği). Bir şekilde işaretten kurtulmak gerekiyor. (2) Mutlak değer de işaret sorununu çözerdi, ama kare almanın iki üstünlüğü var: türevlenebilir olduğu için matematiksel olarak kolay işlenir, ve büyük sapmaları orantısız biçimde cezalandırır. İkincisi bir özellik, kusur değil: 10 dakikalık bir gecikme, 1 dakikalık on gecikmeden çok daha kötüdür.

Standart sapma niçin ayrıca var? Varyansın birimi *gram karedir* — kimse “penguenlerin ağırlığı 648.372 gram kare değişiyor” diyemez. Karekökünü alınca birim gerçek dünyaya döner: “ ± 805 gram”. Yani varyans matematiğin, standart sapma insanın dilidir.

Niçin $n - 1$? Sapmaları gerçek anakütle ortalaması μ ’ye göre değil, *aynı veriden hesapladığın $\bar{x}$* ’e göre ölçüyorsun. $\bar{x}$ ise tanımı gereği kendi verisine “en yakın” noktadır — bu yüzden sapmalar sistematik olarak olduğundan küçük çıkar. Paydayı n yerine $n - 1$ yapmak bu küçülmeyi tam olarak telafi eder. Sezgisel bakış: $\bar{x}$ ’i verinin kendisinden hesapladığın an bir *serbestlik derecesini* harcamış olursun; n sapmadan yalnızca $n - 1$ tanesi bağımsızdır, sonuncusu diğerlerinden belirlenir.

Standart Sapmanın Para Ettiği Yerler

- **Finans:** Bir hisse yıllık ortalama %15 getiriyor, standart sapması %40. Bir tahvil %8 getiriyor, sapması %3. Yüksek getiri *bedavaya* gelmiyor — Sharpe oranı tam olarak “getiri / standart sapma” oranıdır, yani “aldığın risk başına kazanç”.
- **Üretim (Six Sigma):** Bir vidanın çapı 10 mm hedefleniyor, tolerans $\pm 0,3$ mm. Süreç sapması $\sigma = 0,1$ mm ise tolerans sınırı 3σ uzaklıktadır ve binde 3 hatalı üretirsin. σ ’yı 0,05’e düşürürsen sınır 6σ olur, hata oranı milyarda ikiye iner. “Six Sigma” adı buradan gelir — *ortalamayı değil, sapmayı küçültme* felsefesidir.
- **Sunucu izleme:** Ortalama yanıt süresi 200 ms olan bir API iyi görünür; ama sapma büyükse kullanıcıların %1’i 3 saniye bekliyordur ve şikâyet eden onlardır. Bu yüzden SRE ekipleri ortalamayı değil $p95/p99$ yüzdeliklerini izler.
- **Makine öğrenmesi:** StandardScaler her özneliliği $(x - \bar{x})/s$ ile dönüştürür. Amaç, “metrekare” ile “oda sayısı” gibi farklı ölçekteki değişkenleri aynı dile çevirmektir — yoksa k -NN ve SVM gibi uzaklık temelli modeller büyük ölçekli değişkene körü körüne ağırlık verir.

Dikkat

pandas varsayılan olarak $n - 1$ (ddof=1), numpy ise n (ddof=0) kullanır! `x.std()` ile `np.std(x)` farklı sonuç verir. Örneklem verisiyle çalışıyorsan ddof=1 doğrudur; bu rehberde $n = 333$ için fark küçük (805,2 vs 804,0) ama $n = 10$ için ciddidir.

Ölçü	Formül	Özelliği
Değişim aralığı	$\max - \min$	Yalnız iki gözleme bakar; çok kırılğan.
Varyans	s^2	Kare birim; matematiksel olarak kullanışlı.
Standart sapma	s	Veriyle aynı birim; yorumlanabilir.
IQR	$Q_3 - Q_1$	Ortadaki %50; aykırı değerden etkilenmez.
MAD	$\text{med } x_i - \text{med}(x) $	En dayanıklı yayılım ölçüsü.
Değişim katsayısı	$s/\bar{x}$	Birimsiz; farklı ölçekleri karşılaştırır.

```

1 import numpy as np
2 print(f"var (ddof=1) = {x.var():.1f}")
3 print(f"std (ddof=1) = {x.std():.1f}")
4 print(f"np.std      = {np.std(x):.1f}      # ddof=0!")
5 print(f"range      = {x.max() - x.min():.1f}")
6 print(f"IQR        = {x.quantile(.75) - x.quantile(.25):.1f}")
7 print(f"MAD         = {st.median_abs_deviation(x):.1f}")
8 print(f"CV          = {x.std()/x.mean():.3f}")

```

```

var (ddof=1) = 648372.5
std (ddof=1) = 805.2
np.std      = 804.0      # ddof=0!
range      = 3600.0
IQR        = 1225.0
MAD         = 600.0
CV          = 0.191

```

Standart Sapmayı Yorumlamak

Yaklaşık normal bir dağılımda **68–95–99,7 kuralı** geçerlidir:

$$\mathbb{P}(|X - \mu| < \sigma) \approx 0,68, \quad \mathbb{P}(|X - \mu| < 2\sigma) \approx 0,95, \quad \mathbb{P}(|X - \mu| < 3\sigma) \approx 0,997.$$

Dağılım normal değilse **Chebyshev eşitsizliği** her dağılım için garanti verir: $\mathbb{P}(|X - \mu| \geq k\sigma) \leq 1/k^2$.

Nedir? Niçin? Nerede?

Bu kural niçin bu kadar kullanışlı? Çünkü standart sapmayı soyut bir sayı olmaktan çıkarıp *ölçü birimi* haline getirir. “Ortalamadan 2 sapma uzakta” demek, “en uçtaki %5’e giriyor” demektir — dağılımın ölçeğinden bağımsız olarak. Bir penguenin ağırlığı da, bir öğrencinin sınav puanı da, bir sunucunun gecikmesi de aynı cetvelle konuşabilir hale gelir. $z = (x - \bar{x})/s$ dönüşümünün (*z-skoru*) tüm gücü budur.

Nerede kullanılır?

- **Kalite kontrol grafikleri:** Üretim hattında $\mu \pm 3\sigma$ “kontrol sınırı” çizilir. Bir ölçüm dışarı taşarsa alarm çalar; çünkü süreç normalse bunun olma ihtimali binde 3’tür — yani muhtemelen tesadüf değil, makinede bir şey bozulmuştur.
- **Aykırı değer tespiti:** $|z| > 3$ eşığı doğrudan bu kuraldan gelir.
- **Sınav notu standardizasyonu:** Farklı zorluktaki iki sınavın notlarını karşılaştırmanın tek adil yolu *z-skoru*dur.
- **Anomali tespiti:** Kredi kartı işlem tutarı, kullanıcının kendi geçmişine göre 4σ dışındaysa sistem işlemi askıya alır.

Chebyshev niçin ayrıca gerekli? 68–95–99,7 kuralı *yalnızca* normal dağılımda geçerlidir. Verin çarpıksa (gelir, gecikme süresi, hasar tutarı) bu sayılar tamamen yanıltır. Chebyshev eşitsizliği ise *hiçbir dağılım varsayımı yapmadan* garanti verir: hangi dağılım olursa olsun, gözlemlerin en az %75’i $\mu \pm 2\sigma$, en az %89’u $\mu \pm 3\sigma$ içinde kalmak zorundadır. Karşılığında zayıf bir garantidir (%95 yerine %75) — ama *her zaman* doğrudur. Kural: dağılımı biliyorsan normal kuralını, bilmiyorsan Chebyshev’i kullan.

7. Dağılımın Şekli: Çarpıklık ve Basıklık

Hangi Soruyu Cevaplıyor?

Ortalama dağılımın *nerede* olduğunu, standart sapma *ne kadar geniş* olduğunu söyler. Peki *şekli*? Simetrik mi, bir tarafa mı sarkıyor, uçlarda beklenenden fazla gözlem var mı? Çarpıklık ve basıklık bu iki soruyu sayısallaştırır.

Niçin umursayalım? Çünkü istatistiğin büyük kısmı sessizce “dağılım aşağı yukarı normaldir” varsayar. Çarpıklık ve basıklık, bu varsayımın ne kadar ihlal edildiğinin **erken uyarı göstergeleridir** — ve ihlal edildiğinde ortalama yanıltır, güven aralıkları hatalı çıkar, *t*-testi güvenilirliğini yitirir.

Formül

$$\text{Çarpıklık} = \mathbb{E} \left[\left(\frac{X - \mu}{\sigma} \right)^3 \right], \quad \text{Fazla basıklık} = \mathbb{E} \left[\left(\frac{X - \mu}{\sigma} \right)^4 \right] - 3.$$

Çarpıklık > 0 : sağ kuyruk uzun (ortalama $>$ medyan). Fazla basıklık > 0 : normalden kalın kuyruklu (*leptokurtic*); < 0 : basık, kuyruksuz (*platykurtic*).

Nedir? Niçin? Nerede?

Formüller ne yapıyor? İkisi de aynı fikrin farklı üsleridir: önce veriyi standartlaştır ($z = (X - \mu)/\sigma$ — böylece birimden ve ölçekten kurtul), sonra kuvvetini al ve ortalamasını hesapla.

Niçin küp (çarpıklık için)? Çünkü tek kuvvetler *işareti korur*. Ortalamanın sağındaki bir gözlem pozitif, solundaki negatif katkı yapar; küp almak büyük sapmaları abartır. Sonuç: sağ kuyruk uzunsa (birkaç çok büyük değer) toplam pozitif, sol kuyruk uzunsa negatif çıkar. Sıfır ise simetri.

Niçin dördüncü kuvvet (basıklık için)? Çift kuvvet işareti siler, yani hangi tarafta olduğu değil *ne kadar uzakta* olduğu önemlidir. Dördüncü kuvvet, uzaktaki gözlemleri o kadar şiddetle büyütür ki sonuç neredeyse tamamen *kuyruklar* tarafından belirlenir. Bu yüzden basıklık “tepe ne kadar sivri” sorusundan çok **“uçlarda ne kadar sürpriz var”** sorusunun cevabıdır. Normal dağılımın basıklığı tam olarak 3’tür; formülden 3 çıkarılmasının sebebi “normale göre” bir referans noktası kurmaktır — bu yüzden *fazla* basıklık denir.

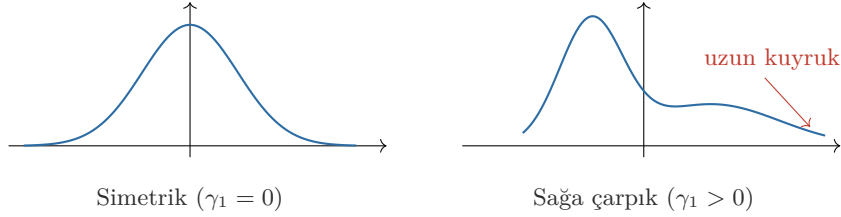
Nerede kullanılır?

- **Risk yönetimi:** Borsa getirileri normal görünür ama fazla basıklığı yüksektir (kalın kuyruk). Normal varsayan risk modelleri, 2008 gibi krizlerde “500 yılda bir görülür” dedikleri olayı arka arkaya üç gün gözlemledi. Kalın kuyruğu görmemek gerçek para kaybettirdi.
- **Model ön kontrolü:** Regresyona girmeden önce hedef değişkenin çarpıklığına bakılır; $|\gamma_1| > 1$ ise genellikle log veya Box–Cox dönüşümü uygulanır (bkz. Kısım 4).
- **Sigortacılık:** Hasar tutarları aşırı sağa çarpıktır — poliçe fiyatı ortalamaya göre değil, kuyruğa göre belirlenir.
- **Sinyal işleme:** Basıklık, bir sinyalin gürültüden mi yoksa seyrek ani darbelerden mi oluştuğunu ayırmakta kullanılır (rulman arıza tespitinin klasik göstergesidir).

Çarpıklığı Elle Hissetmek

Bir kafede 10 müşterinin hesabı: 60, 65, 70, 70, 75, 80, 80, 85, 90, 1200 TL (sonuncusu 20 kişilik bir kurumsal masa). Ortalama 287,5 TL, medyan 77,5 TL. Ortalama medyanın neredeyse dört katı — bu, **aşırı sağ çarpıklığın** tanımıdır.

Şimdi pratik sonuç: Kafe sahibi “ortalama hesabımız 287 TL” diye fiyat ve stok planlaması yaparsa iflas eder, çünkü müşterilerin %90’ı 90 TL’nin altında harcıyor. Doğru soru şudur: “*tipik* müşteri ne harcıyor?” (medyan) ve “*en iyi* müşterilerim kim?” (kuyruğu ayrı analiz et). Çarpık veride tek bir sayı asla yeterli değildir — ya medyan + yüzdelikler raporlanır, ya da dağılım log ölçekte incelenir.



```
1 for col in ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']:
2     print(f"{col:>20}: skew={peng[col].skew():+.3f} kurt={peng[col].kurt():+.3f}")
```

```
bill_length_mm: skew=+0.045 kurt=-0.883
bill_depth_mm: skew=-0.150 kurt=-0.892
flipper_length_mm: skew=+0.360 kurt=-0.961
body_mass_g: skew=+0.472 kurt=-0.733
```

Bilgi

Dört değişkenin de *fazla basıklığı* belirgin biçimde negatif ($\approx -0,9$). Bunun nedeni gerçek bir kalınlık farkı değil, verinin üç farklı türün karışımı olması: iki tepeli (bimodal) bir dağılım basık görünür. Bu, ham istatistiklere bakıp “normal değil” demeden önce *gruplara* bakmanın neden gerektiğinin güzel bir örneğidir.

8. Aykırı Değerler

Hangi Soruyu Cevaplıyor?

Aykırı değer, “diğerlerine hiç benzemeyen” gözlemdir. Onları bulmak iki zıt sebeple önemlidir: bazen **gürültüdür** (veri girişinde fazladan basılmış bir sıfır, bozuk sensör) ve analizini zehirler; bazen de **aradığın şeyin ta kendisidir** (dolandırıcılık işlemi, siber saldırı, arızalanmak üzere olan makine, nadir hastalık).

Bu yüzden aykırı değer tespiti hem bir *veri temizleme* adımı, hem de başlı başına bir *makine öğrenmesi problemi* (anomali tespiti, Kısım 11).

Yöntem	Ölçüt	Not
IQR (Tukey)	$x < Q_1 - 1,5 \text{ IQR}$ ya da $x > Q_3 + 1,5 \text{ IQR}$	Boxplot'un bıyıkları; dağılımdan bağımsız.
z-skoru	$ z > 3$	Normallik varsayar; aykırı değer kendisi s 'yi şişirir.
Düzeltilmiş z	$0,6745 \frac{x - \tilde{x}}{\text{MAD}} > 3,5$	Dayanıklı; küçük örnekleme tercih edilir.
Isolation Forest	model tabanlı	Çok değişkenli aykırılık (Kısım 10).

```

1 q1, q3 = x.quantile(.25), x.quantile(.75)
2 iqr = q3 - q1
3 lo, hi = q1 - 1.5*iqr, q3 + 1.5*iqr
4 print(f"IQR sinirlari: [{lo:.0f}, {hi:.0f}]   aykiri sayisi = {((x<lo)|(x>hi)).sum()}")
5
6 z = np.abs(st.zscore(x))
7 print(f"|z| > 3               -> {(z > 3).sum()} gozlem")
8
9 mz = 0.6745*(x - x.median())/st.median_abs_deviation(x)
10 print(f"|duzeltilmis z| > 3.5 -> {(np.abs(mz) > 3.5).sum()} gozlem")

```

```

IQR sinirlari: [1712, 6612]   aykiri sayisi = 0
|z| > 3               -> 0 gozlem
|duzeltilmis z| > 3.5 -> 0 gozlem

```

Üç Ölçüt, Üç Farklı Felsefe

IQR (Tukey) kuralı — niçin 1,5? Bu sayı keyfi görünür ama değildir: veri normal dağılımlıysa $Q_1 - 1,5 \text{ IQR}$ ile $Q_3 + 1,5 \text{ IQR}$ aralığı yaklaşık $\mu \pm 2,7\sigma$ 'ya karşılık gelir ve gözlemlerin %99,3'ünü kapsar. Yani Tukey, “normal veride yüzde birden az yanlış alarm ver” diye ayarlamıştır. En büyük üstünlüğü: Q_1 , Q_3 ve IQR aykırı değerlerden *etkilenmez*, dolayısıyla ölçüt kendi kendini sabote etmez. Boxplot'un bıyıkları tam olarak budur — her boxplot'a baktığında aslında bu kuralı görüyorsun.

z -skoru — niçin tehlikeli? $|z| > 3$ ölçütü hem normallik varsayar hem de bir kısır döngü içerir: aykırı değer kendisi $\bar{x}$ 'i kaydırır ve s 'yi şişirir, bu da z 'sini küçültür. Sonuç: **maskeleme**. 1, 2, 3, 4, 1000 dizisinde 1000'in z -skoru yalnızca 1,79'dur — eşiği geçemez! Küçük örnekleme z -skoru bir aykırı değeri asla 3'ün üstüne çıkaramaz; matematiksel üst sınır $(n - 1)/\sqrt{n}$ 'dir.

Düzeltilmiş z (MAD tabanlı) — niçin en güvenlisi? $\bar{x}$ yerine medyanı, s yerine MAD'ı kullanır; ikisi de aykırı değerlerden etkilenmez, bu yüzden maskeleme olmaz. 0,6745 katsayısı, normal dağılımda MAD'ı standart sapmayla aynı ölçeğe getiren dönüşüm sabitidir — böylece eşik yine “ σ cinsinden” okunabilir. Aynı 1, 2, 3, 4, 1000 dizisinde düzeltilmiş z skoru 337 çıkar; yani problemi anında yakalar.

Nerede hangisi? Keşif ve görselleştirmede IQR (boxplot ile aynı dili konuşur); büyük ve yaklaşık normal veride z ; küçük, kirli veya kirliliğinden şüphelendiğin veride MAD tabanlı düzeltilmiş z ; birden çok değişkeni aynı anda değerlendirmek gerektiğinde model tabanlı yöntemler (Isolation Forest, Mahalanobis uzaklığı).

Ozon Deliğini 9 Yıl Geciktiren Aykırı Değer Filtresi

1970’lerde NASA’nın uydu verilerini işleyen yazılım, Antarktika üzerinde ölçülen çok düşük ozon değerlerini “fiziksel olarak imkânsız, demek ki sensör hatası” diyerek otomatik olarak eliyordu. Ozon deliği 1985’te ancak yer istasyonundan elle yapılan ölçümlerle keşfedildi; geri dönüp filtrelenen ham veriye bakıldığında deliğin yıllardır orada olduğu görüldü.

Ders şu: **aykırı değeri otomatik silen her boru hattı, aynı zamanda keşif yapma ihtimalini de siliyordur.** Doğru refleks silmek değil, *işaretlemek* ve incelemektir — ve sildiğin her gözlemi raporunda belirtmektir.

Dikkat

Aykırı değer bulmak, onu *silmek* demek değildir. Üç soruyu sor: (1) Ölçüm hatası mı? (2 metre boyunda bir penguen → sil.) (2) Farklı bir popülasyondan mı geliyor? (→ ayrı modelle.) (3) Gerçek ve önemli mi? (Dolandırıcılık tespitinde *aykırı değerler zaten aradığın şeydir.*) Sessizce silmek, en ilginç bulguyu çöpe atmanın en hızlı yoludur.

9. İki Değişken Arasındaki İlişki: Korelasyon

Hangi Soruyu Cevaplıyor?

Buraya kadar her değişkene tek tek baktık. Ama gerçek sorular neredeyse hep *ilişki* sorularıdır: Reklam harcaması artınca satış artıyor mu? Uyku süresi sınav notunu etkiliyor mu? Hangi iki sensör aslında aynı şeyi ölçüyor? Korelasyon, iki değişkenin **birlikte hareket edip etmediğini** tek sayıya indirir.

Bu, tüm rehberin geri kalanının kapısıdır: regresyon “bu ilişkiyi bir denklemle yaz” der, öznitelik seçimi “hangi değişken hedefle ilişkili?” diye sorar, PCA “birbiriyle ilişkili değişkenleri birleştir” der. Hepsinin altında korelasyon vardır.

Pearson Korelasyonu

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2} \sqrt{\sum (y_i - \bar{y})^2}} = \frac{\text{Cov}(x, y)}{s_x s_y} \in [-1, 1]$$

Doğrusal ilişkinin gücünü ölçer. r^2 , bir değişkenin diğerindeki değişkenliğin ne kadarını açıkladığını verir.

Nedir? Niçin? Nerede?

Formülü parçalayalım. Pay $\sum (x_i - \bar{x})(y_i - \bar{y})$ *kovaryanstır* ve bütün fikri o taşır: her gözlem için “ x kendi ortalamasının hangi tarafında?” ve “ y kendi ortalamasının hangi tarafında?” diye sorar, ikisini çarpar. Her ikisi de üstteyse çarpım pozitif, her ikisi de alttıysa (eksi $\times$ eksi) yine pozitif; biri üstte biri alttıysa negatif. Toplam pozitifse “birlikte artıyorlar”, negatifse “biri artarken diğeri azalıyor” demektir.

Payda niçin var? Çünkü kovaryansın tek başına bir sorunu vardır: birimi x ’in birimi çarpı y ’nin birimidir ve ölçeğe bağlıdır. Boy–kilo kovaryansı, boyu santimetre yerine milimetre ölçersen 10 katına çıkar — oysa ilişki değişmedi. İki standart sapmaya bölmek bu keyfiliği yok eder ve sonucu $[-1, +1]$ aralığına hapseder. Yani r , “**birimsizleştirilmiş kovaryanstır**” ve bu sayede boy–kilo ilişkisiyle sıcaklık–satış ilişkisi aynı cetvelle karşılaştırılabilir.

r^2 ’yi nasıl okumalı? $r = 0,873$ (yüzgeç uzunluğu – ağırlık) ise $r^2 = 0,76$: ağırlıktaki değişkenliğin %76’sı yüzgeç uzunluğuyla “açıklanıyor”, %24’ü başka şeylerden geliyor. Bu, regresyondaki R^2 ’nin ta kendisidir.

Nerede kullanılır?

- **Öznitelik seçimi:** Hedefle korelasyonu sıfıra yakın öznitelikler ilk elemeye düşer; birbiriyle $|r| > 0,9$ olan iki öznitelikten biri atılır (çoklu doğrusal bağlantı, Kısım 7).
- **Portföy yönetimi:** Riski düşürmenin yolu getirileri birbiriyle *düşük korelasyonlu* varlıkları bir arada tutmaktır. Çeşitlendirmenin tüm matematiği korelasyon matrisidir.
- **Ölçüm doğrulama:** Yeni ve ucuz bir sensörün, pahalı referans cihazla korelasyonu ölçülerek güvenilirliği sınanır.
- **Öneri sistemleri:** “Senin gibi kullanıcılar” derken kastedilen, puanlama vektörleri yüksek korelasyonlu kullanıcılardır.

Dikkat

$r = 0$ “ilişki yok” demek *değildir*; “**doğrusal** ilişki yok” demektir. $y = x^2$ verisinde $-3 \dots 3$ aralığında $r \approx 0$ çıkar, oysa y tamamen x tarafından belirlenir. Bu yüzden korelasyona bakmadan önce daima saçılım grafiği çizilir — bir sonraki iki bölüm tam olarak bunun niçin hayati olduğunu gösteriyor.

Katsayı	Ölçtüğü	Ne zaman?
Pearson r	Doğrusal ilişki	Sürekli, yaklaşık normal, aykırı değer yok.
Spearman ρ	Monoton ilişki (sıralara Pearson)	Sıralayıcı veri, doğrusal olmayan monoton ilişki, aykırı değer var.
Kendall τ	Uyumlu/uyumsuz çift oranı	Küçük örneklem, çok sayıda eşit değer (tie).

```
1 num = peng.select_dtypes('number')
2 print(num.corr().round(3))
```

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
bill_length_mm	1.000	-0.229	0.653	0.589
bill_depth_mm	-0.229	1.000	-0.578	-0.472
flipper_length_mm	0.653	-0.578	1.000	0.873
body_mass_g	0.589	-0.472	0.873	1.000

```

1 from scipy import stats as st
2 a, b = peng.bill_length_mm, peng.body_mass_g
3 print(f"Pearson = {st.pearsonr(a, b).statistic:.3f}")
4 print(f"Spearman = {st.spearmanr(a, b).statistic:.3f}")
5 print(f"Kendall = {st.kendalltau(a, b).statistic:.3f}")
6 # Pearson = 0.589 / Spearman = 0.576 / Kendall = 0.428

```

9.1 Simpson Paradoksu: Gerçek Bir Örnek

Korelasyon matrisinde gaga uzunluğu ile gaga derinliği arasında **negatif** bir ilişki ($r = -0,229$) görünüyor: uzun gagalı penguinlerin gagası daha ince. Şimdi aynı hesabı *tür içinde* yapalım.

```

1 r_all = st.pearsonr(peng.bill_length_mm, peng.bill_depth_mm)
2 print(f"TUM veri: r = {r_all.statistic:+.3f} (p = {r_all.pvalue:.2e})")
3
4 for sp, g in peng.groupby('species'):
5     r = st.pearsonr(g.bill_length_mm, g.bill_depth_mm)
6     print(f"{sp:>10}: r = {r.statistic:+.3f} (p = {r.pvalue:.4f}, n={len(g)})")

```

```

TUM veri: r = -0.229 (p = 2.53e-05)
Adelie: r = +0.386 (p = 0.0000, n=146)
Chinstrap: r = +0.654 (p = 0.0000, n=68)
Gentoo: r = +0.654 (p = 0.0000, n=119)

```

Dikkat

Simpson paradoksu: ilişkinin işareti gruplar birleştirilince tersine döner. Burada gerçek şu: her tür içinde uzun gaga = derin gaga (pozitif). Ama Gentoo'lar hem uzun hem *ince* gagalıdır ve veri setinin üçte birini oluşturur; türleri karıştırınca bu üçüncü değişken (tür) ilişkinin işaretini çevirir.

Ders: bir korelasyonu raporlamadan önce “hangi gizli grup değişkeni bunu açıklıyor olabilir?” diye sor.

```

1 import matplotlib.pyplot as plt, seaborn as sns
2
3 fig, (a1, a2) = plt.subplots(1, 2, figsize=(11, 4.5), sharey=True)
4 sns.regplot(data=peng, x='bill_length_mm', y='bill_depth_mm',
5             ax=a1, scatter_kws={'s': 18, 'alpha': .6}, line_kws={'color': 'C3'})
6 a1.set_title('Tum veri: r = -0.23 (yaniltici)')
7 sns.scatterplot(data=peng, x='bill_length_mm', y='bill_depth_mm',
8                 hue='species', ax=a2, s=22)
9 for sp, g in peng.groupby('species'):
10     sns.regplot(data=g, x='bill_length_mm', y='bill_depth_mm',
11                 ax=a2, scatter=False, ci=None)
12 a2.set_title('Tur bazinda: r > 0 (dogru)')
13 plt.tight_layout(); plt.show()

```

İpucu

Korelasyon nedensellik değildir — ama nedensellik neredeyse her zaman korelasyon üretir. Bir r değeri gördüğünde üç alternatif açıklama her zaman masadadır: (1) $X \rightarrow Y$, (2) $Y \rightarrow X$, (3) gizli bir Z ikisini de etkiliyor (*confounding*). Üçüncüsünü elemenin tek kesin yolu **randomize deneydir**; gözlemsel veride ancak kontrol değişkenleriyle yaklaşabilirsin.

9.2 Anscombe Dörtlüsü: Neden Grafik Çizmelişin?

```
1 ans = sns.load_dataset('anscombe')
2 ozet = ans.groupby('dataset').apply(
3     lambda g: pd.Series({
4         'x_mean': g.x.mean(), 'y_mean': g.y.mean(),
5         'x_std': g.x.std(), 'y_std': g.y.std(),
6         'r': st.pearsonr(g.x, g.y).statistic}),
7     include_groups=False).round(2)
8 print(ozet)
```

dataset	x_mean	y_mean	x_std	y_std	r
I	9.0	7.5	3.32	2.03	0.82
II	9.0	7.5	3.32	2.03	0.82
III	9.0	7.5	3.32	2.03	0.82
IV	9.0	7.5	3.32	2.03	0.82

Dikkat

Dört veri setinin ortalaması, standart sapması ve korelasyonu *aynı* — ama grafikleri tamamen farklıdır: biri doğrusal, biri parabol, biri tek aykırı değerın sürüklediği bir doğru, biri de tek bir noktanın yarattığı sahte ilişki. **Özet istatistik asla grafiğin yerini tutmaz.**

KISIM 3

Olasılık Teorisi

10. Temel Kavramlar ve Aksiyomlar

Tanım

Bir *rastgele deneyin* tüm olası sonuçlarının kümesine **örnek uzay** (Ω), örnek uzayın alt kümelerine **olay** denir. Olasılık, olaylara $[0, 1]$ aralığında sayı atayan bir fonksiyondur.

Kolmogorov Aksiyomları

1. $\mathbb{P}(A) \geq 0$ (negatif olmama),
2. $\mathbb{P}(\Omega) = 1$ (normalizasyon),
3. Ayrık $A_1, A_2, \dots$ için $\mathbb{P}(\bigcup_i A_i) = \sum_i \mathbb{P}(A_i)$ (toplanabilirlik).

Bütün olasılık kuralları bu üçünden türetilir.

Nedir? Niçin? Nerede?

Aksiyom ne demek, niçin gerekli? Aksiyom, ispatlanmayan ama kabul edilen başlangıç kuralıdır. Kolmogorov 1933'te şunu fark etti: olasılığı “uzun vadede görülme sıklığı” ya da “inanç derecesi” diye tanımlamaya çalışmak felsefi tartışma doğuruyor, oysa *nasil davranması gerektiğini* üç kuralla sabitlersen bütün matematik kendiliğinden gelir. Bu üç madde, olasılığın “oyun kurallarıdır”.

Her biri sezgisel olarak ne diyor?

- (1) *Negatif olamaz.* “Yarın yağmur yağma olasılığı $-0,3$ ” cümlesinin bir anlamı yok.
- (2) *Bir şeyin olma olasılığı 1'dir.* Tüm sonuçları listeledinse, bunlardan birinin gerçekleşeceği kesindir. Bütün yüzdelerin toplamı %100 etmelidir.
- (3) *Ayrık olayların olasılıkları toplanır.* Zar 1 veya 2 gelme olasılığı $1/6 + 1/6$ 'dır — çünkü ikisi aynı anda olamaz. Aynı anda olabilseydi çakışmayı iki kez sayardık; toplama kuralındaki $-\mathbb{P}(A \cap B)$ terimi tam olarak bu düzeltmedir.

Bu soyutluk nerede işe yarıyor? Aksiyomlar sayesinde *olasılık* kelimesi çok farklı şeyleri aynı matematikle modelleyebilir: bir zarın yüzü, bir e-postanın spam olması, bir hastanın iyileşmesi, bir sinir ağının çıktı katmanındaki softmax değerleri. softmax'ın niçin üstel alıp toplama böldüğünü merak ettiysen cevabı budur: negatif olmama (üstel) ve toplamın 1 olması (normalizasyon) aksiyomlarını zorla sağlatıyor. Bir modelin çıktısına “olasılık” diyebilmenin bedeli bu üç kuralı sağlamaktır.

Kural	Formül	Not
Tümleyen	$\mathbb{P}(A^c) = 1 - \mathbb{P}(A)$	“En az bir” problemlerinde vazgeçilmez.
Toplama	$\mathbb{P}(A \cup B) = \mathbb{P}(A) + \mathbb{P}(B) - \mathbb{P}(A \cap B)$	Kesişim iki kez sayılmasın.
Koşullu	$\mathbb{P}(A B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}$	$\mathbb{P}(B) > 0$ olmalı.
Çarpma	$\mathbb{P}(A \cap B) = \mathbb{P}(A B)\mathbb{P}(B)$	Zincirleme genişletilebilir.
Bağımsızlık	$\mathbb{P}(A \cap B) = \mathbb{P}(A)\mathbb{P}(B)$	Eşdeğeri: $\mathbb{P}(A B) = \mathbb{P}(A)$.
Toplam olasılık	$\mathbb{P}(A) = \sum_i \mathbb{P}(A B_i)\mathbb{P}(B_i)$	$\{B_i\}$ örnek uzayı böler.

Dikkat

Bağımsızlık ile **ayrıklık** (mutually exclusive) karıştırılır ama zıt kavramlardır. Ayrık olaylar ($A \cap B = \emptyset$) olabildiğince *bağımlıdır*: biri olduysa diğeri kesinlikle olmamıştır. Olasılığı pozitif iki olay hem ayrık hem bağımsız olamaz.

11. Koşullu Olasılık: Titanic Üzerinde

Koşullu olasılık, “elimizdeki bilgiyi kullanarak olasılığı güncellemek” demektir. 891 Titanic yolcusuyla somutlaştıralım.

Niçin En Önemli Kavram?

$\mathbb{P}(A \mid B) = \mathbb{P}(A \cap B) / \mathbb{P}(B)$ formülü basit görünür, ama yaptığı şey derindir: **örnek uzayı daraltır**. Payda artık Ω değil B 'dir — yani “ B olduğunu öğrendim, dünyam küçüldü, şimdi bu küçük dünyada A ne kadar yer kaplıyor?” diye soruyorsun. Titanic’te “bir yolcunun hayatta kalma olasılığı %38” cümlesi, “kadın olduğunu biliyorsam %74” cümlesine dönüşür. Aynı olay, farklı bilgi, farklı olasılık.

Niçin bu kadar merkezî? Çünkü *tahmin* kelimesinin kendisi koşullu olasılıktır. Bir makine öğrenmesi modeli ne yapar? $\mathbb{P}(\text{sınıf} \mid \text{öznitelikler})$ hesaplar. Lojistik regresyon, Naive Bayes, rastgele orman, sinir ağı — hepsi bu tek ifadeyi farklı yollardan kestirmeye çalışır. Koşullu olasılığı anlamak, bütün denetimli öğrenmenin ne yaptığını anlamaktır.

Nerede karşına çıkar? Tıbbi teşhis (semptom verildiğinde hastalık), spam filtresi (kelimeler verildiğinde spam), kredi skoru (geçmiş verildiğinde temerrüt), öneri sistemi (izlenenler verildiğinde beğeni), dil modelleri (önceki kelimeler verildiğinde sonraki kelime). Sonuncusu dikkat çekicidir: büyük dil modelleri devasa bir $\mathbb{P}(\text{sonraki jeton} \mid \text{bağlam})$ hesaplayıcısıdır.

```
1 import pandas as pd, seaborn as sns
2 tit = sns.load_dataset('titanic')
3
4 tab = pd.crosstab(tit['sex'], tit['survived'])
5 print(tab)
6 print("\nSatir yuzdeleri:")
7 print((tab.div(tab.sum(axis=1), axis=0)*100).round(1))
```

```
survived    0    1
sex
female      81   233
male       468   109
```

```
Satir yuzdeleri:
survived    0    1
sex
female     25.8  74.2
male      81.1  18.9
```

```
1 print("P(survived)           =", round(tit.survived.mean(), 4))
2 print("P(survived | female) =", round(tit.loc[tit.sex=='female', 'survived'].mean(), 4))
3 print("P(survived | male)   =", round(tit.loc[tit.sex=='male', 'survived'].mean(), 4))
4 print("P(female | survived) =", round((tit.loc[tit.survived==1, 'sex']=='female').mean(), 4))
```

```
P(survived)           = 0.3838
P(survived | female) = 0.742
P(survived | male)   = 0.1889
P(female | survived) = 0.6813
```

Dikkat

$\mathbb{P}(A \mid B)$ ile $\mathbb{P}(B \mid A)$ **aynı şey değildir** ve bu, istatistikteki en yaygın akıl yürütme hatasıdır (*koşulun ters çevrilmesi*). Burada $\mathbb{P}(\text{hayatta} \mid \text{kadın}) = 0,742$ iken $\mathbb{P}(\text{kadın} \mid \text{hayatta}) = 0,681$. İkisi ancak marjinal olasılıklar eşitse çakışır.

11.1 Bayes Teoremi

Bayes

$$\mathbb{P}(A | B) = \frac{\mathbb{P}(B | A) \mathbb{P}(A)}{\mathbb{P}(B)} = \frac{\mathbb{P}(B | A) \mathbb{P}(A)}{\sum_i \mathbb{P}(B | A_i) \mathbb{P}(A_i)}$$

Sözle: $\text{sonsal} \propto \text{olabilirlik} \times \text{önsel}$.

Nedir? Niçin? Nerede?

Bayes ne yapıyor? Koşulu *ters çevirir*. Elinde $\mathbb{P}(B | A)$ var ama sana lazım olan $\mathbb{P}(A | B)$. Bu ikisi *aynı şey değildir* ve karıştırmak istatistikteki en pahalı hatadır: “hasta olan birinin testi pozitif çıkma olasılığı” ile “testi pozitif çıkan birinin hasta olma olasılığı” tamamen farklı sayılardır.

Dört terimin adı ve anlamı:

- $\mathbb{P}(A)$ — **önsel** (prior): Kanıtı görmeden önceki inancın. “Bu hastalık toplumda binde bir görülür.”
- $\mathbb{P}(B | A)$ — **olabilirlik** (likelihood): Hipotez doğruysa bu kanıtı görme şansın. “Hastaysa test %99 pozitif verir.”
- $\mathbb{P}(B)$ — **kanıt** (evidence): Kanıtın her ihtimal altında toplam görülme olasılığı. Yalnızca normalizasyon içindir; sonucun 0–1 aralığında kalmasını sağlar.
- $\mathbb{P}(A | B)$ — **sonsal** (posterior): Kanıtı gördükten sonraki güncellenmiş inancın. Aradığın sayı budur.

Niçin bu kadar güçlü? Çünkü *öğrenmenin* matematiksel tarifidir: bir inancın vardı, veri geldi, inancını güncelledin. Yeni veri gelince bugünün sonsalı yarının önseli olur. Bu döngü, bir hekimin teşhis sürecinden bir Kalman filtresinin uydu konumu düzeltmesine kadar aynıdır.

En kritik ders — önsel her şeyi değiştirir. Aynı test sonucu, hastalığın yaygınlığına göre “%2 ihtimalle hastasın” ya da “%83 ihtimalle hastasın” anlamına gelebilir. Bir sonraki sayfadaki tablo tam olarak bunu gösteriyor. Kanıtı önselden koparıp yorumlamak, sayının kendisini anlam-sızlaştırır.

Bayes’in Günlük Hayattaki Dört Yüzü

- **Spam filtresi:** “Ücretsiz” kelimesi geçen bir e-posta geldi. Filtre $\mathbb{P}(\text{spam} | \text{“ücretsiz”})$ hesaplar. Önsel: gelen postaların %40’ı spam. Olabilirlik: spam’lerin %15’inde, normal postaların %1’inde bu kelime geçer. Sonsal $\approx$ %91. Naive Bayes sınıflandırıcısı (Kısım 9) tam olarak budur — yüzlerce kelime için bunu yapıp çarpar.
- **Adli tıp:** DNA eşleşme olasılığı milyonda birdir. Savcı “o hâlde sanığın masum olma ihtimali milyonda bir” der — bu **savcı yanlıgısıdır**. 10 milyon kişilik bir şehirde tesadüfen eşleşen $\approx$ 10 kişi vardır; başka kanıt yoksa sanığın suçlu olma olasılığı %10’dur, %99,9999 değil. Bu hata gerçek mahkemelerde masum insanları hapse attı.
- **Arama kurtarma:** Kayıp bir denizaltı ya da uçak aranırken deniz haritası hücrelere bölünür, her hücreye bir önsel olasılık verilir. Bir hücre aranıp bulunamazsa *tüm haritanın* olasılıkları Bayes ile güncellenir. Air France 447 enkazı 2011’de tam bu yöntemle bulundu.
- **A/B testi:** “B versiyonu A’dan iyidir” inancını, gelen her yeni ziyaretçiyle güncellersin. Kısım 6’daki Bayesçi çıkarım bölümü bunun uygulamasıdır — ve bir pazarlama müdürüne “ $p < 0,05$ ” demektense “%94 ihtimalle B daha iyi” demek çok daha kullanışlıdır.

```
1 pf = (tit.sex == 'female').mean()
```

```
# onsel P(female)
```

```

2 psf = tit.loc[tit.sex == 'female', 'survived'].mean() # P(surv | female)
3 ps = tit.survived.mean() # kanıt P(surv)
4 print("Bayes ile P(female | survived) =", round(psf*pf/ps, 4)) # 0.6813

```

11.2 Klasik Tuzak: Tıbbi Tarama Testi

Soru 1

Bir hastalık toplumda %0,1 oranında görülüyor. Test, hastaları %99 doğrulukla yakalıyor (*duyarlılık*) ve sağlıklıları %95 doğrulukla eliyor (*özgüllük*). Testin *pozitif* çıktı. Gerçekten hasta olma olasılığın nedir?

Çözüm. Sezgi “%99” der; doğru cevap %2’dir.

$$\mathbb{P}(H | +) = \frac{0,99 \times 0,001}{0,99 \times 0,001 + 0,05 \times 0,999} = \frac{0,00099}{0,05094} = 0,0194.$$

```

1 sens, spec = 0.99, 0.95
2 for prev in [0.001, 0.01, 0.05, 0.2]:
3     p_pos = sens*prev + (1 - spec)*(1 - prev)
4     ppv = sens*prev/p_pos
5     print(f"yayginlik = {prev:6.3f} -> P(hasta | +) = {ppv:.4f}")

```

```

yayginlik = 0.001 -> P(hasta | +) = 0.0194
yayginlik = 0.010 -> P(hasta | +) = 0.1667
yayginlik = 0.050 -> P(hasta | +) = 0.5103
yayginlik = 0.200 -> P(hasta | +) = 0.8319

```

Bilgi

Taban oranı yanılgısı (base rate fallacy): nadir bir olayda, mükemmel yakın bir test bile çok sayıda *yalancı pozitif* üretir. Bu, makine öğrenmesinde dengesiz veri probleminin (Kısım 11) tam karşılığıdır: %99,9 doğruluk, taban oranı %0,1 olan bir problemde hiçbir şey ifade etmez. Aynı hesap, sınıflandırma metriklerinde *kesinlik* (precision) adıyla karşımıza çıkacak.

11.3 Kontenjans Tablosunda Bağımsızlık

```

1 from scipy import stats as st
2 chi2, p, dof, exp = st.chi2_contingency(tab)
3 print(f"chi2 = {chi2:.2f}   dof = {dof}   p = {p:.3e}")
4 print("Bağımsızlık altında BEKLENEN sayılar:")
5 print(pd.DataFrame(exp, index=tab.index, columns=tab.columns).round(1))

```

```

chi2 = 260.72   dof = 1   p = 1.197e-58
Bağımsızlık altında BEKLENEN sayılar:
survived      0      1
sex
female      193.5  120.5
male       355.5  221.5

```

Gözlenen 233 kadın hayatta kalanına karşılık bağımsızlık altında beklenen sayı yalnızca 120,5. Cinsiyet ile hayatta kalma *kesinlikle* bağımsız değil.

11.4 Koşullamaya Devam: Üçüncü Değişken

```
1 print(tit.groupby(['pclass', 'sex'])['survived'].agg(['count', 'mean']).round(3))
```

		count	mean
pclass	sex		
1	female	94	0.968
	male	122	0.369
2	female	76	0.921
	male	108	0.157
3	female	144	0.500
	male	347	0.135

Örnek

1. sınıf kadın yolcuların %96,8'i, 3. sınıf erkeklerin ise yalnızca %13,5'i hayatta kalmış. “Kadınlar ve çocuklar önce” kuralı sınıf farkıyla birleşince 7 kata varan bir fark üretiyor. Bu tablo, Kısım 7’de kuracağımız lojistik regresyonun neden sex ve pclass değişkenlerini birlikte kullanması gerektiğini gösteriyor.

12. Kombinatorik ve Simülasyon

Sayma Niçin Olasılığın Temeli?

Tüm sonuçlar eşit olasılıklıysa olasılık sadece bir bölme işlemidir: $\mathbb{P}(A) = \frac{\text{istenen sonuç sayısı}}{\text{tüm sonuç sayısı}}$. Sorun şu ki bu iki sayı çoğunlukla elle sayılamayacak kadar büyüktür — 52 kartlık desteden 5 kart seçmenin 2.598.960 yolu vardır. Kombinatorik, **listelemeden sayma sanatıdır**.

Dört formülü ayıran iki soru: (1) *Sıra önemli mi?* Şifre için evet (“ab” $\neq$ “ba”), loto kuponu için hayır. (2) *İade var mı?* Zar atmada evet (aynı sayı tekrar gelebilir), kart çekmede hayır. Bu iki sorunun cevabı hangi formülü kullanacağını doğrudan belirler — ezberlemek yerine her seferinde bu iki soruyu sor.

Kombinasyon formülü niçin öyle? $\binom{n}{r} = \frac{n!}{r!(n-r)!}$ şöyle okunur: önce sıralı seç $(n!/(n-r)!)$, sonra aynı r elemanın $r!$ farklı sıralanışını tek saymak için $r!$ ’e böl. Yani kombinasyon, “permütasyonun fazla saydığını düzeltmesidir”.

Nerede kullanılır?

- **Şifre güvenliği:** 8 karakterli, 62 sembollü bir şifrenin $62^8 \approx 2,2 \times 10^{14}$ ihtimali vardır. Modern bir GPU saniyede 10^{10} deneme yapar — yani 6 saat. Şifreyi 12 karaktere çıkarınca süre 34.000 yıla fırlar. Güvenlik politikalarının arkasındaki tek matematik budur.
- **A/B/n test tasarımı:** 4 başlık $\times$ 3 görsel $\times$ 2 buton rengi = 24 varyant. Çarpım kuralı, “kaç kullanıcıya ihtiyacım var?” sorusunun ilk adımıdır.
- **Makine öğrenmesi:** k katlı çapraz doğrulamada kaç farklı eğitim kümesi oluşur; GridSearchCV’de parametre ızgarasının boyutu (3 parametrenin her biri 5 değer $\rightarrow$ 125 model); rastgele ormanın her düğümde $\binom{p}{m}$ öznelilik alt kümesinden seçim yapması.
- **Karma çakışması:** Doğum günü problemi, 128 bitlik bir karma fonksiyonunun niçin 2^{64} kayıttan sonra çakışmaya başladığını söyler — kriptografideki “doğum günü sınırı”.

Sayma	Formül	Ne zaman?
Çarpım kuralı	$n_1 n_2 \cdots n_k$	Bağımsız aşamalar.
Permütasyon	$P(n, r) = \frac{n!}{(n-r)!}$	Sıra önemli, iadesiz.
Kombinasyon	$\binom{n}{r} = \frac{n!}{r!(n-r)!}$	Sıra önemsiz, iadesiz.
Tekrarlı	n^r	Sıra önemli, iadeli.

```

1 from math import comb, perm, factorial
2 from itertools import combinations, permutations
3
4 print(comb(49, 6))           # 13983816 -> 6/49 loto: tek kombinasyon olasılığı 1/13.98M
5 print(perm(10, 3))          # 720
6 print(factorial(5))          # 120
7
8 # Poker: full house olasılığı
9 toplam = comb(52, 5)
10 full_house = 13*comb(4,3)*12*comb(4,2)
11 print(f"P(full house) = {full_house/toplam:.6f}")    # 0.001441

```

12.1 Doğum Günü Problemi

Soru 2

Bir sınıfta kaç kişi olursa en az ikisinin doğum günü aynı olma olasılığı %50'yi geçer?

Çözüm. Tümleyenle çalışmak çok daha kolaydır:

$$\mathbb{P}(\text{en az bir ortak}) = 1 - \frac{365}{365} \cdot \frac{364}{365} \cdots \frac{365 - n + 1}{365}.$$

```

1 def p_ortak_dogumunu(n):
2     p = 1.0
3     for i in range(n):
4         p *= (365 - i)/365
5     return 1 - p
6
7 for n in [10, 23, 30, 50, 70]:
8     print(f"n = {n:>3}: P = {p_ortak_dogumunu(n):.4f}")

```

```

n = 10: P = 0.1169
n = 23: P = 0.5073
n = 30: P = 0.7063
n = 50: P = 0.9704
n = 70: P = 0.9992

```

Bilgi

23 kişi yeter. Sezgiye aykırı gelmesinin nedeni, “benimle aynı” değil “herhangi iki kişi aynı” sorusunun sorulmasıdır: 23 kişide $\binom{23}{2} = 253$ çift vardır. Aynı mantık, karma (hash) çakışmalarında ve kriptografideki *doğum günü saldırısında* karşımıza çıkar.

12.2 Monty Hall: Simülasyonla İnanmak

```

1 import numpy as np
2 rng = np.random.default_rng(42)
3 n = 200_000
4
5 odul = rng.integers(0, 3, n)      # odulun olduğu kapi
6 secim = rng.integers(0, 3, n)    # yarışmacının ilk secimi
7
8 # Sunucu bos bir kapi acar; DEGISTIRMEK, ilk secim yanlissa kazandırır
9 print(f"kalma      -> kazanma orani = {(secim == odul).mean():.4f}")
10 print(f"degistirme -> kazanma orani = {(secim != odul).mean():.4f}")

```

```

kalma      -> kazanma orani = 0.3353
degistirme -> kazanma orani = 0.6647

```

İpucu

Bir olasılık sonucundan emin olamadığında **simüle et**. Beş satırlık bir Monte Carlo, saatlerce süren tartışmayı bitirir. Aynı teknik Kısım 5'te bootstrap, Kısım 6'da permütasyon testi olarak karşımıza çıkacak — hepsi aynı fikrin farklı kılıklarıdır.

13. Rastgele Değişkenler

Tanım

Rastgele değişken X , örnek uzaydaki her sonuca bir sayı atayan fonksiyondur. *Kesikli* ise olasılık kütle fonksiyonu (PMF) $p(x) = \mathbb{P}(X = x)$, *sürekli* ise olasılık yoğunluk fonksiyonu (PDF) $f(x)$ ile tanımlanır. Her ikisinde de birikimli dağılım fonksiyonu $F(x) = \mathbb{P}(X \leq x)$ 'tir.

Nedir? Niçin? Nerede?

Niçin böyle bir fonksiyona ihtiyaç var? Çünkü olasılık teorisi “yazı/tura”, “kırmızı/mavi” gibi *etiketlerle* değil, sayılarla çalışmak ister — ancak o zaman toplama, ortalama alma, integral gibi araçları kullanabilirsin. Rastgele değişken, gerçek dünyadaki sonuçları sayı doğrusuna taşıyan çevirmen katmanıdır: “tura” $\rightarrow 1$, “yazı” $\rightarrow 0$; “müşteri geldi” $\rightarrow$ harcadığı tutar; “sunucu yanıtladı” $\rightarrow$ milisaniye.

Kesikli mi sürekli mi — niçin önemli? Bu ayrım hangi araçları kullanacağını belirler: kesikli olanda *toplam*, sürekli olanda *integral*; kesikli olanda $\mathbb{P}(X = 3)$ anlamlıdır, sürekli olanda sıfırdır. Pratikte: sayılabilen şeyler (günlük sipariş adedi, hatalı ürün sayısı, tıklama var/yok) kesikli; ölçülen şeyler (ağırlık, süre, sıcaklık, gelir) sürekli.

CDF niçin PMF/PDF'ten daha kullanışlı? Çünkü $F(x) = \mathbb{P}(X \leq x)$ her ikisi için de aynı biçimde tanımlıdır ve doğrudan cevap verdiği sorular gerçek hayatta sorulan sorulardır: “Teslimatın 3 günden erken gelme olasılığı?” ($F(3)$), “Yüklerin %95'i hangi değerin altında?” ($F^{-1}(0,95)$, yani *yüzdelik*). `scipy.stats`'ta bunlar `cdf` ve `ppf` metotlarıdır; hipotez testlerinde p -değeri hesaplamak da, güven aralığı kurmak da bu iki fonksiyonun çağrılmasından ibarettir.

Dikkat

Sürekli dağılımda $f(x)$ bir *olasılık değildir*; 1'den büyük olabilir. Olasılık ancak alanla gelir: $\mathbb{P}(a < X < b) = \int_a^b f(x) dx$ ve tek bir noktanın olasılığı sıfırdır: $\mathbb{P}(X = c) = 0$.

Beklenen Değer ve Varyans

$$\mathbb{E}[X] = \sum_x x p(x) \quad \text{veya} \quad \int x f(x) dx,$$

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}X)^2] = \mathbb{E}[X^2] - (\mathbb{E}X)^2.$$

Nedir? Niçin? Nerede?

Beklenen değer ne demek? “Beklenen” kelimesi yanıltıcıdır: $\mathbb{E}[X]$ çoğu zaman *hiç gerçekleşmeyen* bir değerdir. Bir zarın beklenen değeri 3,5’tur ama zar asla 3,5 gelmez. Doğru okuma şudur: **deneyi sonsuz kez tekrarlasan uzun vadedeki ortalama budur**. Formül de tam olarak bunu söyler — her değeri, gerçekleşme olasılığıyla ağırlıklandırıp toplar. Yani $\mathbb{E}[X]$, “olasılıkla ağırlıklandırılmış ortalama”dır.

Betimsel istatistikteki $\bar{x}$ ile farkı ne? $\bar{x}$ elindeki *örneklemin* ortalamasıdır, gözlemlenen bir sayıdır. $\mathbb{E}[X]$ ise *dağılımın* ortalamasıdır, teorik ve bilinmeyen bir sayıdır. Bütün çıkarımsal istatistik (Kısım 5) tek bir cümleyle özetlenebilir: elimizdeki $\bar{x}$ ’ten, bilmediğimiz $\mathbb{E}[X]$ hakkında ne söyleyebiliriz?

$\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}X)^2$ niçin kullanışlı? İki eşdeğer formül var; soldaki tanım (ortalama karesel sapma), sağdaki *hesap* formülüdür. Sağdaki tek geçişte hesaplanabilir: veriyi bir kez dolaşırken hem $\sum x$ hem $\sum x^2$ biriktirirsin, ortalamaı önceden bilmene gerek kalmaz. Akan veride (streaming) ve büyük veride varyans bu yüzden hep bu formülle hesaplanır.

Nerede kullanılır?

- **Sigortacılık ve kumar:** Bir poliçenin primi, beklenen hasar tutarının üstüne kâr marjı eklenerek belirlenir. Rulette bahsin beklenen değeri her zaman negatiftir ($-1/37$ katı) — kasanın kazanması bir şans değil, aritmetik zorunluluktur.
- **Karar analizi:** “A projesi %20 ihtimalle 10M kazandırır, B projesi %90 ihtimalle 2M.” Beklenen değerler 2M ve 1,8M. Ama varyanslar çok farklı — bu yüzden ciddi karar analizi yalnızca $\mathbb{E}[X]$ ’e bakmaz.
- **Makine öğrenmesi:** Eğitilen her model bir *beklenen kayıp* (expected loss / risk) minimize eder. Yanlılık–varyans ayrışması (Kısım 8) doğrudan $\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}X)^2$ kimliğinin bir uygulamasıdır.
- **A/B testi kararı:** “Bu değişiklik kullanıcı başına beklenen geliri kaç TL artırır?” sorusu doğrudan $\mathbb{E}[X]$ farkıdır.

Beklenen Değer Tek Başına Yeterli mi?

Sana iki bahis öneriliyor: (A) kesin 100 TL kazanırsın; (B) %50 ihtimalle 200 TL, %50 ihtimalle 0 TL. İkisinin de beklenen değeri 100 TL — yani “rasyonel” karar veren biri kayıtsız kalmalı. Ama neredeyse herkes A’yı seçer.

Şimdi rakamları büyütelim: (A) kesin 1 milyon TL; (B) %50 ihtimalle 2 milyon, %50 ihtimalle 0. Beklenen değerler yine eşit ama artık A’yı seçmek çok daha güçlü bir tercih. Buna **riskten kaçınma** denir ve beklenen değerini niçin *tek başına* bir karar ölçütü olamayacağını gösterir. Bu yüzden finasta “getiri/risk” (Sharpe), sigortacılıkta “fayda fonksiyonu”, makine öğrenmesinde “ortalama skor $\pm$ standart sapma” birlikte raporlanır. **Kural:** $\mathbb{E}[X]$ ile $\text{Var}(X)$ **hiçbir zaman ayrı ayrı okunmaz**.

Özellik	Koşul / not
$\mathbb{E}[aX + b] = a\mathbb{E}[X] + b$	Her zaman (doğrusallık).
$\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$	Her zaman — bağımsızlık gerekmez.
$\text{Var}(aX + b) = a^2 \text{Var}(X)$	Sabit eklemek yayılımı değiştirmez.
$\text{Var}(X + Y) = \text{Var} X + \text{Var} Y + 2 \text{Cov}(X, Y)$	Genel hâl.
$\text{Var}(X + Y) = \text{Var} X + \text{Var} Y$	Yalnızca ilişkisizse.
$\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$	Yalnızca bağımsızsa.

Dikkat

$\text{Cov}(X, Y) = 0$ (ilişkisizlik) **bağımsızlık demek değildir**. $X \sim \mathcal{N}(0, 1)$ ve $Y = X^2$ alırsak $\text{Cov}(X, Y) = 0$ 'dır ama Y tamamen X tarafından belirlenir. Korelasyon yalnızca *doğrusal* bağımlılığı görür; bu, korelasyon matrisine bakıp “bu iki öz nitelik bağımsız” demenin neden yanlış olduğunun sebebidir.

```

1 import numpy as np
2 rng = np.random.default_rng(0)
3
4 X = rng.normal(size=100_000)
5 Y = X**2
6 print(f"Cov(X, Y) = {np.cov(X, Y)[0,1]:+.4f}")      # ~0 (iliskisiz)
7 print(f"Corr      = {np.corrcoef(X, Y)[0,1]:+.4f}") # ~0
8 print("Ama Y tamamen X tarafından belirleniyor -> bagimsiz DEGIL")

```

13.1 Eşbileşik, Marjinal ve Koşullu Dağılımlar**Formül**

$$p_X(x) = \sum_y p(x, y) \quad (\text{marjinal}), \quad p_{Y|X}(y | x) = \frac{p(x, y)}{p_X(x)} \quad (\text{koşullu}).$$

Bağımsızlık $\iff p(x, y) = p_X(x)p_Y(y)$ her (x, y) için.

Nedir? Niçin? Nerede?

Üç dağılım, üç farklı soru. Bir kontenjans tablosuna bakarken aslında üç ayrı şey sorabilirsin ve üçü farklı sayılar verir:

- **Eşbileşik** $p(x, y)$: “Rastgele seçilen bir yolcunun kadın *ve* 1. sınıf olma olasılığı?” Payda tüm yolcular.
- **Marjinal** $p_X(x)$: “Kadın olma olasılığı?” Diğer değişkeni *topla ve unut* — tablonun kenar (margin) toplamaları olduğu için bu adı alır.
- **Koşullu** $p_{Y|X}$: “Kadın olduğunu bildiğime göre 1. sınıf olma olasılığı?” Payda artık yalnızca kadınlar.

Raporlama hatalarının büyük kısmı bu üçünü karıştırmaktan doğar. “Kazaların %70’i erkek sürücülerde” cümlesi eşbileşiktir ve neredeyse hiçbir şey söylemez — eğer sürücülerin %70’i zaten erkekse. Anlamalı olan koşullu orandır: “*erkek sürücü başına* kaza sayısı.”

Marjinalleştirme niçin bu kadar sık kullanılır? Çünkü “umursamadığın değişkeni yok etmenin” resmî yoludur. Modelin 20 değişkenle çalışıyor ama sen yalnızca ikisinin ilişkisini merak ediyorsan, kalan 18’i toplayarak (veya integre ederek) elersin. Bayesçi istatistikte bir parametrenin sonsal dağılımını elde etmek, diğer tüm parametreleri marjinalleştirmek demektir.

Nerede kullanılır? Öneri sistemlerinde kullanıcı–ürün eşbileşik matrisi; pazarlama segmentasyonunda çapraz tablolar; olasılıksal grafik modellerde (Bayes ağları) çıkarım; pandas’ta `crosstab(..., normalize=)` parametresinin `True` / `'index'` / `'columns'` değerleri sırasıyla tam olarak bu üç dağılımı verir.

```
1 # Titanic üzerinde esbilesik / marjinal / kosullu
2 ortak = pd.crosstab(tit.sex, tit['class'], normalize=True)      # esbilesik
3 print(ortak.round(3))
4 print("\nMarjinal (satir):", ortak.sum(axis=1).round(3).to_dict())
5 print("Marjinal (sutun):", ortak.sum(axis=0).round(3).to_dict())
6
7 kosullu = pd.crosstab(tit.sex, tit['class'], normalize='index') # P(class | sex)
8 print("\nP(class | sex):"); print(kosullu.round(3))
```

Toplam Beklenti ve Varyans Ayrıştırması

$$\mathbb{E}[Y] = \mathbb{E}[\mathbb{E}[Y | X]], \quad \text{Var}(Y) = \underbrace{\mathbb{E}[\text{Var}(Y | X)]}_{\text{grup içi}} + \underbrace{\text{Var}(\mathbb{E}[Y | X])}_{\text{gruplar arası}}.$$

İkinci eşitlik, ANOVA’nın (Kısım 6) ve karar ağaçlarındaki bölme ölçütünün (Kısım 9) ortak matematiksel temelidir.

Nedir? Niçin? Nerede?

Varyans ayırıştırması ne diyor? Bir büyüklüğün toplam değişkenliği ikiye bölünebilir: *gruplar arasındaki fark* ve *grup içinde kalan fark*. Penguen örneğinde: ağırlıklar niçin farklı? Kısmen farklı türler olduğu için (gruplar arası), kısmen aynı tür içinde de bireysel farklar olduğu için (grup içi). **Niçin bu kadar merkezî bir fikir?** Çünkü “açıklamak” kelimesinin matematiksel tanımıdır. Bir değişken, hedefteki değişkenliğin ne kadarını gruplar arası farka çevirebiliyorsa o kadar açıklayıcıdır. Aynı sayı üç ayrı isimle karşına çıkar — ANOVA’da η^2 , regresyonda R^2 , karar ağacında “varyans azalması”. Üçü de aynı bölme işlemidir: *gruplar arası / toplam*.

Karar ağacı niçin böyle çalışır? Bir ağaç, her düğümde tüm olası bölmeleri dener ve “hangi bölme gruplar arası varyansı en çok büyütür / grup içi varyansı en çok küçültür?” diye sorar. Yani bir karar ağacı, bu formülü açgözlü bir döngü içinde defalarca uygulamaktan başka bir şey yapmaz. Sınıflandırmada varyans yerine Gini veya entropi kullanılır ama mantık birebir aynıdır.

Toplam beklenti yasası ($E[Y] = E[E[Y | X]]$) **nerede işe yarar?** Karmaşık bir ortalamayı, parçalara ayırıp sonra ağırlıklı ortalamasını almak için. Örnek: bir e-ticaret sitesinin ortalama sepet tutarını doğrudan hesaplamak yerine, önce her müşteri segmentinin ortalamasını bul, sonra segment paylarıyla ağırlıklandır. Sonuç aynıdır ama ikinci yol *yorumlanabilir* — hangi segmentin toplamı nasıl etkilediğini görürsün.

Ayrıştırmanın İşe Yaradığı Bir Karar

Bir çağrı merkezinde ortalama görüşme süresi 6 dakika, standart sapma 4 dakika. Yönetim “süreyi düşürelim” diyor. Hangi müdahale?

Varyansı ayırıştırıyorsun: toplam varyansın %15’i *temsilciler arası* farktan, %85’i *aynı temsilcinin görüşmeleri arasındaki* farktan geliyor. Bu tek sayı stratejiyi belirler — temsilcilere eğitim vermek (gruplar arası farkı kapatmak) en fazla %15’lik kısma dokunabilir. Asıl kaynak grup içi: yani *çağrının türü*. Doğru hamle, çağrılarını konuya göre ayırıp sık sorulanlara self-servis çözüm üretmektir.

Ayrıştırma yapmadan “temsilcileri eğitelim” demek, bütçenin %85’ini yanlış yere harcamak olurdu.

```
1 # Varyans ayırıştırması: penguen kutlesi, tür değişkeniyle açıklanıyor mu?
2 peng = sns.load_dataset('penguins').dropna()
3 grup = peng.groupby('species')['body_mass_g']
4
5 toplam_var = peng['body_mass_g'].var(ddof=0)
6 grup_ici = (grup.transform('mean') - peng['body_mass_g']).pow(2).mean()
7 gruplar_arasi = (grup.transform('mean') - peng['body_mass_g'].mean()).pow(2).mean()
8
9 print(f"toplam varyans      = {toplam_var:,.0f}")
10 print(f"grup içi          = {grup_ici:,.0f}  ({grup_ici/toplam_var:.1%})")
11 print(f"gruplar arası      = {gruplar_arasi:,.0f}  ({gruplar_arasi/toplam_var:.1%})")
```

```
toplam varyans      = 646,425
grup içi           = 210,419   (32.6%)
gruplar arası      = 436,007   (67.4%)
```

Bilgi

Vücut kütlesindeki değişkenliğin %67,4’ü yalnızca *tür* bilinerek açıklanıyor. Bu oran, ANOVA’da η^2 (eta-kare) etki büyüklüğü olarak; regresyonda R^2 olarak; karar ağaçlarında “varyans azalması” olarak karşınıza çıkacak. Üçü de aynı sayıdır.

KISIM 4

Olasılık Dağılımları

Dağılım Nedir, Niçin Ezberlenmez?

Bir **olasılık dağılımı**, “bu rastgele büyüklük hangi değerleri, hangi sıklıkla alır?” sorusunun tam cevabıdır. Ama asıl önemli olan şudur: her dağılım, gerçek dünyadaki bir *mekanizmanın* matematiksel imzasıdır. Binom “ n bağımsız denemede kaç başarı” mekanizmasından, Poisson “çok sayıda fırsat, her birinde küçük ihtimal” mekanizmasından, üstel “hafızasız bekleme” mekanizmasından doğar.

Bu yüzden dağılımları formülleriyle ezberlemek yanlış stratejidir. Doğru soru şudur: “**verimi üreten süreç hangi mekanizmaya benziyor?**” Mekanizmayı tanıdığın an dağılım kendiliğinden gelir — ve dağılımı bildiğin an, tek bir veri noktan bile olmadan olasılık hesaplayabilir, kapasite planlayabilir, simülasyon kurabilirsin.

Dağılım bilmenin pratik faydası nedir?

- **Az veriyle çok şey söylemek:** Süreç Poisson ise, ortalamayı bilmek tüm dağılımı bilmek demektir — “en kötü günde kaç arıza?” sorusunu 5 veri noktasıyla cevaplayabilirsin.
- **Simülasyon:** Gerçek veri toplamak pahalysa, dağılımdan üretirsin (Monte Carlo).
- **Hipotez testi:** Her test istatistiği (t , F , χ^2) bilinen bir dağılıma sahiptir; p -değeri o dağılımın kuyruk alanıdır.
- **Makine öğrenmesi:** Naive Bayes özniteliklerin dağılımını varsayar, GLM hedefin dağılımını seçtirir, üretici modeller (VAE, difüzyon) doğrudan veri dağılımını öğrenmeye çalışır.

14. scipy.stats Arayüzü

`scipy.stats`’taki her dağılım aynı arayüze sahiptir; bir kez öğrenince hepsini bilirsin.

Metot	Ne verir?
<code>pdf(x) / pmf(k)</code>	Yoğunluk (sürekli) / kütle (kesikli).
<code>cdf(x)</code>	$\mathbb{P}(X \leq x)$.
<code>sf(x)</code>	$\mathbb{P}(X > x) = 1 - \text{cdf'ten sayısal olarak daha kararlı.}$
<code>ppf(q)</code>	Yüzdelik (cdf'in tersi); kritik değerler buradan gelir.
<code>isf(q)</code>	Üst kuyruk kritik değeri.
<code>rvs(size, random_state)</code>	Rastgele örnek üretir.
<code>mean() / var() / std() / median()</code>	Teorik momentler.
<code>interval(0.95)</code>	Merkezî %95 aralığı.
<code>fit(data)</code>	MLE ile parametre tahmini (sürekli dağılımlar).

```

1 from scipy import stats as st
2 import numpy as np
3
4 d = st.norm(loc=100, scale=15)          # "donmus" (frozen) dagilim nesnesi
5
6 print(f"pdf(115) = {d.pdf(115):.6f}")
7 print(f"cdf(115) = {d.cdf(115):.4f}")    # P(X <= 115)
8 print(f"ppf(0.95) = {d.ppf(0.95):.2f}")  # %95 yuzdeligi
9 print(f"mean={d.mean():.1f} std={d.std():.1f} median={d.median():.1f}")
10 print("rvs(5) =", np.round(d.rvs(5, random_state=42), 2))

```

```

pdf(115) = 0.016131
cdf(115) = 0.8413
ppf(0.95) = 124.67
mean=100.0 std=15.0 median=100.0
rvs(5) = [107.45  97.93 109.72 122.85  96.49]

```

İpucu

`loc` ve `scale` her dağılımda vardır ve anlamı *kaydırma* ile *ölçeklemedir*. Normalde `loc =  $\mu$` , `scale =  $\sigma$` ; üstelde `scale =  $1/\lambda$` — bu ikincisi en sık yapılan hatadır. Şüphedeysen `d.mean()` ile kontrol et.

15. Kesikli Dağılımlar

Hangi Dağılım, Hangi Soru?

Aşağıdaki tablodaki altı dağılım aslında tek bir hikâyenin varyasyonlarıdır: *tekrarlanan başarı/başarısızlık denemeleri*. Aralarındaki farkı “neyi sabitleyip neyi saydığın” belirler.

- **Bernoulli** — “**Bu tek deneme başarılı mı?**” Yapı taşıdır: bir madenî para, bir tıklama, bir e-postanın spam olması. Diğer her şey bunun üstüne kurulur.
- **Binom** — “ **n denemede kaç başarı?**” Deneme sayısı sabit, başarı sayısı rastgele. *Örnek:* 1000 kişiye e-posta gönderdin, tıklama oranı %3; kaç tıklama bekliyorsun ve 50’den fazla gelmesi ne kadar şaşırtıcı olurdu?
- **Poisson** — “**Bu zaman aralığında kaç olay?**” Deneme sayısı belirsizce çok, her birinin olasılığı çok küçük. *Örnek:* bir saatte gelen destek talebi sayısı, bir kilometre yolda çukur sayısı, bir sayfada dizgi hatası sayısı, sunucuda dakikadaki hata logu.
- **Geometrik** — “**İlk başarıya kadar kaç deneme?**” Şimdi *başarı sayısı* sabit (bir tane) ve *deneme sayısı* rastgele; binomun tam tersi. *Örnek:* kaç soğuk aramadan sonra ilk satış gelir; bir A/B testinde ilk dönüşüme kadar kaç ziyaretçi.
- **Negatif binom** — “ **r başarıya kadar kaç deneme?**” Geometriğin genellemesi. Ayrıca ikinci bir hayatı vardır: Poisson’un varsayımı ($\mathbb{E} = \text{Var}$) tutmadığında, yani veri *aşırı yayılım* gösterdiğinde sayım verisi için yedek modeldir. Gerçek sayım verilerinde bu neredeyse kuraldır.
- **Hipergeometrik** — “**İadesiz çekilişte kaç başarı?**” Binomla aynı soruyu sorar ama çektiğini geri koymaz, dolayısıyla denemeler bağımsız değildir. *Örnek:* 500 ürünlük partiden 20 ürün örnekleyip kalite kontrol; 52 karttan 5 kart; bir seçimde sandık örnekleme. Anakütle örnekleme oranına göre çok büyükse ($n/N < 0,05$) binom ile hipergeometrik neredeyse aynıdır — pratikte bu yüzden binom kullanılır.

Dağılım	PMF	$\mathbb{E}$, Var	Modellediği
Bernoulli	$p^k(1-p)^{1-k}$	$p, p(1-p)$	Tek deneme: başarı/başarısızlık.
Binom	$\binom{n}{k} p^k (1-p)^{n-k}$	$np, np(1-p)$	n bağımsız denemede başarı sayısı.
Poisson	$\frac{\lambda^k e^{-\lambda}}{k!}$	λ, λ	Sabit aralıkta nadir olay sayısı.
Geometrik	$(1-p)^{k-1} p$	$\frac{1}{p}, \frac{1-p}{p^2}$	İlk başarıya kadar deneme sayısı.
Negatif binom	$\binom{k-1}{r-1} p^r (1-p)^{k-r}$	$\frac{r}{p}$	r başarıya kadar deneme sayısı.
Hipergeometrik	$\frac{\binom{K}{k} \binom{N-K}{n-k}}{\binom{N}{n}}$	$n \frac{K}{N}$	İadesiz çekilişte başarı sayısı.

```

1 b = st.binom(n=10, p=0.3)
2 print("pmf(0..5):", np.round([b.pmf(k) for k in range(6)], 4))
3 print(f"P(X<=3) = {b.cdf(3):.4f}    mean = {b.mean():.2f}    var = {b.var():.2f}")
4
5 po = st.poisson(mu=3)

```

```
6 print("pmf(0..5):", np.round([po.pmf(k) for k in range(6)], 4))
7 print(f"P(X>=5) = {po.sf(4):.4f}")
```

```
pmf(0..5): [0.0282 0.1211 0.2335 0.2668 0.2001 0.1029]
P(X<=3) = 0.6496    mean = 3.00    var = 2.10
pmf(0..5): [0.0498 0.1494 0.224 0.224 0.168 0.1008]
P(X>=5) = 0.1847
```

Poisson Yaklaşımı

n büyük, p küçük ve $np = \lambda$ sabitken

$$\text{Binom}(n, p) \rightarrow \text{Poisson}(\lambda).$$

Pratik ölçüt: $n \geq 20$ ve $p \leq 0,05$. Bu yüzden “günde ortalama 3 arıza” gibi *sayım* verileri Poisson ile modellenir.

Nedir? Niçin? Nerede?

Bu yaklaşım niçin var? Binom formülü $\binom{n}{k}$ içerir; $n = 10\,000$ için bu sayı astronomiktir ve elle hesaplanamaz. Poisson ise yalnızca $\lambda = np$ 'ye bağlıdır — tek parametre. Ama asıl önemli olan kavramsal taraf: **Poisson, “ n 'yi bilmediğin binomdur”**.

Bir web sunucusuna gelen istekleri düşün. “Bu dakikada kaç kişi siteme girer?” sorusunda n nedir — internete bağlı 5 milyar insan mı? p nedir — her birinin bu dakikada girme olasılığı mı? İkisini de ölçemezsin. Ama çarpımlarını, yani *dakikada ortalama kaç istek geldiğini* kolayca ölçersin. Poisson tam olarak bunu kullanır: n ve p 'yi ayrı ayrı bilmene gerek yok, çarpımları yeter. **Poisson'un üç varsayımı** (ihlal edilirse model bozulur): (1) olaylar bağımsızdır — biri diğerini tetiklemez; (2) hız sabittir — λ zaman içinde değişmez; (3) aynı anda iki olay olmaz.

Nerede kırılır? Sosyal medyada bir gönderi viral olduğunda paylaşımlar birbirini tetikler (bağımsızlık ihlali); çağrı merkezine gelen çağrılar öğle saatinde yoğunlaşır (sabit hız ihlali). Her iki durumda da gözlenen varyans ortalamadan büyük çıkar — *aşırı yayılım*. Teşhis kolaydır: Var /E oranını hesapla; 1'e yakınsa Poisson uygundur, belirgin biçimde büyükse negatif binoma geç.

Poisson'un Klasik Üç Örneği

- **Prusya süvari istatistikleri (1898):** Bortkiewicz, 20 yıl boyunca 14 Prusya süvari birliğinde “at tepmesinden ölen asker sayısını” inceledi. Yılda ortalama 0,7 ölüm; gözlenen dağılım Poisson'a şaşırtıcı doğrulukta uydu. Nadir ve bağımsız olayların ders kitabı örneğidir.
- **Kapasite planlama:** Bir çağrı merkezine saatte ortalama 40 çağrı geliyorsa, “%99 ihtimalle yeterli olacak temsilci sayısı” Poisson kuyruğundan hesaplanır. Ortalamaya göre personel planlarsan zamanın yarısında yetersiz kalırsın — **ortalamaya göre değil, kuyruğa göre planlanır**.
- **A/B testinde sayım metrikleri:** “Kullanıcı başına görüntülenen sayfa sayısı” gibi metrikler sayımdır. t -testi yerine Poisson (ya da aşırı yayılım varsa negatif binom) regresyonu kullanmak, çok daha doğru güven aralıkları verir.

Gerçek Kullanım

Bir web sunucusuna dakikada ortalama 3 istek geliyorsa, bir dakikada 5'ten fazla istek gelme olasılığı $\mathbb{P}(X \geq 5) = 0,1847$ 'dir — yani her 5–6 dakikada bir. Kapasite planlaması tam olarak bu hesaba dayanır. Poisson'un ayırt edici özelliği $\mathbb{E}[X] = \text{Var}(X)$ olmasıdır; gerçek veride varyans ortalamadan büyükse (*aşırı yayılım*) negatif binom kullanılır.

16. Sürekli Dağılımlar

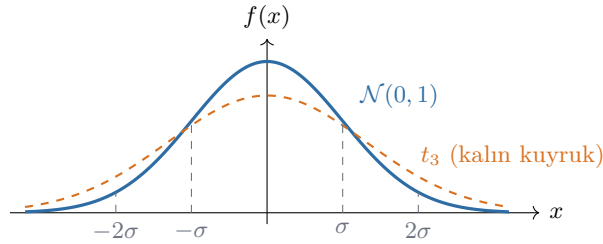
Her Birinin Doğduğu Yer

Süreklili dağılımlar da rastgele icat edilmiş formüller değildir; her biri belirli bir mekanizmanın sonucudur. Aşağıdaki tabloyu okumadan önce bu haritayı aklında tut:

- **Normal** — *çok sayıda küçük ve bağımsız etkinin toplamı*. Bir insanın boyu yüzlerce genin ve çevresel etkenin toplamıdır; bir ölçüm hatası, sayısız minik hatanın toplamıdır. Merkezi Limit Teoremi bunun niçin kaçınılmaz olduğunu söyler. Normal dağılımın her yerde olmasının sebebi bir “doğa kanunu” değil, *toplama işleminin* matematiğidir.
- **Log-normal** — *çok sayıda küçük etkinin çarpımı*. Logaritmasını alınca çarpım toplama dönüşür ve normal olur. Gelir, ev fiyatı, şirket büyüklüğü, dosya boyutu, bir web sayfasında geçirilen süre: hepsi çarpımsal büyür (“%10 zam” toplamsal değil çarpımsaldır), bu yüzden hepsi log-normaldir.
- **Üstel** — *hafızasız bekleme*. “Bir sonraki müşteri ne zaman gelir?” Hafızasızlık şu demektir: 10 dakikadır beklediysen, daha 5 dakika bekleme olasılığın hiç beklememiş gibi aynıdır. Ampul ömrü için gerçekçi değildir (ampuller yaşlanır) ama radyoaktif bozunma ve rastgele gelen istekler için tam doğrudur. Poisson'un ikizidir: olay *sayısı* Poisson ise, olaylar arası *süre* üsteldir.
- **Gamma** — α *tane üstel beklemenin toplamı*. “Beşinci müşteri ne zaman gelir?” Sigortacılıkta toplam hasar, kuyruk teorisinde toplam bekleme, yağış modellerinde birikimli miktar.
- **Beta** — $[0, 1]$ *aralığındaki bir oran hakkındaki inanç*. Tıklama oranı, dönüşüm oranı, bir oyuncunun isabet yüzdesi. Bayesçi çıkarımda binomun eşlenik önselidir — yani “şu ana kadar 7 tıklama / 100 gösterim gördüm” bilgisini doğrudan bir Beta dağılımına çevirebilirsin.
- χ^2 , t , F — bunlar veriyi değil, *test istatistiklerini* modeller. Normal örneklemelerden türetilirler: χ^2 karelerin toplamıdır (varyans çıkarımı), t “ σ bilinmediğinde normalin şişmiş hâlidir” (küçük örneklem), F iki varyansın oranıdır (ANOVA). Kısım 6'da üçünü de iş başında göreceğiz.
- **Düzgün** — *hiçbir değeri diğerine tercih etmeme*. Tüm rastgele sayı üreteçlerinin çıkış noktasıdır; her dağılım $U(0, 1)$ 'den ters dönüşümle üretilebilir.

Pratik teşhis: Verin *negatif olabiliyorsa* ve simetrikse normal; *yalnızca pozitifse* ve sağa çarpıksa log-normal veya gamma; $0-1$ *arasındaysa* beta; *bekleme süresiyse* üstel. Bu dört soru vakaların büyük kısmını çözer.

Dağılım	$\mathbb{E}, \text{Var}$	Nerede karşına çıkar?
Düzgün $U(a, b)$	$\frac{a+b}{2}, \frac{(b-a)^2}{12}$	Rastgele sayı üretimi, bilgisiz önsel.
Normal $\mathcal{N}(\mu, \sigma^2)$	μ, σ^2	Ölçüm hataları, MLT'nin limiti, artıklar.
Üstel $\text{Exp}(\lambda)$	$\frac{1}{\lambda}, \frac{1}{\lambda^2}$	Olaylar arası süre; hafızasızdır.
Gamma (α, β)	$\frac{\alpha}{\beta}$	α olayın toplam süresi; bekleme süreleri.
Beta (α, β)	$\frac{\alpha}{\alpha+\beta}$	$[0, 1]$ 'de oranlar; Bayesçi önsel.
Ki-kare χ_k^2	$k, 2k$	Varyans çıkarımı, uyum iyiliği testleri.
Student t_ν	$0, \frac{\nu}{\nu-2}$	Küçük örnekleme ortalama; kalın kuyruklu.
$F(d_1, d_2)$	$\frac{d_2}{d_2-2}$	Varyans oranı; ANOVA ve regresyon testi.
Log-normal	$e^{\mu+\sigma^2/2}$	Gelir, fiyat, süre — çarpımsal etkiler.



16.1 Gerçek Veriye Dağılım Uydurmak

Normallik Testleri Ne Yapar, Ne Yapmaz?

Aşağıdaki üç test de aynı H_0 'ı sınar: “bu veri normal dağılımdan geliyor.” Farkları, dağılımın neresine baktıklarıdır:

- **Kolmogorov–Smirnov:** Gözlenen birikimli dağılım ile teorik CDF arasındaki *en büyük dikey farka* bakar. Genel amaçlıdır ama kuyruklarda duyarsızdır.
- **Shapiro–Wilk:** Özellikle normallik için tasarlanmıştır ve küçük örneklerde en güçlü testtir. Varsayılan tercih budur.
- **Anderson–Darling:** Kuyruklara ağırlık verir. Uç değerlerin önemli olduğu durumlarda (risk analizi, sigortacılık) daha uygundur.

Niçin normallığı umursuyoruz? Çünkü t -testi, ANOVA, doğrusal regresyonun güven aralıkları ve p -değerleri normallik varsayımı üzerine kuruludur. Ama — kritik nokta — bu yöntemler *verinin* değil, *istatistiğin* normallikini gerektirir ve MLT bunu büyük örneklerde zaten sağlar.

Bu testlerin büyük tuzağı. Örneklem büyüdükçe testler *her* sapmayı yakalar. $n = 5000$ 'de, pratikte normalden ayırt edilemeyecek bir veri bile $p < 0,001$ ile reddedilir. Tersine $n = 15$ 'te, açıkça çarpık bir veri bile reddedilmez — test zayıftır. Yani normallik testi, tam da ona en çok ihtiyaç duyduğün küçük örneklerde işe yaramaz, hiç gerekmediği büyük örneklerde ise sürekli alarm verir.

O hâlde ne yapmalı? *Teste değil grafiğe bak.* Q–Q grafiği sana yalnızca “normal mi?” değil, *nasıl* normal olmadığını da gösterir: noktalar S çiziyorsa çarpıklık, uçlarda kıvrılıyorsa kalın kuyruk, basamaklıysa kesikli veri var demektir. Sonra sor: “bu sapma, kullanacağım yöntemi gerçekten bozar mı?” Çoğu zaman cevap hayırdır.

Nerede gerçekten kullanılır? Süreç yeterlilik analizinde (kalite kontrolde C_{pk} hesabı normallik gerektirir); finansal risk modellerinde kalın kuyruk tespitinde; ve bir dağılıma parametrik model uydurmadan önce uyum kontrolünde.

```
1 import seaborn as sns
2 peng = sns.load_dataset('penguins').dropna()
3 x = peng['body_mass_g']
4
5 mu, sd = st.norm.fit(x) # maksimum olabilirlik tahmini
6 print(f"MLE: mu = {mu:.1f}    sigma = {sd:.1f}")
7
8 # Uyum iyiliği: uc farklı test
9 ks = st.kstest(x, st.norm(mu, sd).cdf)
10 print(f"Kolmogorov-Smirnov: D = {ks.statistic:.4f}    p = {ks.pvalue:.4f}")
11 sh = st.shapiro(x)
12 print(f"Shapiro-Wilk:      W = {sh.statistic:.4f}    p = {sh.pvalue:.2e}")
13 ad = st.anderson(x, 'norm')
14 print(f"Anderson-Darling:  A2 = {ad.statistic:.3f}    kritik(5) = {ad.critical_values[2]}")
```

```
MLE: mu = 4207.1    sigma = 804.0
Kolmogorov-Smirnov: D = 0.1059    p = 0.0010
Shapiro-Wilk:      W = 0.9580    p = 3.57e-08
Anderson-Darling:  A2 = 4.615    kritik(5) = 0.75
```

Dikkat

Üç test de normalliği reddediyor — ama bu, “normal varsayan hiçbir yöntem kullanılamaz” demek değildir. Büyük örneklerde normallik testleri *önemsiz* sapmaları bile yakalar; $n = 333$ 'te kalın kuyruk olmasa da iki tepelilik (Kısım 2'de gördüğümüz tür karışımı) reddettirir. Doğru yaklaşım: teste değil, **Q–Q grafiğine** ve yöntemin sağlamlığına bak. t -testi ve ANOVA, Merkezi Limit Teoremi sayesinde $n > 30$ 'da normallikten sapmalara oldukça dayanıklıdır.

```
1 import matplotlib.pyplot as plt
2 fig, (a1, a2) = plt.subplots(1, 2, figsize=(11, 4))
3 sns.histplot(x, kde=True, stat='density', ax=a1, color='#2E6DA4')
4 xs = np.linspace(x.min(), x.max(), 200)
5 a1.plot(xs, st.norm(mu, sd).pdf(xs), 'r--', lw=2, label='normal uyum')
6 a1.legend(); a1.set_title('Histogram + uydurulan normal')
7 st.probplot(x, dist='norm', plot=a2) # Q-Q grafiği
8 a2.set_title('Q-Q grafiği')
9 plt.tight_layout(); plt.show()
```

16.2 Çarpık Veri: Log-normal Dönüşüm

```
1 tit = sns.load_dataset('titanic')
2 fare = tit.loc[tit.fare > 0, 'fare']
3 print(f"n={len(fare)} mean={fare.mean():.2f} median={fare.median():.2f} "
4       f"skew={fare.skew():.2f}")
5
6 s_, loc_, scale_ = st.lognorm.fit(fare, floc=0)
7 print(f"lognormal uyum: s = {s_:.3f} scale = {scale_:.2f} -> mu = {np.log(scale_):.3f}")
8 print(f"log(fare) carpıklığı = {np.log(fare).skew():.3f}")
```

```
n=876 mean=32.76 median=14.50 skew=4.77
lognormal uyum: s = 0.936 scale = 18.98 -> mu = 2.943
log(fare) carpıklığı = 0.901
```

Bilgi

Bilet ücretinin çarpıklığı 4,77 (aşırı sağa çarpık); logaritmasını alınca 0,90'a iniyor. **Log dönüşümü** çarpımsal süreçlerin (fiyat, gelir, nüfus, süre) standart tedavisidir: çarpımsal etkileri toplamsal hâle getirir, varyansı dengeler ve doğrusal modellerin varsayımlarını sağlar. Sıfır değerler varsa np.log1p kullan.

17. Büyük Sayılar Yasası ve Merkezi Limit Teoremi**Büyük Sayılar Yasası (BSY)**

$X_1, \dots, X_n \stackrel{\text{iid}}{\sim}$ ve $\mathbb{E}[X] = \mu$ sonlu ise

$$\bar{X}_n \xrightarrow{\mathbb{P}} \mu \quad (n \rightarrow \infty).$$

Örneklem ortalaması, anakütle ortalamasına yakınsar.

Nedir? Niçin? Nerede?

BSY ne diyor? “Yeterince çok ölçersen, ortalamanın gerçeğe yaklaşır.” Kulağa apaçık geliyor — ama bu, **tüm istatistiğin varlık sebebidir**. Eğer bu teorem doğru olmasaydı, bir örneklemden anakütle hakkında hiçbir şey söyleyemezdik; anket yapmanın, A/B testinin, model eğitiminin hiçbir anlamı olmazdı.

Nerede işe yarar?

- **Simülasyon:** Monte Carlo yönteminin tamamı BSY’ye dayanır — π ’yi rastgele nokta atarak hesaplayabilmemizin sebebi budur.
- **Kumarhane ekonomisi:** Tek bir masada kasa kaybedebilir; ama milyonlarca el sonunda ortalama, teorik beklenen değere yakınsar. Kasanın kârı şansa değil, BSY’ye dayanır.
- **Sigortacılık:** Tek bir poliçe sahibinin hasarı öngörülemez; 500.000 poliçenin ortalama hasarı ise şaşırtıcı doğrulukla öngörülebilir. Sigortacılığın iş modeli budur.
- **Model değerlendirme:** Test kümesi büyüdükçe ölçtüğün doğruluk, gerçek performansa yakınsar. 50 örneklik test kümesiyle ölçülen %92 doğruluk gürültüdür; 50.000 örnekle ölçülen %92 bilgidir.

Dikkat

Kumarbaz yanılgısı — BSY’nin en yaygın yanlış okunuşu. Rulette arka arkaya 10 kez kırmızı geldi; “artık siyah gelmeli, denge kurulmalı” düşüncesi yanlıştır. BSY, geçmişin *telaflı edileceğini* söylemez — gelecekteki milyonlarca atışın o 10 atışı *seyrelteceğini* söyler. Çark hafızasızdır. Aynı hata veri biliminde de görülür: “son üç aydır dönüşüm oranımız düşük, ortalamaya döner” cümlesi ancak altta yatan süreç değişmediyse doğrudur.

```
1 rng = np.random.default_rng(7)
2 for n in [10, 100, 1000, 10_000, 100_000]:
3     s = rng.integers(1, 7, n) # adil zar, gerçek ortalama 3.5
4     print(f"n = {n:>6}: ortalama = {s.mean():.4f}")
```

```
n =      10: ortalama = 4.1000
n =     100: ortalama = 3.5900
n =    1000: ortalama = 3.5340
n =   10000: ortalama = 3.4953
n =  100000: ortalama = 3.5065
```

Merkezi Limit Teoremi (MLT)

$X_i \stackrel{\text{iid}}{\sim}, \mathbb{E}[X] = \mu, \text{Var}(X) = \sigma^2 < \infty$ ise *dağılım ne olursa olsun*

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} \mathcal{N}(0, 1).$$

Ortalamanın standart sapması — **standart hata** — $\sigma/\sqrt{n}$ olarak küçülür.

Nedir? Niçin? Nerede?

MLT niçin “merkezi”? Adındaki “merkezi” kelimesi “ortalama” anlamında değil, “*temel, merkezî öneme sahip*” anlamındadır — ve haklı olarak. Söylediği şu: elindeki verinin dağılımı ne olursa olsun (çarpık, iki tepeli, kesikli, ne olursa), yeterince gözlemin *ortalaması* normal dağılır.

Bu niçin mucize gibi? Yukarıdaki kodda anakütle üstel dağılımlı — son derece çarpık, hiç normale benzemiyor (çarpıklık 1,99). Ama 100’erlik gruplar alıp ortalamalarına baktığında çarpıklık 0,23’e düşüyor ve elinde neredeyse mükemmel bir çan eğrisi kalıyor. Verinin şekli hakkındaki bilgi *buharlaşıyor* ve geriye yalnızca iki sayı kalıyor: μ ve $\sigma/\sqrt{n}$.

Pratik sonucu nedir? Bu teorem sayesinde:

- Gelir gibi aşırı çarpık bir veride bile *ortalama gelir* için normal tabanlı güven aralığı kurabilirsin.
- *t*-testi, ANOVA, regresyonun *p*-değerleri — hepsi “veri normal” değil, “*istatistik* normal” varsayımına dayanır ve MLT bunu sağlar.
- Bir A/B testinde dönüşüm oranı 0/1 gibi kesikli bir veriden gelir; yine de ortalamalarının farkı normal kabul edilebilir.

$\sqrt{n}$ niçin bu kadar önemli? Standart hatanın $\sigma/\sqrt{n}$ olması, **belirsizliği yarıya indirmek için örnekleme dört katına çıkarman gerektiği** anlamına gelir. Bu, veri toplamanın ekonomisini belirleyen tek formüldür: 400 kişilik bir ankette hata payı $\pm 5\%$ ise, $\pm 2,5\%$ istiyorsan 1600 kişi gerekir — bütçe dört katına çıkar. Aynı sebeple bir modelin doğruluğunu %1 daha hassas ölçmek için test kümesini büyütme hızla pahalılaşır.

Anket Sonuçlarındaki “ ± 3 Puan” Nereden Geliyor?

Seçim anketlerinde hep aynı cümleyi görürsün: “1000 kişiyle görüşüldü, hata payı ± 3 puan.” Bu sayı doğrudan MLT’den gelir.

Bir oranın standart hatası $\sqrt{p(1-p)/n}$ ’dir ve $p = 0,5$ ’te en büyük değerini alır: $\sqrt{0,25/1000} = 0,0158$. %95 güven için bunu 1,96 ile çarparsın: 0,031, yani $\pm 3,1$ puan. Anketi kim yaparsa yapsın, hangi ülkede olursa olsun, 1000 kişiyle hata payı hep budur.

Buradan çıkan iki pratik ders: (1) İki adayın farkı 2 puansa, anket “başa baş” demektir — manşetler bunu sürekli yanlış okur. (2) Hata payı *anakütle büyüklüğüne bağlı değildir*. 80 milyonluk bir ülkeyi de, 8 milyonluk bir şehri de 1000 kişiyle aynı hassasiyette ölçersin. Formülde N yok, yalnızca n var. Çorbanın tuzunu anlamak için bir kaşık yeter — tencerenin büyüklüğü fark etmez, yeter ki iyice karıştırılmış olsun (*rastgele örnekleme*).

```
1 rng = np.random.default_rng(1)
2 pop = rng.exponential(scale=2.0, size=1_000_000) # ustel: cok carpik
3 print(f"anakutle: mean={pop.mean():.3f} std={pop.std():.3f} "
4       f"skew={st.skew(pop):.3f}")
5
6 for n in [1, 5, 30, 100]:
7     means = rng.choice(pop, size=(20_000, n)).mean(axis=1)
8     print(f"n={n}>4: ort={means.mean():.3f} std={means.std():.4f} "
9           f"teorik SE={pop.std()/np.sqrt(n):.4f} carpiklik={st.skew(means):+.3f}")
```

```
anakutle: mean=1.996 std=1.992 skew=1.989
n= 1: ort=2.003 std=2.0013 teorik SE=1.9922 carpiklik=+1.966
n= 5: ort=1.996 std=0.8901 teorik SE=0.8909 carpiklik=+0.877
n= 30: ort=1.996 std=0.3619 teorik SE=0.3637 carpiklik=+0.353
n= 100: ort=1.998 std=0.1987 teorik SE=0.1992 carpiklik=+0.227
```

Bilgi

Anakütle çarpıklığı 1,99 iken, 100'lük ortalamaların çarpıklığı 0,23'e düşüyor ve standart sapma teorik $\sigma/\sqrt{n}$ ile neredeyse tam örtüşüyor. **MLT bu rehberdeki her güven aralığının ve t-testinin temelidir:** veri normal olmasa bile *ortalaması* yaklaşık normaldir.

Dikkat

MLT'nin iki sınırı vardır: (1) σ^2 sonlu olmalıdır — Cauchy gibi dağılımlarda MLT geçerli değildir; (2) yakınsama hızı çarpıklığa bağlıdır. Kaba kural $n > 30$ 'dur ama çok çarpık dağılımlarda $n > 200$ bile gerekebilir. Emin değilsen bootstrap kullan (Kısım 5).

18. Monte Carlo ve Örneklem Yöntemleri

Fikir Nedir?

Monte Carlo yönteminin tek cümlelik özeti şudur: **“Hesaplayamıyorsan, simüle et.”** Bir olasılığı, integrali ya da beklenen değeri analitik olarak çözmek yerine, süreci binlerce kez taklit edip sonuçların ortalamasını alırsın. BSY bu ortalamanın doğru cevaba yakınsayacağını garanti eder.

Aşağıdaki π örneği fikri çıplak hâliyle gösteriyor: birim kareye rastgele nokta at, çeyrek daire içine düşenlerin oranını bul, 4 ile çarp. Hiç geometri bilmene gerek yok — yalnızca rastgele sayı üretebilmen yeter.

Niçin bu kadar kullanışlı? Çünkü gerçek problemlerin çoğunun analitik çözümü *yoktur*. Formül yazılamayan yerde simülasyon her zaman çalışır. Ayrıca simülasyon kurmak, formül türetmekten çok daha az hata yapılan bir iştir — kodun kendisi modelin tarifidir.

Nerede kullanılır?

- **Finans:** Opsiyon fiyatlaması, portföy riski (VaR). “Bu portföy 1 yılda en kötü %1 senaryoda ne kaybeder?” sorusu 100.000 fiyat yolu simüle edilerek cevaplanır.
- **Proje planlama:** Her görevin süresi belirsizse, projenin toplam süresi için tek bir sayı değil bir *dağılım* elde edersin (“%90 ihtimalle 14 haftadan önce biter”).
- **Mühendislik:** Tolerans yığılması — 12 parçanın her birinin üretim toleransı verildiğinde montajın toplam sapması.
- **İstatistik:** Bootstrap (Kısım 5), permütasyon testi (Kısım 6), Bayesçi MCMC — üçü de birer Monte Carlo uygulamasıdır.
- **Yapay zekâ:** Monte Carlo Ağaç Araması (MCTS), AlphaGo'nun temel algoritmasıdır: hangi hamlenin iyi olduğunu hesaplamak yerine, o hamleden sonra binlerce oyunu rastgele oynatıp kazanma oranına bakar.

```
1 rng = np.random.default_rng(3)
2 for n in [10**3, 10**4, 10**5, 10**6]:
3     p = rng.random((n, 2))
4     pi = 4*np.mean((p**2).sum(axis=1) <= 1)
5     print(f"n = {n:>8}: pi ~ {pi:.5f}    hata = {abs(pi - np.pi):.5f}")
```

```
n =      1000: pi ~ 3.13200    hata = 0.00959
n =     10000: pi ~ 3.12800    hata = 0.01359
n =    100000: pi ~ 3.13508    hata = 0.00651
```

```
n = 1000000: pi ~ 3.14135 hata = 0.00024
```

Monte Carlo Yakınsama Hızı

Hata $O(1/\sqrt{n})$ ile azalır: basamak kazanmak için örnek sayısını *100 katına* çıkarman gerekir. Yavaşdır ama boyuttan bağımsızdır — bu yüzden yüksek boyutlu integrallerde tek pratik yöntemdir.

Nedir? Niçin? Nerede?

“Boyuttan bağımsız” niçin bu kadar önemli? Klasik sayısal integrasyon (yamuk kuralı, Simpson) tek boyutta Monte Carlo’dan çok daha hızlıdır. Ama d boyutta ızgara kurmak için her eksen m nokta gerekir, yani m^d hesap — 10 boyutta ve eksen başına 100 noktada bu 10^{20} eder, imkânsız. Buna **boyut laneti** denir.

Monte Carlo’nun hatası ise $O(1/\sqrt{n})$ ’dir ve bu ifadede d yoktur. 1 boyutta da 100 boyutta da aynı yavaşlıkta yakınsar. Yavaş ama boyuta duyarsız olması, yüksek boyutlu problemlerde onu tek seçenek hâline getirir — ve makine öğrenmesindeki hemen her problem yüksek boyutludur.

Pratik okuma: Sonucunun ilk basamağını doğru istiyorsan ~ 100 örnek, iki basamak için $\sim 10\,000$, üç basamak için $\sim 10^6$ örnek gerekir. Bu yüzden bootstrap’ta 1000–10.000 tekrar standarttır: karar vermek için gereken hassasiyet zaten o civardadır, daha fazlası boşa hesaptır.

18.1 Ters Dönüşüm ve Kabul-Ret Yöntemleri

```
1 # 1) Ters donusum:  $U \sim \text{Uniform}(0,1)$  ise  $F^{-1}(U) \sim F$ 
2 u = rng.random(100_000)
3 ustel = -np.log(1 - u)/0.5 # Exp(lambda=0.5)
4 print(f"ortalama = {ustel.mean():.3f}") # 1.998 (teorik 2.0)
5
6 # 2) Kabul-ret: hedef yogunluk f, onerici g,  $f \leq M \cdot g$ 
7 def kabul_ret(f, n, a=0, b=1, M=2.0, rng=rng):
8     kabul = []
9     while len(kabul) < n:
10         x = rng.uniform(a, b, n)
11         u = rng.random(n)
12         kabul.extend(x[u < f(x)/M])
13     return np.array(kabul[:n])
14
15 ornek = kabul_ret(lambda x: 2*x, 50_000) #  $f(x) = 2x, 0 < x < 1$ 
16 print(f"ortalama = {ornek.mean():.4f}") # 0.6664 (teorik 2/3)
```

İpucu

`numpy.random` zaten tüm standart dağılımlar için hızlı üreteçlere sahiptir; ters dönüşüm ve kabul-ret yöntemlerini öğrenmek için değil, *standart olmayan* bir dağılımdan örneklem gerektiğinde kullanırsın. Bayesçi çıkarımda (Kısım 6) aynı fikir MCMC’ye dönüşür.

KISIM 5

Çıkarımsal İstatistik: Tahmin

19. Örnekleme Dağılımı ve Standart Hata

Elimizde *anakütle* değil *örneklem* vardır. Örneklemden hesapladığımız her istatistik $(\bar{x}, s, r, \hat{p} \dots)$ bir rastgele değişkendir ve kendi dağılımına sahiptir: **örnekleme dağılımı**.

Bu Kısım Niçin Var?

Buraya kadar veriyi *betimledik* (Kısım 2) ve belirsizliğin dilini kurduk (Kısım 3–4). Şimdi asıl soruya geliyoruz: **elimizdeki 333 pengüenden, dünyadaki bütün pengüenler hakkında ne söyleyebiliriz?** Bu sıçramaya *çıkarım* denir ve istatistiğin var oluş sebebidir.

Sıçramanın bedeli belirsizliktir. 333 pengüenin ortalaması 4207 g çıktı — ama başka 333 pengüen ölçseydik 4180 ya da 4240 çıkardı. İşte **örnekleme dağılımı**, “bu deneyi tekrar tekrar yapsaydım hesapladığım sayı nasıl oynardı?” sorusunun cevabıdır. Bu kısımdaki her şey — standart hata, güven aralığı, bootstrap — o oynamanın büyüklüğünü ölçmenin farklı yollarıdır.

Üç dağılımı asla karıştırma: (1) *Anakütle dağılımı* — tüm pengüenlerin ağırlıkları. (2) *Örneklem dağılımı* — elindeki 333 gözlemin histogramı. (3) *Örnekleme dağılımı* — tekrarlanan örneklemelerden hesaplanan *ortalamaların* dağılımı. Üçüncüsü hayalîdir, onu asla göremezsin — ama Merkezi Limit Teoremi şekli, standart hata genişliğini verir. Çıkarımın tamamı bu görünmez dağılım üzerine kuruludur.

Standart Hata

Bir istatistiğin örnekleme dağılımının standart sapmasına *standart hata* denir. Ortalama için:

$$SE(\bar{X}) = \frac{\sigma}{\sqrt{n}} \approx \frac{s}{\sqrt{n}}.$$

Standart sapma verinin yayılımını, **standart hata** tahminin belirsizliğini ölçer. İkisini karıştırmak, bilimsel makalelerdeki en yaygın hatadır.

Nedir? Niçin? Nerede?

Farkı bir cümlede: Standart sapma *penguenler* hakkında, standart hata *senin ölçümün* hakkındadır.

Somutlaştıralım: $s = 805$ g, “penguenler birbirinden ne kadar farklı?” sorusunu cevaplar — ve daha çok penguen ölçersen bu sayı *değişmez*, çünkü penguenlerin çeşitliliği senin kaç tane ölçtüğüne bağlı değildir. $SE = 44$ g ise “hesapladığım 4207 rakamı ne kadar güvenilir?” sorusunu cevaplar — ve n büyüdükçe *sıfıra gider*, çünkü daha çok ölçüm daha kesin bir ortalama demektir.

Niçin karıştırmak ciddi bir hata? Bir grafikte hata çubuklarını $\pm s$ yerine $\pm SE$ çizersen çubuklar $\sqrt{n}$ kat daha kısa görünür; iki grubun çubukları ayrılmış gibi durur ve okuyucu var olmayan bir fark görür. Bilimsel dergilerin “hata çubuğunun neyi gösterdiğini yazın” diye ısrar etmesinin sebebi budur.

Hangisini ne zaman raporlarsın?

- *Bireylerin çeşitliliğini* anlatıyorsan (s): “Hastaların kan basıncı 120 ± 15 mmHg aralığında değişiyor.” Bir sonraki hastanın nerede olacağını tahmin etmek için bu gerekir.
- *Tahmininin kesinliğini* anlatıyorsan (SE): “Ortalama kan basıncı $120 \pm 1,5$ mmHg.” Güven aralığı, hipotez testi, A/B testi kararı — hepsi SE ile çalışır.

Nerede karşına çıkar? Anketlerdeki “hata payı”; regresyon çıktısındaki her katsayının yanındaki std err sütunu; makine öğrenmesinde çapraz doğrulama skorlarının $\pm$ ’si; ilaç denemelerinde etki büyüklüğünün güven aralığı. Hepsi standart hatadır.

```
1 import numpy as np, seaborn as sns
2 peng = sns.load_dataset('penguins').dropna()
3 x = peng['body_mass_g'].to_numpy()
4 n = len(x)
5
6 print(f"n = {n}")
7 print(f"ortalama = {x.mean():.1f} g")
8 print(f"std      = {x.std(ddof=1):.1f} g      <- verinin yayilimi")
9 print(f"SE       = {x.std(ddof=1)/np.sqrt(n):.2f} g      <- tahminin belirsizligi")
```

```
n = 333
ortalama = 4207.1 g
std      = 805.2 g      <- verinin yayilimi
SE       = 44.13 g      <- tahminin belirsizligi
```

Dikkat

$\sqrt{n}$ kuralı acımasızdır: belirsizliği yarıya indirmek için örnekleme **dört katına** çıkarman gerekir. 333 penguen yerine 1332 penguen ölçersen SE 44,13 $\rightarrow$ 22,07’ye iner. Bu, deney tasarımı maliyetin neden hızla arttığının matematiksel nedenidir.

20. Nokta Tahmini: MLE ve Momentler Yöntemi

Maksimum Olabilirlik (MLE)

Veriyi *en olası* kılan parametreyi seç:

$$\hat{\theta}_{MLE} = \arg \max_{\theta} \mathcal{L}(\theta | x) = \arg \max_{\theta} \prod_i f(x_i | \theta) = \arg \max_{\theta} \sum_i \log f(x_i | \theta).$$

Çarpım yerine logaritma toplamı kullanılır: hem sayısal taşmayı önler hem de türev almayı kolaylaştırır.

Nedir? Niçin? Nerede?

Fikir nedir? Elinde veri var, ama onu üreten parametreyi bilmiyorsun. MLE şu soruyu sorar: “*Hangi parametre değeri, tam olarak bu veriyi görme ihtimalimi en yüksek yapar?*” Yani mantığı tersine çevirir: normalde parametreden veriye gideriz, MLE veriden parametreye gider.

Somut örnek. Bir madenî parayı 10 kez attın, 7 tura geldi. Tura olasılığı p nedir? $p = 0,1$ olsaydı bu sonucu görme şansın binde bir bile değil; $p = 0,9$ olsaydı da düşük. $p = 0,7$ 'de olabilirlik en yüksek çıkar. MLE cevabı $\hat{p} = 0,7$ 'dir — ve bu, sezgisel olarak yapacağın şeyin ta kendisidir. MLE'nin güzelliği, bu sezgiyi *her* dağılım için işleyen bir tarife dönüştürmesidir.

Olabilirlik ile olasılık aynı şey değildir. Olasılıkta parametre sabit, veri değişkendir (“ $p = 0,5$ ise 7 tura gelme olasılığı?”). Olabilirlikte veri sabit, parametre değişkendir (“7 tura gördüm, hangi p bunu en olası kılar?”). $\mathcal{L}(\theta | x)$ bir olasılık dağılımı *değildir*; θ üzerinden integrali 1 etmez.

Niçin logaritma? İki sebep: (1) 1000 gözlemin olasılıklarını çarparsan sonuç 10^{-3000} mertebesinde olur ve bilgisayarda sıfıra yuvarlanır (*underflow*); logaritma bunu toplamaya çevirip sorunu bitirir. (2) Türev almak çarpımda zincir kuralı yığını demektir, toplamda ise terim terim türevidir. Kayıp fonksiyonlarının hep “log” ile başlamasının sebebi budur.

Nerede kullanılır — yani neredeyse her yerde:

- **Lojistik regresyon** tam olarak MLE ile eğitilir; minimize ettiği *log-loss* (çapraz entropi), negatif log-olabilirliktir.
- **Doğrusal regresyon:** Hataların normal olduğu varsayımı altında MLE, en küçük kareler yöntemiyle *birebir aynı* sonucu verir. “Kareleri niçin topluyoruz?” sorusunun gerçek cevabı budur.
- **Sinir ağları:** Sınıflandırmada çapraz entropi, regresyonda MSE kaybı — ikisi de negatif log-olabilirliktir. Bir ağı eğitmek, MLE yapmaktır.
- **Dağılım uydurma:** `scipy`'deki `dist.fit(data)` çağrısı MLE çalıştırır.
- **Zaman serileri:** ARIMA, GARCH ve durum-uzayı modelleri MLE ile kestirilir.

Kayıp Uçağı MLE ile Aramak

Bir hava aracının son bilinen konumu ve hız verileri var. “Nereye düştü?” sorusu bir MLE problemidir: her olası düşüş noktası için “bu nokta doğruysa, elimdeki radar ve uydu sinyallerini görme olasılığım ne?” hesaplanır; olabilirliği en yüksek nokta arama merkezine seçilir.

Aynı mantığı her gün kullanan bir sistem daha var: telefonundaki **GPS**. Uydulardan gelen sinyal süreleri gürültülüdür; alıcı, “hangi konum bu ölçüm setini en olası kılar?” sorusunu çözerek konumunu bulur. Konum bulma, gizli bir MLE'dir.

Normal dağılım için türetim

$\log \mathcal{L} = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum (x_i - \mu)^2$ ifadesinin türevlerini sıfıra eşitlersek:

$$\frac{\partial}{\partial \mu} : \sum (x_i - \mu) = 0 \Rightarrow \hat{\mu} = \bar{x}, \quad \frac{\partial}{\partial \sigma^2} : \hat{\sigma}^2 = \frac{1}{n} \sum (x_i - \bar{x})^2.$$

Dikkat: MLE varyans tahmini n 'e böler, yani **yanlıdır**.

```
1 from scipy import stats as st
2 from scipy.optimize import minimize
3
4 def neg_loglik(params, data):
5     mu, log_sigma = params                                # log_sigma: sigma>0 kisitini kaldırır
6     return -np.sum(st.norm.logpdf(data, mu, np.exp(log_sigma)))
7
8 res = minimize(neg_loglik, x0=[4000, np.log(500)], args=(x,), method='Nelder-Mead')
9 print(f"sayisal MLE : mu = {res.x[0]:.2f}    sigma = {np.exp(res.x[1]):.2f}")
10 print(f"kapalı form : mu = {x.mean():.2f}    sigma = {x.std(ddof=0):.2f}")
```

```
sayisal MLE : mu = 4207.06    sigma = 804.01
kapalı form : mu = 4207.06    sigma = 804.01
```

Momentler Yöntemi

Teorik momentleri örneklem momentlerine eşitle: $\mathbb{E}[X] = \bar{x}$, $\mathbb{E}[X^2] = \overline{x^2}$, ... MLE'den daha basit ama genellikle daha az verimlidir. Bazı dağılımlarda (örn. gamma) hızlı bir başlangıç değeri üretmek için kullanılır.

Nedir? Niçin? Nerede?

Fikir çok basit: Dağılımın teorik ortalaması $\mu(\theta)$ parametreye bağlıdır. Elindeki verinin ortalaması $\bar{x}$ ise, “bu ikisi eşit olsun” deyip θ 'yı çöz. Kaç parametre varsa o kadar moment denklemi yazarsın.

Örnek: Bir üstel dağılımda $\mathbb{E}[X] = 1/\lambda$ 'dır. Bekleme süreleri ortalaman 4 dakikaysa $\hat{\lambda} = 1/4$ dersin. Bitti — optimizasyon yok, türev yok.

Niçin MLE varken buna ihtiyaç var? Üç durumda: (1) MLE'nin kapalı çözümü yoksa ve sayısal optimizasyon bir başlangıç noktası istiyorsa (gamma dağılımı klasik örnektir); (2) MLE hesaplaması çok pahalıysa ve kaba bir tahmin yeterliyse; (3) Dağılımın tam biçimini varsaymak istemeyip yalnızca momentlere dayanmak istediğinde — ekonometride *Genelleştirilmiş Momentler Yöntemi* (GMM) bu fikrin ciddi bir genellemesidir ve Nobel getirmiştir.

Bedeli ne? Genellikle MLE'den daha büyük varyanslı tahminler verir, yani aynı veriden daha az bilgi çıkarır. Bazen anlamsız sonuç bile üretebilir (parametre tanım aralığının dışına düşebilir). Kural: MLE hesaplanabiliyorsa MLE kullan.

20.1 İyi Bir Tahmin Edici Nedir?

Özellik	Tanım	Anlamı
Yansızlık	$\mathbb{E}[\hat{\theta}] = \theta$	Ortalamada doğruyu vurur.
Tutarlılık	$\hat{\theta} \xrightarrow{\mathbb{P}} \theta$	n büyüdükçe doğruya yakınsar.
Verimlilik	$\min \text{Var}(\hat{\theta})$	Aynı yansızlıkta en küçük varyans.
MSE	$\mathbb{E}[(\hat{\theta} - \theta)^2] = \text{yanlılık}^2 + \text{Var}$	Toplam hata ölçüsü.

```
1 # Bessel düzeltmesi gerçekten yanlılığı gideriyor mu? Simüle edelim.
2 rng = np.random.default_rng(42)
3 b_n, b_n1 = [], []
4 for _ in range(20_000):
```

```

5     s = rng.normal(0, 10, 10)                # gerçek varyans = 100
6     b_n.append(s.var(ddof=0))                # n'e bolen
7     b_n1.append(s.var(ddof=1))              # n-1'e bolen
8     print(f"ddof=0 ortalama = {np.mean(b_n):6.2f}    yanlilik = {np.mean(b_n)-100:+.2f}")
9     print(f"ddof=1 ortalama = {np.mean(b_n1):6.2f}    yanlilik = {np.mean(b_n1)-100:+.2f}")

```

```

ddof=0 ortalama =  90.57    yanlilik = -9.43
ddof=1 ortalama = 100.63    yanlilik = +0.63

```

Bilgi

$n = 10$ 'da n 'e bölmek varyansı sistematik olarak %10 küçük tahmin ediyor — tam olarak $(n - 1)/n = 0,9$ oranında. Bessel düzeltmesi bunu gideriyor. **Yanlılık–varyans dengesi** burada başlar ve Kısım 8'de makine öğrenmesinin merkezî kavramı olarak geri döner: bazen *yanlı* ama düşük varyanslı bir tahmin (Ridge regresyonu gibi), yansız ama oynak bir tahminden daha iyidir.

21. Güven Aralıkları

Ortalama için Güven Aralığı

$$\bar{x} \pm t_{\alpha/2, n-1} \cdot \frac{s}{\sqrt{n}}$$

σ bilinmediği için z yerine t dağılımı kullanılır; n büyüdükçe $t \rightarrow z$ olur.

Nedir? Niçin? Nerede?

Niçin tek bir sayı yetmiyor? “Ortalama penguen ağırlığı 4207 g” cümlesi eksiktir; hiçbir ölçüm sonsuz kesinlikte değildir. Güven aralığı, tahmine *dürüstlük* ekler: “4207, ama gerçek değer büyük olasılıkla 4120 ile 4294 arasında.” Karar veren kişi için ikinci cümle birincisinden kat kat kullanışlıdır.

Formülün anatomisi. Üç parça var ve her biri ayrı bir şey söyler:

- $\bar{x}$ — *en iyi tahminim*, aralığın merkezi.
- $s/\sqrt{n}$ — *standart hata*; verinin oynaklığı ve örneklem büyüklüğü aralığı ne kadar genişletiyor.
- $t_{\alpha/2, n-1}$ — *ne kadar emin olmak istiyorum*. %95 için ≈ 2 , %99 için $\approx 2,6$. Daha yüksek güven, daha geniş aralık: bedava emin olamazsın.

Niçin z değil t ? σ 'yı bilmiyor, onu da aynı veriden tahmin ediyorsun (s). Yani iki katmanlı bir belirsizlik var ve z bunu hesaba katmaz — aralığı olduğundan dar verir. t dağılımının normalden kalın kuyruklu olması tam olarak bu ekstra belirsizliğin telafisidir. $n = 10$ 'da $t^* = 2,26$ iken $n = 333$ 'te 2,0'a iner; büyük örneklemde $s \approx \sigma$ olduğu için fark kapanır.

Güven düzeyi nasıl seçilir? %95 bir gelenektir, doğa kanunu değil. Yanlış karar pahalıysa (ilaç onayı, uçak parçası) %99 kullanılır; keşif amaçlı analizde %90 yeterlidir. Seçimi *veriye baktıktan sonra* yapmak ise ciddi bir usul hatasıdır.

Nerede kullanılır? İlaç etkinliği raporları (“risk azalması %23, GA: %12–%33”); A/B testi sonuçları; anket hata payları; regresyon katsayılarının aralıkları; model doğruluğunun çapraz doğrulama aralığı.

Aralık, Nokta Tahmininden Niçin Daha Çok Şey Söyler?

İki A/B testi düşün. İkisinde de dönüşüm oranı artışı +%2 çıktı.

- *Test A*: %95 GA = [+1,8; +2,2]. Etki küçük ama *kesin*. Uygula.
- *Test B*: %95 GA = [-15; +19]. Aynı “+%2” rakamı, ama veri neredeyse hiçbir şey söylemiyor. Doğru karar: daha fazla veri topla.

Yalnızca nokta tahminine bakan bir yönetici bu iki testi *aynı* görür. Aralık ise ikisi arasındaki uçurumu tek bakışta gösterir.

Aynı sebeple, “fark anlamlı çıkmadı” cümlesi de tek başına yetersizdir. GA = [-0,3; +0,4] ise “etki yok, kapatalım” diyebilirsiniz; GA = [-20; +25] ise doğru cümle “*henüz bilmiyoruz*”dur. İkisi arasındaki farkı *p*-değeri gizler, güven aralığı gösterir.

```

1 se = x.std(ddof=1)/np.sqrt(n)
2
3 t_crit = st.t.ppf(0.975, df=n-1)
4 print(f"t* = {t_crit:.4f}")
5 print(f"t-CI 95%: [{x.mean()-t_crit*se:.1f}, {x.mean()+t_crit*se:.1f}]")
6
7 # Kısa yol
8 lo, hi = st.t.interval(0.95, df=n-1, loc=x.mean(), scale=se)
9 print(f"scipy : [{lo:.1f}, {hi:.1f}]")
10
11 for lvl in [0.80, 0.90, 0.95, 0.99]:
12     lo, hi = st.t.interval(lvl, n-1, x.mean(), se)
13     print(f"  %[{lvl*100:.0f}]: [{lo:.1f}, {hi:.1f}]   genislik = {hi-lo:.1f}")

```

```

t* = 1.9671
t-CI 95%: [4120.3, 4293.9]
scipy : [4120.3, 4293.9]
%80: [4150.4, 4263.7]   genislik=113.3
%90: [4134.3, 4279.8]   genislik=145.6
%95: [4120.3, 4293.9]   genislik=173.6
%99: [4092.7, 4321.4]   genislik=228.6

```

Dikkat

Güven aralığının doğru yorumu. “Gerçek ortalamanın bu aralıkta olma olasılığı %95’tir” cümlesi *yanlıştır* (sıklıkçı çerçevede gerçek ortalama sabittir, rastgele olan aralıktır). Doğru yorum: “*Aynı yöntemi tekrar tekrar uygularsam, ürettiğim aralıkların %95’i gerçek değeri içerir.*” Aşağıda bunu simüle ediyoruz.

```

1 kapsama = 0
2 M = 10_000
3 for _ in range(M):
4     s = rng.normal(100, 15, 20) # gercek ortalama = 100
5     lo, hi = st.t.interval(0.95, 19, s.mean(), s.std(ddof=1)/np.sqrt(20))
6     kapsama += (lo <= 100 <= hi)
7 print(f"gercek kapsama orani = {kapsama/M:.4f}   (hedef 0.95)")

```

```
gercek kapsama orani = 0.9507   (hedef 0.95)
```

21.1 Oran için Güven Aralığı

```
1 tit = sns.load_dataset('titanic')
2 k, N = tit.survived.sum(), len(tit)
3 p = k/N
4 print(f"p_hat = {p:.4f}    (k={k}, n={N})")
5
6 from statsmodels.stats.proportion import proportion_confint
7 for m in ['normal', 'wilson', 'beta']:
8     lo, hi = proportion_confint(k, N, alpha=0.05, method=m)
9     print(f"{m:>8}: [{lo:.4f}, {hi:.4f}]")
```

```
p_hat = 0.3838    (k=342, n=891)
normal: [0.3519, 0.4158]
wilson: [0.3525, 0.4162]
beta: [0.3518, 0.4167]
```

İpucu

normal (Wald) aralığı p uçlara (0 veya 1) yakınken ya da n küçükken kötü çalışır — hatta $[0, 1]$ dışına taşabilir. Varsayılan tercihini **Wilson** yap; kesin garanti istiyorsan beta (Clopper–Pearson) kullan. Bu fark, tıklama oranı (CTR) gibi küçük oranlarda ciddileşir.

22. Bootstrap: Formülsüz Çıkarım

Bir istatistiğin örnekleme dağılımını formülle bulamıyorsan (medyan, korelasyon, çeyrekler arası açıklık, $R^2 \dots$), veriyi *kendisinden* yeniden örnekleyerek yaklaş.

Bootstrap Algoritması

1. Veriden *iadeli* olarak n gözlem çek.
2. İstatistiği hesapla.
3. 1–2'yi B kez ($B \geq 2000$) tekrarla.
4. Elde edilen B değerlerin dağılımı, örnekleme dağılımının tahminidir; yüzdelikleri güven aralığını verir.

Nedir? Niçin? Nerede?

Fikir neden “imkânsız” görünüyor? Bootstrap, adını “kendi ayakkabı bağcığınan tutup kendini havaya kaldırmak” deyiminden alır — çünkü tek bir örneklemden, sanki defalarca örneklem almışsın gibi bilgi üretir. Mantığı şudur: elindeki örneklem, anakütlenin en iyi kopyasıdır. O hâlde anakütleden tekrar tekrar örnek çekemiyorsan, *örnekleminden* tekrar tekrar çek. Anakütle : örneklem ilişkisini, örneklem : yeniden örneklem ilişkisiyle taklit edersin.

İadeli çekmek niçin şart? İadesiz çekersen her seferinde aynı 333 gözlemi farklı sırayla alırsın ve ortalama hiç değişmez — hiçbir bilgi üretmezsin. İade sayesinde bazı gözlemler iki kez girer, bazıları hiç girmez; işte bu, “başka bir örneklem çekseydim ne olurdu?” sorusunun taklididir.

Niçin bu kadar değerli? Çünkü *formülden kurtarır*. Klasik istatistik her istatistik için ayrı bir örnekleme dağılımı türetmek zorundadır ve bu çoğu zaman ya çok zordur ya da imkânsızdır. Bootstrap ise tek bir tarifile her şeye uygulanır: medyan, çeyrekler açıklığı, korelasyon, R^2 , Gini katsayısı, iki grubun medyan farkı, bir makine öğrenmesi modelinin AUC skoru. . . Ortak özellikleri: hiçbirinin kapalı formülü yoktur, hepsinin bootstrap güven aralığı beş satır kodla hesaplanır.

Nerede kullanılır?

- **Model belirsizliği:** “Modelimin AUC’si 0,84” yerine “AUC = 0,84, %95 GA = [0,79; 0,88]” demek — test kümesini bootstrap’layarak.
- **Topluluk yöntemleri:** Rastgele ormandaki “bagging” kelimesi *bootstrap aggregating*’in kısaltmasıdır. Her ağaç, bootstrap örneği üzerinde eğitilir; eğitime girmeyen %36,8’lik kısım (*out-of-bag*) bedava doğrulama kümesi olur.
- **İş metrikleri:** “Müşteri yaşam boyu değerinin medyanı” gibi çarpık dağılımlı metriklerde güven aralığı kurmanın pratikteki tek yolu.
- **A/B testi:** Metrik ortalama değil de %90’lık yüzdeler ya da gelirin medyanı ise, klasik test yoktur — bootstrap vardır.

```

1 def boot_ci(data, stat=np.mean, B=10_000, alpha=0.05, rng=rng):
2     """Yüzdelik (percentile) bootstrap guven araligi."""
3     idx = rng.integers(0, len(data), (B, len(data))) # iadeli ornekleme
4     boots = stat(data[idx], axis=1)
5     return np.percentile(boots, [100*alpha/2, 100*(1-alpha/2)]), boots
6
7 (lo, hi), boots = boot_ci(x)
8 print(f"bootstrap ORTALAMA GA: [{lo:.1f}, {hi:.1f}] (bootstrap SE = {boots.std():.2f})")
9
10 (mlo, mhi), _ = boot_ci(x, stat=np.median)
11 print(f"bootstrap MEDYAN GA: [{mlo:.1f}, {mhi:.1f}] <- formulu yok!")
12
13 # scipy'nin BCa (yanlililik duzeltmeli) surumu
14 res = st.bootstrap((x,), np.mean, confidence_level=0.95,
15                   n_resamples=10_000, random_state=42)
16 print(f"scipy BCa : [{res.confidence_interval.low:.1f}, "
17       f"{res.confidence_interval.high:.1f}]")

```

```

bootstrap ORTALAMA GA: [4122.8, 4293.8] (bootstrap SE = 44.16)
bootstrap MEDYAN GA: [3900.0, 4200.0] <- formulu yok!
scipy BCa : [4122.9, 4295.0]

```

Bilgi

Bootstrap ortalama aralığı ([4122,8, 4293,8]), t -aralığıyla ([4120,3, 4293,9]) neredeyse aynı — teoriyi doğruluyor. Ama medyan için kapalı formül *yoktur*; bootstrap tek pratik yoldur. Aynı yöntemle korelasyonun, R^2 'nin ya da bir ML modelinin doğruluk skorunun güven aralığını da hesaplayabilirsin.

Dikkat

Bootstrap sihirli değildir: (1) örneklem anakütleyi temsil etmiyorsa bootstrap de etmez, (2) çok küçük n 'de (< 20) güvenilmezdir, (3) maksimum gibi uç istatistiklerde bozulur, (4) bağımlı veride (zaman serisi) blok bootstrap gerekir.

23. Örneklem Büyüklüğü Belirleme

Formül

İstenen hata payı E için:

$$n = \left(\frac{z_{\alpha/2} \sigma}{E} \right)^2 \quad (\text{oran için}) \quad n = \frac{z_{\alpha/2}^2 p(1-p)}{E^2}.$$

p bilinmiyorsa en kötü durum $p = 0,5$ alınır.

Nedir? Niçin? Nerede?

Bu formül niçin veri toplamadan önce kullanılır? Çünkü veri toplamak pahalıdır ve iki yönde de hata yapılabilir: az örneklem alırsan sonuç kullanılamayacak kadar belirsiz çıkar (para boşa gitmiştir); gereğinden fazla alırsan yine para boşa gitmiştir. Bu formül, “istediğim hassasiyet için minimum kaç gözlem gerekli?” sorusunu cevaplar.

Formülü ters çevrilmiş güven aralığı olarak okuyun. Güven aralığının yarı genişliği $z\sigma/\sqrt{n}$ 'dir. Buna “hata payı” E diyoruz ve n 'i yalnız bırakıyoruz: $n = (z\sigma/E)^2$. Yani hiçbir yeni fikir yok — aynı denklem, farklı bilinmeyen için çözülmüş.

Üç girdi, üç karar:

- E — ne kadar hassasiyet istiyorum? Bu bir istatistik sorusu değil, iş sorusudur. “%1’lik bir dönüşüm farkı bizim için önemli mi?”
- z — yanılma riskim ne olsun? %95 için 1,96.
- σ (veya p) — veri ne kadar oynak? Bilmiyorsan pilot çalışma yap, geçmiş veriye bak ya da en kötü durumu ($p = 0,5$) al.

Karesel ilişkinin acımasızlığı: E paydada ve karede olduğu için, hassasiyeti iki kat artırmak örnekleme dört kat büyütür. Tabloda görüldüğü gibi ± 200 g için 63 penguen yeterken, ± 50 g için 996 penguen gerekiyor. Bütçe planlarken bu eğriyi bilmek, sonradan “biraz daha veri toplayalım” demekten çok daha ucuzdur.

Nerede kullanılır? Klinik araştırma protokolleri (etik kurul onayı için örneklem hesabı zorunludur); A/B testi süresi planlama (“bu testi kaç gün çalıştırmalıyım?”); anket tasarımı; kalite kontrolde parti örnekleme planları; makine öğrenmesinde “etiketleme bütçesi ne olmalı?” kararı.

```
1 sigma = 805
2 for E in [50, 100, 200]:
```

```
3 print(f" +-{E:>3} g hassasiyet icin n = {np.ceil((1.96*sigma/E)**2):.0f}")
4
5 # Oran icin (anket): %3 hata payi, %95 guven
6 E, p = 0.03, 0.5
7 print(f" anket: n = {np.ceil(1.96**2*p*(1-p)/E**2):.0f}")
```

```
+ - 50 g hassasiyet icin n = 996
+ -100 g hassasiyet icin n = 249
+ -200 g hassasiyet icin n = 63
anket: n = 1068
```

Örnek

Seçim anketlerinin neden hep “1000–1200 kişi” ile yapıldığını artık biliyorsun: %95 güvenle $\pm\%3$ hata payı tam olarak $n \approx 1068$ ister. Hata payını %1,5’e indirmek istersen $n \approx 4269$ — dört kat maliyet.

KISIM 6

Hipotez Testleri

24. Mantık: H_0 , p -değeri ve Hatalar

Tanım

Sıfır hipotezi H_0 , “fark yok / etki yok” varsayımdır. **p -değeri**, H_0 *doğruyken* gözlediğimiz kadar veya daha uç bir sonuç görme olasılığıdır. Küçük p , verinin H_0 ile bağdaşmadığını söyler.

Hipotez Testi Aslında Ne Yapıyor?

Hipotez testinin mantığı **çelişkiyle ispata** benzer, ama olasılıksal bir sürümüdür:

1. Kanıtlamak istediğinin *tersini* varsay: “bu ilacın hiçbir etkisi yok” (H_0).
2. Bu varsayım doğruyken, elindeki kadar çarpıcı bir sonuç görme olasılığını hesapla (p -değeri).
3. Bu olasılık çok küçükse: ya çok nadir bir tesadüf yaşadın, ya da başlangıç varsayımın yanlıştı. İkincisini tercih et.

Niçin doğrudan “ilaç işe yarıyor” hipotezini test etmiyoruz? Çünkü “etki var” cümlesi belirsizdir — ne kadar etki? Oysa “etki tam olarak sıfır” cümlesi kesindir, dolayısıyla ondan somut olasılık hesaplanabilir. H_0 ’ın hep “fark yok” olmasının sebebi budur: sınanabilir tek nokta odur.

Mahkeme benzetmesi. H_0 = “sanık masumdur”. Veri = deliller. α = “masum birini mahkûm etme riski” (Tip I). β = “suçluyu serbest bırakma riski” (Tip II). Mahkeme, masumu mahkûm etmeyi daha ağır bir hata saydığı için α ’yı küçük tutar — tıpkı istatistikte 0,05’te tutulması gibi. Ve dikkat: beraat, “sanığın masum olduğu kanıtlandı” demek değildir; “yeterli delil yok” demektir. Aynı şekilde “ $p > 0,05$ ” **asla “fark yoktur” anlamına gelmez** — “farkı gösterecek kadar veri yoktu” anlamına gelir.

İki hata türü niçin birbiriyle çelişir? α ’yı küçültmek (daha az yanlış alarm) kaçınılmaz olarak β ’yı büyütür (daha çok kaçırma). İkisini aynı anda küçültmenin tek yolu *daha çok veri toplamaktır*. Bu denge, makine öğrenmesinde *precision–recall* dengesi olarak aynen karşına çıkacak: sınıflandırma eşliğini kaydırmak, tam olarak α ile β arasında yer değiştirmektir.

Dikkat

p -değeri **şunlar değildir**: (1) H_0 ’ın doğru olma olasılığı, (2) etkinin büyüklüğü, (3) sonucun pratik önemi, (4) bulgunun tekrarlanma olasılığı. $p = 0,049$ ile $p = 0,051$ arasında niteliksel bir fark yoktur; 0,05 eşiği bir gelenektir, doğa yasası değil.

Karar	H_0 doğru	H_0 yanlış
H_0 reddedilmez	Doğru karar ($1 - \alpha$)	Tip II hata (β) — gerçek etkiyi kaçırma
H_0 reddedilir	Tip I hata (α) — yanlış alarm	Doğru karar ($\text{güç} = 1 - \beta$)

Test Kurma Adımları

1. H_0 ve H_1 'i veriye bakmadan yaz.
2. α seç (genelde 0,05).
3. Varsayımları kontrol et (bağımsızlık, dağılım, varyans eşitliği).
4. Test istatistiğini ve p -değerini hesapla.
5. Karar ver ve etki büyüklüğü ile güven aralığını raporla.

Hangi Hata Daha Pahalı?

α ve β 'yi “teknik ayar” sanmak yaygın bir hatadır; bunlar aslında *maliyet* kararlarıdır ve bağlama göre tamamen değişir:

- **Kanser taraması:** Tip II hata (hastayı kaçırmak) ölümcül; Tip I hata (yanlış alarm) sadece ek tetkik demek. Bu yüzden tarama testleri *yüksek duyarlılıkla* tasarlanır — bilerek çok yanlış alarm verirler.
- **Uçak motoru parçası:** Tip I hata (sağlam parçayı çöpe atmak) pahalı ama katlanılabilir; Tip II hata (çatlak parçayı geçirmek) felaket.
- **Spam filtresi:** Tip I hata (önemli e-postayı spam'e atmak) kullanıcı için çok can sıkıcı; Tip II hata (spam'i geçirmek) sadece rahatsız edici. Bu yüzden filtreler *yüksek kesinliğe* ayarlanır — şüphedeysen gelen kutusuna bırakırlar.
- **İlaç ruhsatı:** Tip I hata (işe yaramayan ilacı onaylamak) toplum sağlığı ve maliyet açısından ağırdır; bu yüzden ilaç denemelerinde α genellikle 0,05 yerine 0,025 veya daha düşük tutulur.

Ders: $\alpha = 0,05$ 'i düşünmeden kabul etmek, bir maliyet kararını geleneğe devretmektir. Doğru soru her zaman “bu bağlamda hangi hatanın bedeli daha ağır?”dır.

25. t-Testleri

t-Testi Ne Yapar?

t-testi, istatistiğin en çok kullanılan aracıdır ve tek bir soruyu sorar: “Gözlediğim fark, sadece şaňstan kaynaklanabilir mi?”

Formülün yapısı her üç türde de aynıdır ve son derece sezgiseldir:

$$t = \frac{\text{gözlenen fark}}{\text{farkın gürültü düzeyi}}$$

Payda, gürültünün büyüklüğüdür (standart hata). Yani t , **sinyal / gürültü oranıdır**. $t = 0,4$ demek “fark, gürültünün yarısı kadar bile değil” demektir — kayda değer bir şey yok. $t = 8$ demek “fark, gürültünün sekiz katı” demektir — bunu şansa yormak zor.

Üç t-testi, üç farklı tasarım:

- **Tek örneklem:** Bir grubu *bilinen bir referans değerle* karşılaştırırsın. *Örnek:* “Üretilen paketlerin ortalama ağırlığı etiketteki 500 g mı?” “Sunucumuzun ortalama yanıt süresi taahhüt ettiğimiz 200 ms’nin altında mı?”
- **Bağımsız iki örneklem:** *Farklı* iki grubu birbiriyle karşılaştırırsın. *Örnek:* A/B testinde kontrol ve deney grubu; ilaç alan ve plasebo alan hastalar; iki farklı tedarikçinin parçaları.
- **Eşleştirilmiş:** *Aynı* birimleri iki kez ölçersin. *Örnek:* Eğitim öncesi/sonrası sınav puanı; aynı sunucuda eski/yeni algoritma; aynı hastanın sol/sağ gözü.

Niçin “t” de z değil? Eğer anakütle standart sapmasını bilseydik z kullanırdık. Bilmiyoruz, onu da veriden tahmin ediyoruz — bu ek belirsizlik, dağılımı normalden biraz daha kalın kuyruklu yapar. William Gosset bu dağılımı 1908’de Guinness bira fabrikasında, *küçük* arpa örneklemeleriyle çalışırken buldu; şirket sırrı sayıldığı için makaleyi “Student” takma adıyla yayımladı. “Student’s t ” adı buradan gelir.

Varsayımları: (1) gözlemler bağımsız, (2) veri (ya da MLT sayesinde ortalaması) yaklaşık normal, (3) iki örneklemli sürümde varyanslar benzer (Welch bu varsayımı kaldırır). Bunlardan en kritiğı *bağımsızlıktır*; ihlal edilirse hiçbir düzeltme kurtarmaz.

25.1 Tek Örneklem t-Testi

Formül

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}, \quad \text{df} = n - 1$$

Nedir? Niçin? Nerede?

Formülü kelimelerle: “Gözlediğim ortalama, iddia edilen değerden kaç standart hata uzakta?” Pay $(\bar{x} - \mu_0)$ farkın kendisidir; payda $(s/\sqrt{n})$ o farkın ne kadar gürültülü olduğudur.

Sayılarla: Adelie penguenlerinin ortalaması 3706,2 g, iddia 3700 g, standart hata ≈ 38 g. Fark 6,2 g yalnızca 0,16 standart hata eder — yani tamamen gürültü içinde. $p = 0,87$ bunu söylüyor: “3700 doğruysa, böyle bir fark görmek son derece sıradan.”

Serbestlik derecesi $(n - 1)$ niçin var? Sapmaları hesaplarırken kullandığın $\bar{x}$ ’i aynı veriden ürettin; bu bir kısıt getirir ve n sapmadan yalnızca $n - 1$ tanesi serbestçe değişebilir. Küçük n ’de bu ciddi bir fark yaratır: $n = 5$ ’te kritik değer 2,78 iken $n = 100$ ’de 1,98’e iner.

Nerede kullanılır? Kalite kontrolde etiket uyumu (“500 g yazıyor, gerçekten 500 g mı?”); SLA doğrulama (“taahhüt edilen 200 ms tutuyor mu?”); laboratuvar kalibrasyonu (“cihaz bilinen standardı doğru ölçüyor mu?”); üretim spesifikasyonu denetimi.

```

1 import seaborn as sns, numpy as np
2 from scipy import stats as st
3
4 peng = sns.load_dataset('penguins').dropna()
5 x = peng.loc[peng.species == 'Adelie', 'body_mass_g']
6
7 # H0: Adelie penguenlerinin ortalama kutlesi 3700 g
8 r = st.ttest_1samp(x, 3700)
9 print(f"n = {len(x)}   ortalama = {x.mean():.1f}")
10 print(f"t = {r.statistic:.4f}   p = {r.pvalue:.4f}")
11 print("%95 GA:", np.round(r.confidence_interval(0.95), 1))

```

```

n = 146   ortalama = 3706.2
t = 0.1624   p = 0.8712
%95 GA: [3631.1 3781.2]

```

$p = 0,87$: veri, 3700 g varsayımıyla tamamen uyumlu. Güven aralığı da 3700'ü rahatlıkla içeriyor.

25.2 Bağımsız İki Örneklem t-Testi

Nedir? Niçin? Nerede?

En sık kullanılan istatistik testi budur — çünkü “bu iki grup farklı mı?” sorusu, bilimden ürün geliştirmeye kadar her yerde sorulur. Test istatistiği yine aynı yapıdadır:

$$t = \frac{\bar{x}_1 - \bar{x}_2}{SE(\bar{x}_1 - \bar{x}_2)}, \quad SE = \sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}.$$

Paydadaki karekök şuradan gelir: bağımsız iki rastgele değişkenin farkının varyansı, varyansların toplamıdır (çıkarma değil!). Sezgisi şu: iki gürültülü ölçümün farkı, tek bir ölçümden daha gürültülüdür — iki belirsizlik birbirini götürmez, birikir.

Nerede kullanılır? A/B testleri (en yaygın kullanım); klinik denemelerde tedavi vs. plasebo; iki tedarikçinin kalite karşılaştırması; eski ve yeni algoritmanın çalışma süresi; iki şubenin satış performansı.

Student mi Welch mi? Klasik Student sürümü, iki grubun varyansının eşit olduğunu varsayar ve onları tek bir “birleşik” varyansta toplar. Gerçek hayatta bu varsayım çoğu zaman tutmaz — özellikle grup boyutları farklıysa. Welch düzeltmesi varyansları ayrı tutar ve serbestlik derecesini buna göre düşürür. Bedeli neredeyse sıfırdır, kazancı büyüktür; bu yüzden aşağıdaki ipucu “her zaman Welch” der.

```

1 a = peng.loc[peng.species == 'Adelie', 'body_mass_g']
2 c = peng.loc[peng.species == 'Chinstrap', 'body_mass_g']
3
4 print(f"Adelie      : n={len(a)}   ort={a.mean():.1f}   sd={a.std():.1f}")
5 print(f"Chinstrap  : n={len(c)}   ort={c.mean():.1f}   sd={c.std():.1f}")
6
7 print(f"Levene (varyans esitligi) p = {st.levene(a, c).pvalue:.4f}")
8 print(f"Student t = {st.ttest_ind(a, c, equal_var=True).statistic:.4f}   "
9       f"p = {st.ttest_ind(a, c, equal_var=True).pvalue:.4f}")
10 w = st.ttest_ind(a, c, equal_var=False)           # Welch
11 print(f"Welch      t = {w.statistic:.4f}   p = {w.pvalue:.4f}   df = {w.df:.1f}")

```

```

Adelie      : n=146   ort=3706.2   sd=458.6
Chinstrap   : n=68    ort=3733.1   sd=384.3
Levene (varyans esitligi) p = 0.0305

```

Student t = -0.4201 p = 0.6748
Welch t = -0.4479 p = 0.6548 df = 154.0

İpucu

Her zaman Welch kullan (`equal_var=False`). Varyanslar eşitse Welch, Student ile neredeyse aynı sonucu verir; eşit değilse (burada Levene $p = 0,03$ ile eşitliği reddediyor) Welch doğru, Student yanlış. “Önce varyans eşitliğini test et, sonra karar ver” yaklaşımı ise iki aşamalı test yaptığı için Tip I hatayı şişirir.

25.3 Etki Büyüklüğü: p -değerinden Daha Önemli

Cohen’s d

$$d = \frac{\bar{x}_1 - \bar{x}_2}{s_{\text{birleşik}}}, \quad s_{\text{birleşik}} = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}}$$

Kaba yorum: $|d| \approx 0,2$ küçük, $0,5$ orta, $0,8$ büyük etki.

Nedir? Niçin? Nerede?

Niçin p -değeri yetmiyor? Çünkü p -değeri iki şeyi karıştırır: etkinin *büyüklüğü* ile örneklemin *büyüklüğü*. Yeterince veri toplarsan, pratikte hiçbir anlamı olmayan bir fark bile “istatistiksel olarak anlamlı” çıkar. 10 milyon kullanıcı bir A/B testinde tıklama oranındaki %0,001’lik bir artış $p < 0,001$ verir — ve hiçbir işe yaramaz.

Cohen’s d ne yapar? Farkı, *standart sapma cinsinden* ölçer: “iki grubun ortalaması kaç standart sapma uzakta?” Böylece hem birimden hem örneklem büyüklüğünden kurtulur. $d = 1$ demek, ortalamalar arasındaki farkın grup içi tipik değişkenlik kadar olması demektir.

d ’yi görselleştirmek: $d = 0,2$ ’de iki dağılım %92 örtüşür — rastgele seçilen bir 1. grup üyesinin, rastgele bir 2. grup üyesinden büyük olma olasılığı yalnızca %56. $d = 0,8$ ’de örtüşme %53’e, $d = 2,9$ ’da (penguen örneğindeki Gentoo–Adelie farkı) neredeyse sıfıra iner — iki grubu tek bir ölçümle neredeyse hatasız ayırabilirsin.

Nerede kullanılır?

- **Meta-analiz:** Farklı ölçeklerde, farklı örneklem büyüklükleriyle yapılmış onlarca çalışmayı birleştirmenin tek yolu etki büyüklüğüdür — p -değerleri birleştirilemez.
- **Güç analizi:** “Kaç kişiyle test yapmalıyım?” sorusunun girdisi p değil d ’dir. Küçük etkileri yakalamak çok daha fazla veri ister.
- **Ürün kararı:** “Anlamlı ama küçük” bir iyileştirme için mühendislik maliyetine değer mi? Bu soruyu ancak d (ya da mutlak fark ve güven aralığı) cevaplayabilir.

Kural: Hiçbir sonucu yalnızca p ile raporlama. Doğru rapor “ $fark = 26,9 \text{ g}$; %95 GA = $[-96; +150]$; $d = -0,06$; $p = 0,65$ ” biçimindedir — okuyucu hem farkın büyüklüğünü, hem belirsizliğini, hem de anlamlılığını birlikte görür.

```
1 def cohen_d(a, b):
2     n1, n2 = len(a), len(b)
3     s = np.sqrt(((n1-1)*a.var(ddof=1) + (n2-1)*b.var(ddof=1))/(n1+n2-2))
4     return (a.mean() - b.mean())/s
5
6 g = peng.loc[peng.species == 'Gentoo', 'body_mass_g']
7 print(f"Adelie vs Chinstrap: d = {cohen_d(a, c):+.4f}    p = {w.pvalue:.4f}")
8 gw = st.ttest_ind(g, a, equal_var=False)
```

```
9 print(f"Gentoo vs Adelie : d = {cohen_d(g, a):+.4f} p = {gw.pvalue:.3e}")
```

```
Adelie vs Chinstrap: d = -0.0617 p = 0.6548
Gentoo vs Adelie : d = +2.8846 p = 1.223e-63
```

Bilgi

$d = 2,88$ olağanüstü büyük bir etkidir: iki grup neredeyse hiç örtüşmüyor. Buna karşılık Adelie–Chinstrap farkı $d = -0,06$ ile pratikte sıfırdır. p -değeri örneklem büyüklüğüne bağlıdır, etki büyüklüğü değildir — yeterince veriyle her anlamsız fark “anamlı” çıkar. Bu yüzden ikisini birlikte raporla.

25.4 Eşleştirilmiş t -Testi

Nedir? Niçin? Nerede?

Püf noktası: Eşleştirilmiş test aslında yeni bir test değildir. Her denek için farkı hesaplar ($d_i = \text{öncesi}_i - \text{sonrası}_i$) ve bu tek fark dizisine *tek örneklem t -testi* uygular (H_0 : ortalama fark sıfır). Yani iki sütunu bire indirger.

Niçin bu kadar güçlü? Çünkü *kişiler arası* değişkenliği tamamen ortadan kaldırır. Bir eğitim programının etkisini ölçtüğünü düşün: öğrenciler arasındaki doğal yetenek farkı devasadır (kimi 40 alır kimi 90). Bağımsız test bu devasa farkı “gürültü” sayar ve gerçek etkiyi içinde boğar. Eşleştirilmiş testte ise her öğrenci *kendi kontrolüdür* — yetenek farkı çıkarma işleminde yok olur, geriye sadece müdahalenin etkisi kalır.

Yukarıdaki örnekte etki yalnızca 4,09 puan, kişiler arası sapma ise 15 puan. Bağımsız testle bu fark kaybolurdu; eşleştirilmiş testle $p = 0,0016$ ile net biçimde yakalanıyor.

Nerede kullanılır? Öncesi/sonrası tasarımlar (eğitim, diyet, tedavi); aynı donanımda iki algoritmanın kıyaslanması; aynı kullanıcıya iki arayüzün gösterilmesi (*within-subject* tasarım); ikiz çalışmaları; aynı hastanın iki gözü/iki eli; aynı tarlanın iki yarısı. **Kural:** Ölçümleri eşleştirebiliyorsan eşleştir — aynı sonucu çok daha az denekle elde edersin.

```
1 rng = np.random.default_rng(42)
2 oncesi = rng.normal(100, 15, 30)
3 sonrası = oncesi + rng.normal(-5, 8, 30) # aynı kişiler, müdahale sonrası
4
5 rp = st.ttest_rel(onceci, sonrası)
6 print(f"eslesmis t = {rp.statistic:.4f} p = {rp.pvalue:.4f}")
7 print(f"ortalama fark = {np.mean(onceci - sonrası):.2f}")
8 print(f"Wilcoxon (parametrik olmayan) p = {st.wilcoxon(onceci, sonrası).pvalue:.4f}")
```

```
eslesmis t = 3.4797 p = 0.0016
ortalama fark = 4.09
Wilcoxon (parametrik olmayan) p = 0.0024
```

Dikkat

Eşleştirilmiş veriye bağımsız t -testi uygulamak ciddi bir hatadır: kişiler arası değişkenliği “gürültü” sayar ve gücü büyük ölçüde düşürür. Aynı denek, eşleştirilmiş hasta, öncesi/sonrası ölçüm → her zaman `ttest_rel`.

26. ANOVA: İkidenden Fazla Grup

Tek Yönlü ANOVA

$$F = \frac{MS_{\text{gruplar arası}}}{MS_{\text{grup içi}}} = \frac{SS_B / (k - 1)}{SS_W / (N - k)}$$

H_0 : bütün grup ortalamaları eşit. F büyükse gruplar arası fark, grup içi gürültüden baskındır.

Nedir? Niçin? Nerede?

Adı niçin “varyans analizi”? Test *ortalamaları* karşılaştırıyor ama adında “varyans” geçiyor — bu tuhaflık aslında yöntemin özüdür. ANOVA şu soruyu sorar: “Gözlediğim toplam değişkenliğin ne kadarı gruplar arasındaki farktan, ne kadarı grupların içindeki doğal dağılımdan geliyor?” Bu, Kısım 3’teki varyans ayrıştırmasının ta kendisidir.

F oranını kelimelerle okuyun:

$$F = \frac{\text{gruplar birbirinden ne kadar uzak}}{\text{grup içinde ne kadar dağınık}} = \frac{\text{sinyal}}{\text{gürültü}}$$

$F \approx 1$ ise gruplar arası fark, tesadüfen beklenen kadar — yani ayrı grupmuş gibi davranmıyorlar. F büyükse gruplar gerçekten ayrışıyor. Penguen örneğinde $F = 343$ çıkıyor: türler arası fark, tür içi dağınıklığın yüzlerce katı.

Niçin ikiye ikiye t -testi yapmıyoruz? Aşağıdaki uyarıda anlatılan çoklu karşılaştırma sorunu yüzünden. Ama daha derin bir sebep de var: ANOVA tüm veriyi *birlikte* kullanarak grup içi varyansı tahmin eder, dolayısıyla ikili testlerden daha güçlüdür.

Nerede kullanılır?

- **A/B/C/D testleri:** Dört farklı ana sayfa tasarımının dönüşüm oranını aynı anda karşılaştırmak.
- **Tarım ve üretim:** Beş farklı gübrenin (ya da beş üretim hattının) verim üzerindeki etkisi — ANOVA zaten tarımsal deneyler için icat edildi (R. A. Fisher, 1920’ler).
- **Klinik denemeler:** Üç doz seviyesi + plasebo karşılaştırması.
- **Makine öğrenmesi:** Beş modelin çapraz doğrulama skorlarını karşılaştırmak; bir hiper-parametrenin farklı değerlerinin etkisi.

Önemli sınır: ANOVA yalnızca “en az bir grup farklı” der, *hangisinin* farklı olduğunu söylemez. Onun için post-hoc testler (Tukey HSD) gerekir — bir sonraki alt bölüm.

Varsayımları: bağımsızlık, her grupta yaklaşık normallik, ve *varyans homojenliği* (gruplar benzer yayılımı sahip). Sonuncusu ihlal edilirse Welch ANOVA (`st.f_oneway` yerine `pinguin.welch_anova`) veya parametrik olmayan Kruskal–Wallis kullanılır.

Dikkat

“Neden üç grubu ikiye ikiye t -testiyle karşılaştırmıyoruz?” Çünkü 3 grup için 3, 5 grup için 10 test yapmak gerekir ve her testte %5 yanlış alarm riski birikir: 10 testte en az bir yanlış pozitif olasılığı %40’a çıkar. ANOVA *tek* bir test yapar.

```
1 gruplar = [g_['body_mass_g'].values for _, g_ in peng.groupby('species')]
2 f = st.f_oneway(*gruplar)
3 print(f"F = {f.statistic:.2f}    p = {f.pvalue:.3e}")
4
5 # statsmodels ile tam ANOVA tablosu
6 import statsmodels.api as sm, statsmodels.formula.api as smf
```

```
7 m = smf.ols('body_mass_g ~ C(species)', data=peng).fit()
8 print(sm.stats.anova_lm(m, typ=2).round(4))
```

```
F = 341.89    p = 3.745e-81
              sum_sq    df    F    PR(>F)
C(species)  1.451902e+08    2.0  341.8949    0.0
Residual    7.006945e+07  330.0     NaN     NaN
```

```
1 ss_b = sm.stats.anova_lm(m, typ=2)['sum_sq'].iloc[0]
2 ss_t = ((peng.body_mass_g - peng.body_mass_g.mean())**2).sum()
3 print(f"eta^2 (etki buyuklugu) = {ss_b/ss_t:.4f}")    # 0.6745
```

Bilgi

$\eta^2 = 0,674$: vücut kütleindeki değişkenliğin %67,4'ü tür farkıyla açıklanıyor. Bu sayı, Kısım 3'te yaptığımız varyans ayrıştırmasıyla *birebir aynıdır* — ANOVA, varyansı gruplara ayırmaktan başka bir şey değildir. Aynı sayı Kısım 7'de R^2 olarak da karşımıza çıkacak.

26.1 Post-hoc: Hangi Gruplar Farklı?

Nedir? Niçin? Nerede?

Niçin ANOVA'dan sonra ayrı bir adım gerekiyor? ANOVA sana “bu üç tür aynı ağırlıkta değil” der ve durur. Ama karar verirken lazım olan bilgi bu değildir — *hangi ikisi* farklı, ve *ne kadar* farklı? Post-hoc testler bu boşluğu doldurur.

Tukey HSD ne yapıyor? Bütün ikili karşılaştırmaları yapar, ama her birine ayrı ayrı $\alpha = 0,05$ vermek yerine *tümünün toplam* yanlış alarm riskini %5'te tutar. “HSD” = *Honestly Significant Difference* — yani “dürüstçe anlamlı fark”; adı bile, düzeltilmesiz ikili testlerin dürüst olmadığını ima eder.

Çıktı nasıl okunur? meandiff farkın büyüklüğü, lower-upper düzeltilmiş güven aralığı, reject ise kararı verir. Aralık sıfırı içeriyorsa fark yoktur. Adelie-Chinstrap için aralık $[-132; +186]$, sıfırı içeriyor → fark yok. Gentoo karşılaştırmalarında aralıklar sıfırdan çok uzak → net fark.

Nerede kullanılır? Çok kollu A/B testlerinde “hangi varyant kazandı?”; klinik denemede hangi dozun etkili olduğu; kalite kontrolde hangi üretim hattının sorunlu olduğu; birden çok modelin karşılaştırılmasında hangi çiftlerin gerçekten farklı olduğu.

```
1 from statsmodels.stats.multicomp import pairwise_tukeyhsd
2 print(pairwise_tukeyhsd(peng.body_mass_g, peng.species, alpha=0.05).summary())
```

```
Multiple Comparison of Means - Tukey HSD, FWER=0.05
=====
group1    group2    meandiff p-adj    lower    upper    reject
-----
Adelie Chinstrap    26.9239 0.9164 -132.3528  186.2005  False
Adelie  Gentoo    1386.2726    0.0 1252.2897 1520.2554  True
Chinstrap  Gentoo    1359.3487    0.0 1194.4304 1524.2671  True
-----
```

Tukey HSD, aile bazında hata oranını (FWER) koruyarak tüm ikili karşılaştırmaları yapar: Gentoo diğer ikisinden farklı, Adelie ile Chinstrap arasında fark yok.

26.2 İki Yönlü ANOVA ve Etkileşim

```
1 m2 = smf.ols('body_mass_g ~ C(species)*C(sex)', data=peng).fit()
2 print(sm.stats.anova_lm(m2, typ=2).round(4))
```

	sum_sq	df	F	PR(>F)
C(species)	1.434016e+08	2.0	749.0157	0.0000
C(sex)	3.709026e+07	1.0	387.4600	0.0000
C(species):C(sex)	1.676557e+06	2.0	8.7570	0.0002
Residual	3.130263e+07	327.0	NaN	NaN

Etkileşim Ne Demek?

Etkileşim terimi anlamlı ($p = 0,0002$): cinsiyetin kütle üzerindeki etkisi *türe göre değişiyor*. Gentoo'larda erkek-dişi farkı, Adelie'lerdekinden büyük. Etkileşim varsa ana etkileri tek başına yorumlamak yanıltıcıdır — önce etkileşim grafiğine bak (`sns.pointplot(data=peng, x='species', y='body_mass_g', hue='sex')`).

27. Kategorik Veri: Ki-Kare Testleri

Ki-Kare

$$\chi^2 = \sum_{i,j} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}, \quad E_{ij} = \frac{(\text{satır toplamı})(\text{sütun toplamı})}{N}$$

H_0 : iki kategorik değişken bağımsızdır.

Nedir? Niçin? Nerede?

Ne zaman ki-kare? Şimdiye kadarki testler *sayısal* veriyle çalışıyordu (ağırlık, süre, puan). Ki-kare ise *kategorik* veri içindir: hayatta kaldı/kalmadı, tıkladı/tıklamadı, A/B/C sınıfı, evet/hayır. Ortalama alamayacağın veriler için tasarlanmıştır — “cinsiyetin ortalaması” diye bir şey yoktur, ama “cinsiyet ile hayatta kalma ilişkili mi?” sorusu son derece anlamlıdır.

Formülün mantığı çok basit ve çok genel:

$$\chi^2 = \sum \frac{(\text{gözlenen} - \text{beklenen})^2}{\text{beklenen}}$$

Önce “eğer bu iki değişken tamamen ilişkisiz olsaydı hangi sayıları görürdüm?” diye sorarsın — işte E_{ij} budur. Sonra gerçek sayılarla karşılaştırırsın. Sapmalar büyükse bağımsızlık varsayımı tutmuyordur.

Niçin beklenene bölünüyor? Çünkü 5 kişilik bir sapma, beklenen sayının 10 olduğu bir hücrede devasa, 1000 olduğu bir hücrede önemsizdir. Bölme işlemi sapmayı *göreceli* hâle getirir. Kare almanın sebebi ise her zamanki gibi işaretten kurtulmaktır.

E_{ij} formülü nereden geliyor? Doğrudan bağımsızlık tanımından: $\mathbb{P}(A \cap B) = \mathbb{P}(A)\mathbb{P}(B)$. Satır olasılığı çarpı sütun olasılığı çarpı N , tam olarak yukarıdaki ifadeyi verir. Yani ki-kare testi, Kısım 3’teki bağımsızlık tanımının doğrudan bir sınamasıdır.

Üç kullanım biçimi:

- **Bağımsızlık testi:** İki kategorik değişken ilişkili mi? (Buradaki örnek.)
- **Uyum iyiliği:** Gözlenen dağılım, teorik bir dağılıma uyuyor mu? (“Zar hileli mi?”, “Bu veri Poisson mu?”)
- **Homojenlik:** Farklı gruplarda aynı kategorik dağılım var mı? (“Üç şubede müşteri profili benzer mi?”)

Nerede kullanılır? A/B testinde dönüşüm oranı karşılaştırması (en yaygın kullanım — çünkü dönüşüm ikili bir sonuçtur); pazar araştırmasında segment–tercih ilişkisi; tıpta risk faktörü–hastalık ilişkisi; kalite kontrolde hata tipi–vardıya ilişkisi; makine öğrenmesinde kategorik öznitelik seçimi (`sklearn.feature_selection.chi2`).

Cramér’s V niçin gerekli? χ^2 ’nin kendisi örneklem büyüklüğüyle doğru orantılı büyür — 10 kat daha fazla veriyle aynı ilişki 10 kat büyük χ^2 verir. V , χ^2 ’yi $[0, 1]$ aralığına normalleştirerek etki büyüklüğüne çevirir. Burada $V = 0,34$: orta güçte bir ilişki. Tıpkı p ile Cohen’s d ilişkisi gibi — χ^2 anlamlılığı, V önemi söyler.

```
1 import pandas as pd
2 tit = sns.load_dataset('titanic')
3 tab = pd.crosstab(tit['class'], tit['survived'])
4 print(tab)
5
6 chi2, p, dof, exp = st.chi2_contingency(tab)
7 print(f"\nchi2 = {chi2:.2f}    dof = {dof}    p = {p:.3e}")
8
9 V = np.sqrt(chi2/(len(tit)*(min(tab.shape) - 1)))    # Cramer's V
10 print(f"Cramer V = {V:.4f}    (etki buyuklugu)")
```

```
survived    0    1
class
First       80   136
Second      97    87
Third      372   119
```

```
chi2 = 102.89    dof = 2    p = 4.549e-23
```

Cramer V = 0.3398 (etki büyüklüğü)

Dikkat

Ki-kare testinin varsayımı: beklenen frekansların en az %80'i ≥ 5 olmalı. Küçük tablolarda (özellikle 2×2) bu sağlanmazsa **Fisher'in kesin testi** (`st.fisher_exact`) kullanılır. Ayrıca ki-kare *ilişkinin varlığını* söyler, *yönünü* söylemez — standartlaştırılmış artıklara bakmak gerekir.

28. Parametrik Olmayan Testler ve Permütasyon

“Parametrik Olmayan” Ne Demek?

Şimdiye kadarki testler *parametrik*: verinin belirli bir dağılımdan (genelde normal) geldiğini varsayıp o dağılımın parametreleri (μ , σ) üzerinden çıkarım yaptılar. Parametrik olmayan testler bu varsayımı **tamamen kaldırır**.

Nasıl kaldırıyorlar? Çoğu, ham değerleri *sıralara* çevirerek. 3800, 4050, 4200, 12000 dizisi 1, 2, 3, 4 olur. Bunun iki büyük sonucu vardır: (1) Dağılımın şekli tamamen önemsizleşir — her veri aynı sıra dizisine dönüşür. (2) Aykırı değerler zararsızlaşır: 12000 ile 4300 arasında fark kalmaz, ikisi de “en büyük” olur.

Ne zaman kullanmalı?

- Örneklem küçük ($n < 15$) ve normallikten emin değilsen — MLT'ye güvenemeyeceğin tek durum budur.
- Veride silmek istemediğin ağır aykırı değerler varsa.
- Veri zaten *sıralayıcı* ise: memnuniyet anketi (1–5 yıldız), Likert ölçeği, kanser evresi, ürün derecelendirmesi. Bunların ortalamasını almak zaten matematiksel olarak kusurludur — “3 yıldız ile 4 yıldız arasındaki fark, 4 ile 5 arasındaki farka eşit” varsayımını yapamazsın.
- Dağılım aşırı çarpıksa ve medyan, ortalamadan daha anlamlı bir özetse (gelir, yanıt süresi).

Bedeli ne? Veri gerçekten normalse, parametrik olmayan testler biraz daha az güçlüdür — yaklaşık %5 civarında verim kaybı (yani aynı sonucu almak için %5 daha fazla veri gerekir). Bu, sanılandan çok küçük bir bedeldir. Ayrıca sıralara geçtiğin için *fark ne kadar?* sorusunu doğrudan cevaplayamazsın; bu yüzden yanında etki büyüklüğü veya güven aralığı (bootstrap ile) raporlamak gerekir.

Kritik nüans: Mann–Whitney testi “ortalamalar eşit mi?” diye sormaz; “bir gruptan rastgele seçilen değer, diğerinden seçilenden büyük olma olasılığı %50 mi?” diye sorar. Genellikle bu “medyanlar eşit mi” demeye gelir, ama dağılım şekilleri çok farklıysa gelmez.

Parametrik	Karşılığı	Ne zaman?
Tek örneklem t	wilcoxon	Simetri var ama normallik yok.
Bağımsız t	mannwhitneyu	Sıralayıcı veri, aykırı değer, küçük n .
Eşleştirilmiş t	wilcoxon	Öncesi/sonrası, normallik şüpheli.
Tek yönlü ANOVA	kruskal	≥ 3 grup, normallik yok.
Pearson r	spearmanr	Monoton ama doğrusal olmayan ilişki.

```
1 print(f"Mann-Whitney U (Adelie vs Chinstrap): p = {st.mannwhitneyu(a, c).pvalue:.4f}")
```

```

2 print(f"Kruskal-Wallis (3 tur): H = {st.kruskal(*gruplar).statistic:.2f}, "
3       f"p = {st.kruskal(*gruplar).pvalue:.3e}")

```

```

Mann-Whitney U (Adelie vs Chinstrap): p = 0.5476
Kruskal-Wallis (3 tur): H = 212.09, p = 8.837e-47

```

28.1 Permütasyon Testi: Varsayimsız Çıkarım

Formül

H_0 doğruysa grup etiketleri *anlamsızdır*. O hâlde etiketleri binlerce kez karıştırıp gözlenen farkın rastgele farklar arasında nerede durduğuna bakarız. Hiçbir dağılım varsayımı gerektirmez.

Nedir? Niçin? Nerede?

Fikir, tüm istatistikteki en zarif fikirlerden biridir. H_0 “iki grup arasında fark yok” diyorsa, mantıksal sonucu şudur: *hangi gözlemin hangi gruba ait olduğu önemli değildir*. O hâlde etiketleri karıştırmak hiçbir şeyi değiştirmemeli.

Deneyelim: etiketleri rastgele karıştır, iki grubun farkını hesapla, kaydet. Bunu 20.000 kez yap. Elinde “ H_0 doğruysa farklar nasıl görünürdü” dağılımı oluşur. Şimdi *gerçek* farkın bu dağılımda nerede durduğuna bak. Uçlardaysa, etiketler önemli demektir. p -değeri, basitçe “rastgele farkların yüzde kaç gerçek farktan büyük?” sorusunun cevabıdır.

Niçin bu kadar güçlü?

- **Hiçbir dağılım varsayımı yok.** Normallik, varyans eşitliği, örneklem büyüklüğü — hiçbirini gerekmez. H_0 ’ın kendisinden başka varsayım yapmaz.
- **Her istatistiğe uygulanabilir.** Ortalama farkı yerine medyan farkı, korelasyon, iki modelin AUC farkı, hatta “en büyük değerin oranı” gibi tuhaf bir istatistik — fonksiyonu değiştirmen yeter, gerisi aynı.
- **Anlaşılması kolay.** Bir yöneticiye t dağılımını anlatmak zordur; “etiketleri 20.000 kez karıştırdık, gerçek farkımız kadar büyük bir fark yalnızca %3 oranında çıktı” cümlesi ise anında anlaşılır.

Bootstrap’tan farkı nedir? İkisi de yeniden örnekleme yöntemidir ama farklı soruları cevaplar. *Bootstrap* iadeli çeker ve “tahminimin belirsizliği ne?” der (güven aralığı). *Permütasyon* iadesiz karıştırır ve “bu fark şansla açıklanabilir mi?” der (hipotez testi). Kabaca: bootstrap → güven aralığı, permütasyon → p -değeri.

Nerede kullanılır? A/B testinde standart dışı metrikler (%95’lik yanıt süresi, gelir medyanı); genomikte binlerce genin karşılaştırılması; makine öğrenmesinde iki modelin performans farkının anlamlılığı; görüntü işlemede permütasyon tabanlı öznelite önemi (Kısım 11’deki *permütasyon önemi* tam olarak bu fikirdir).

Tek maliyeti: hesap zamanı. 20.000 permütasyon × büyük veri kümesi yavaş olabilir. Ama modern donanımda bu genellikle saniyeler meselesidir — ve doğru cevap için ödenmeye değer.

```

1 def perm_test(a, b, B=20_000, rng=rng):
2     gozlenen = a.mean() - b.mean()
3     hepsi = np.concatenate([a, b])
4     n = len(a)
5     sayac = 0
6     for _ in range(B):
7         p = rng.permutation(hepsi)
8         if abs(p[:n].mean() - p[n:].mean()) >= abs(gozlenen):
9             sayac += 1

```

```

10     return gozlenen, (sayac + 1)/(B + 1)           # +1: yanlışlık düzeltmesi
11
12 d, p = perm_test(a.to_numpy(), c.to_numpy())
13 print(f"Adelie vs Chinstrap: fark = {d:.2f}    p = {p:.4f}")
14
15 # scipy'nin hazır surumu
16 r = st.permutation_test((a.to_numpy(), c.to_numpy()),
17                          lambda x, y: np.mean(x) - np.mean(y),
18                          n_resamples=20_000, random_state=42)
19 print(f"scipy permutation p = {r.pvalue:.4f}")

```

```

Adelie vs Chinstrap: fark = -26.92    p = 0.6816
scipy permutation p = 0.6803

```

İpucu

Permütasyon testi t -testiyle ($p = 0,65$) aynı sonucu veriyor — ama hiçbir varsayım yapmadan. Karmaşık istatistikler (medyan farkı, korelasyon, model doğruluğu farkı) için tek pratik yol budur. Modern istatistiğin sloganı: *formül bulamıyorsan yeniden örnekle*.

29. Güç Analizi ve Çoklu Karşılaştırma

İstatistiksel Güç

Güç $= 1 - \beta$ = gerçek bir etkiyi yakalama olasılığı. Dört büyüklük birbirine bağlıdır: α , güç, etki büyüklüğü d , örneklem n . Üçünü verirsen dördüncüsü belirlenir.

Nedir? Niçin? Nerede?

Güç nedir? “Gerçekten bir etki varsa, onu yakalama olasılığım nedir?” Bir mikroskobun büyütme gücü gibi düşün: küçük bir şeyi görmek istiyorsan yeterince güçlü bir mikroskop lazımdır. İstatistikte “mikroskop gücü” = örneklem büyüklüğüdür.

Niçin önceden hesaplanmalı? Çünkü güçsüz bir çalışma **iki yönden birden işe yaramaz**:

- Etki varsa bile büyük ihtimalle bulamazsın — para, zaman ve (klinik çalışmalarda) insan katılımı boşa gider.
- Daha kötüsü: güçsüz bir çalışmada *anlamlı çıkan* sonuçlar sistematik olarak abartılıdır. Gürültünün içinden ancak çok büyük tesadüfi farklar eşığı geçebilir, dolayısıyla yayımlanan etki gerçeğinden büyük görünür. Bilimdeki *tekrarlanabilirlik krizinin* teknik sebeplerinden biri budur.

Dört büyüklük nasıl birbirine bağlı? Sabit α ve d için n artınca güç artar. Sabit n için küçük etkileri yakalamak zordur — tablodaki $d = 0,2$ için grup başına 394, $d = 0,8$ için yalnızca 26 kişi gerekiyor. **Küçük etkileri aramak pahalıdır** ve bu tek satır, deney bütçelerinin neden bu kadar büyük olduğunu açıklar.

Nerede kullanılır?

- **A/B testi planlama:** “Dönüşümde %1 artışı yakalamak istiyorum; günde 5.000 ziyaretçim var; testi kaç gün çalıştırmalıyım?” Cevap doğrudan güç analizinden gelir — ve testi erken durdurmanın niçin yasak olduğunu da açıklar.
- **Klinik protokoller:** Etik kurullar örneklem büyüklüğü gerekçesi olmadan çalışma onaylamaz; gereğinden az hastayla çalışmak etik dışı sayılır (hastalar boşuna riske girer).
- **Sosyal bilim ve psikoloji:** Ön kayıt (pre-registration) sürecinin zorunlu bileşeni.

En sık yapılan hata: Deneyi bitirdikten sonra “gözlenen güç” hesaplamak. Bu sayı p -değerinin bir dönüşümünden ibarettir ve hiçbir yeni bilgi taşımaz. Güç analizi *tasarım* aracıdır, *savunma* aracı değil.

```
1 from statsmodels.stats.power import TTestIndPower
2 an = TTestIndPower()
3
4 for d_ in [0.2, 0.5, 0.8]:
5     n_ = an.solve_power(effect_size=d_, power=0.8, alpha=0.05)
6     print(f"d = {d_}: %80 güc için grup başına n = {np.ceil(n_):.0f}")
7
8 for n_ in [20, 50, 100]:
9     print(f"n = {n_}>3: d=0.5 için güc = {an.power(effect_size=0.5, nobs1=n_, alpha=0.05):.4f}")
```

```
d = 0.2: %80 güc için grup başına n = 394
d = 0.5: %80 güc için grup başına n = 64
d = 0.8: %80 güc için grup başına n = 26
n = 20: d=0.5 için güc = 0.3379
n = 50: d=0.5 için güc = 0.6969
n = 100: d=0.5 için güc = 0.9404
```

Dikkat

$n = 20$ ile orta büyüklükte bir etkiyi yakalama şansın %34. Yani “anlamlı çıkmadı” sonucu çoğu zaman “etki yok” değil, “bakmaya yetecek verim yoktu” demektir. Güç analizini veri toplamadan **önce** yap; sonradan yapılan “gözlenen güç” hesabı bilgi taşımaz.

29.1 Çoklu Karşılaştırma Düzeltmesi

Nedir? Niçin? Nerede?

Sorun nedir? $\alpha = 0,05$ demek, “hiçbir gerçek etki yokken bile 20 testten birinde yanlış alarm veririm” demektir. Tek test yaparken bu kabul edilebilir. Ama 20 test yaparsan, en az bir yanlış alarm alma olasılığın $1 - 0,95^{20} = \%64$ ’e fırlar. 100 test yaparsan $\%99,4$. Aşağıdaki simülasyon bunu tam olarak gösteriyor.

Bu niçin bir tuzak? Çünkü modern veri analizinde *farkında olmadan* yüzlerce test yaparsın: 30 sütunlu bir veri setinde “hangi değişkenler hedefle ilişkili?” diye bakmak 30 testtir; segmentlere bölüp tekrar bakmak 300; bir de zaman aralıklarına bölersen 3000. Bunların içinde “anamlı” bir şey bulacağın garantidir — olmasa bile.

İki farklı garanti, iki farklı yöntem:

- **FWER (aile bazında hata oranı)** — “*tek bir yanlış pozitif bile istemiyorum.*” Bonferroni (α/m) ve Holm bunu sağlar. Muhafazakârdır: az sayıda kritik karar varsa doğru tercih (ilaç onayı, güvenlik testi).
- **FDR (yanlış keşif oranı)** — “*Bulduklarımın en fazla %5’i yanlış olsun.*” Benjamini–Hochberg bunu sağlar. Çok daha güçlüdür; binlerce test yaptığın keşif amaçlı çalışmalarda (genomik, A/B test platformları, öznetelik taraması) standarttır.

Aşağıdaki çıktıda fark görünüyor: Bonferroni 1 keşif, FDR 2 keşif veriyor. Hangisinin doğru olduğu, yanlış pozitifin sana neye mal olduğuna bağlıdır.

En tehlikeli hâli — p -hacking. Veriye bakıp, anlamlı çıkan karşılaştırmayı seçip, sanki en baştan onu planlamışsın gibi raporlamak. Buna “*araştırmacı serbestlik dereceleri*” denir ve teknik olarak yalan söylemeden tamamen yanıltıcı sonuçlar üretmenin en kolay yoludur. Panzehiri: hipotezleri veri toplamadan önce yazmak (ön kayıt) ve yapılan *tüm* testleri raporlamak.

```
1 # Aynı veriyle 20 bağımsız test yaparsak, hiçbir gerçek etki olmasa bile...
2 sayac = 0
3 for _ in range(10_000):
4     ps = [st.ttest_ind(rng.normal(0,1,30), rng.normal(0,1,30)).pvalue
5             for _ in range(20)]
6     sayac += (min(ps) < 0.05)
7 print(f"en az bir 'anamlı' sonuc bulma oranı = {sayac/10_000:.4f}")
8 print(f"teorik 1 - 0.95^20 = {1 - 0.95**20:.4f}")
```

```
en az bir 'anamlı' sonuc bulma oranı = 0.6409
teorik 1 - 0.95^20 = 0.6415
```

```
1 from statsmodels.stats.multitest import multipletests
2 ps = np.array([0.001, 0.008, 0.039, 0.041, 0.042, 0.06, 0.074, 0.205])
3 for m in ['bonferroni', 'holm', 'fdr_bh']:
4     rej, adj, _, _ = multipletests(ps, alpha=0.05, method=m)
5     print(f"{m:>12}: {rej.sum()} red    duzeltilmis p = {np.round(adj, 4)}")
```

```
bonferroni: 1 red    duzeltilmis p = [0.008 0.064 0.312 0.328 0.336 0.48  0.592 1.   ]
holm: 1 red    duzeltilmis p = [0.008 0.056 0.234 0.234 0.234 0.234 0.234 0.234]
fdr_bh: 2 red    duzeltilmis p = [0.008 0.032 0.0672 0.0672 0.0672 0.08  0.0846 0.205]
```

Bilgi

Bonferroni en muhafazakârdır (FWER’i kontrol eder, gücü düşürür); **Holm** aynı garantiyi daha güçlü verir; **Benjamini–Hochberg (FDR)** ise “yanlış keşiflerin oranını” kontrol eder ve keşif amaçlı çalışmalarda (genomik, A/B testi panoları, öznelilik seçimi) standarttır.

Ölü Somon Balığının Beyin Aktivitesi

2009’da bir grup sinirbilimci, fMRI çalışmalarındaki çoklu karşılaştırma sorununu göstermek için bir deney yaptı: **ölü** bir Atlantik somonunu tomografiye soktular ve ona insan fotoğrafları gösterip “bu kişinin duygusunu belirle” dediler.

Standart analiz yöntemiyle (düzeltme yapmadan), somonun beyinde *istatistiksel olarak anlamlı* aktivite bölgeleri bulundu. Sebep basit: bir fMRI taramasında ~ 130.000 vokseldir ve her biri ayrı bir testtir. $\alpha = 0,05$ ile, saf gürültüden bile binlerce “anlamlı” vokseldir çıkar.

Çalışma bir mizah ödülü (Ig Nobel) kazandı ama etkisi ciddi oldu: nörogörüntüleme alanında çoklu karşılaştırma düzeltilmesi artık zorunlu. **Ders:** Yeterince çok yere bakarsan, hiçbir şeyin olmadığı yerde bile bir şey bulursun.

30. Bayesçi Çıkarıma Giriş**Bayesçi Güncelleme**

$$\underbrace{p(\theta | x)}_{\text{sonsal}} \propto \underbrace{p(x | \theta)}_{\text{olabilirlik}} \cdot \underbrace{p(\theta)}_{\text{önsel}}$$

Sıklıkçı yaklaşımda parametre sabit, veri rastgeledir; Bayesçi yaklaşımda veri sabit, parametre hakkındaki *inanç* bir dağılımdır.

Nedir? Niçin? Nerede?

İki felsefe arasındaki gerçek fark nedir? Sorduğun soru değişiyor:

- **Sıklıkçı:** “Etki sıfır olsaydı, bu veriyi görme olasılığım ne olurdu?” (p -değeri) — yani veri hakkında konuşur.
- **Bayesçi:** “Bu veriyi gördüğüme göre, etki hakkında ne düşünmeliyim?” (sonsal dağılım) — yani **parametre hakkında** konuşur.

İkinci soru, herkesin aslında sormak istediği sorudur. “ $p = 0,03$ ” cümlesini duyan hemen herkes onu “%97 ihtimalle etki var” diye anlar — bu yorum sıklıkçı çerçevede *yanlıştır*, ama Bayesçi çerçevede tam olarak elde ettiğin şeydir.

Güven aralığı mı, güvenilir aralık mı? Sıklıkçı %95 güven aralığını “gerçek değer buradadır” diye okuyamazsın (Kısım 5’teki uyarıyı hatırla). Bayesçi %95 *güvenilir aralık* (credible interval) ise tam olarak bunu söyler: “verilen model ve önsele göre, parametrenin bu aralıkta olma olasılığı %95.” Karar vericiye anlatması kat kat kolaydır.

Önsel bir zayıflık mı, güç mü? Eleştiri şudur: “önseli sen seçiyorsun, öznel.” Cevap iki katmanlı: (1) Sıklıkçı analizde de model seçimi, değişken seçimi, α seçimi öznel kararlardır — Bayesçi yaklaşım öznelliği en azından *açıkça yazmaya* zorlar. (2) Önsel, gerçek bir bilgi kaynağıdır: bir ilacın etkisini tahmin ederken önceki 40 çalışmayı yok saymak bilgi israfıdır.

Önselin etkisi ne zaman kaybolur? Veri arttıkça. Aşağıdaki çıktıda 94 gözlemle bile güçlü önsel sonucu 0,958’den 0,886’ya çekiyor; ama 10.000 gözlem olsaydı üç önsel de neredeyse aynı sonucu verirdi. **Az veri varken önsel değerlidir, çok veri varken zararsızdır** — bu, Bayesçi yaklaşımın küçük örneklemelerde niçin daha kararlı olduğunu açıklar.

Nerede kullanılır?

- **A/B testi:** “B’nin A’dan iyi olma olasılığı %94” ve “beklenen kayıp %0,2” gibi doğrudan karar verilebilir çıktılar. Ayrıca testi istediğin an durdurabilirsin — sıklıkçı testte erken bakmak Tip I hatayı şişirir, Bayesçi çerçevede bu sorun yoktur.
- **Soğuk başlangıç problemleri:** Yeni bir ürünün 3 satışı var, 2’si iade edildi. Sıklıkçı tahmin %67 iade oranı der (saçma); Bayesçi yaklaşım, kategori ortalamasını önsel olarak makul bir tahmin üretir. Ürün derecelendirmelerinde kullanılan “Bayesçi ortalama” tam olarak budur.
- **Spam filtreleri, tıbbi teşhis, arıza tahmini:** kanıt biriktikçe inancın güncellenmesi gereken her yer.
- **Hiyerarşik modeller:** Az veriye sahip alt grupları, genel ortalamaya doğru “büzererek” kararlı tahmin üretmek (*partial pooling*).

Eşlenik (conjugate) önsel ne demek? Bazı önsel-olabilirlik çiftlerinde sonsal, önselle aynı ailedendir ve güncelleme basit bir toplama işlemine iner: $\text{Beta}(\alpha, \beta)$ önseli + binom verisi $\rightarrow \text{Beta}(\alpha + k, \beta + n - k)$. Yani “ α ’ya başarıları, β ’ya başarısızlıkları ekle.” Bu, önseli “hayalî geçmiş gözlem” olarak okumayı mümkün kılar: $\text{Beta}(10, 10)$, “daha önce 10 başarı 10 başarısızlık görmüş gibiyim” demektir. Kapalı formül olmayan modellerde ise MCMC (PyMC, NumPyro) devreye girer.

Soru	Sıklıkçı	Bayesçi
Çıktı	p -değeri, güven aralığı	Sonsal dağılım, güvenilir aralık
Yorum	“Yöntemin %95’i kapsar”	“%95 olasılıkla bu aralıkta”
Önbilgi	Kullanılmaz	Önsel olarak modele girer
Küçük veri	Zayıf	Önsel sayesinde daha kararlı

```

1 # Beta-Binom eslenik (conjugate) model: 1. sınıf kadın yolcular
2 sub = tit[(tit.pclass == 1) & (tit.sex == 'female')]
3 k, N = sub.survived.sum(), len(sub)
4 print(f"veri: k = {k} hayatta / n = {N}")
5
6 for a_, b_, ad in [(1, 1, 'Uniform (bilgisiz)'), (2, 2, 'Beta(2,2)'),
7                  (10, 10, 'Beta(10,10) - güçlü')]:
8     post = st.beta(a_ + k, b_ + N - k)          # eslenik guncelleme
9     print(f"{ad:>22}: sonsal ort = {post.mean():.4f}  "
10          f"%95 guvenilir aralik = [{post.ppf(.025):.4f}, {post.ppf(.975):.4f}]")

```

```

veri: k = 91 hayatta / n = 94
  Uniform (bilgisiz): sonsal ort = 0.9583  %95 guvenilir aralik = [0.9105, 0.9884]
    Beta(2,2): sonsal ort = 0.9490  %95 guvenilir aralik = [0.8978, 0.9831]
Beta(10,10) - güçlü: sonsal ort = 0.8860  %95 guvenilir aralik = [0.8218, 0.9373]

```

Bilgi

Güçlü bir önsel (Beta(10,10), yani “oran muhtemelen 0,5 civarı”) sonsalı 0,886’ya çekiyor; bilgisiz önselde 0,958. Veri arttıkça önselin etkisi kaybolur — 94 gözlemde bile fark belirgin. **Önsel seçimi bir tercihtir ve şeffaf biçimde raporlanmalıdır.** Karmaşık modeller için PyMC veya NumPyro ile MCMC kullanılır.

KISIM 7

Regresyon ve İstatistiksel Modelleme

31. Basit Doğrusal Regresyon

Regresyon Niçin Bu Kadar Merkezî?

Korelasyon (Kısım 2) “bu iki değişken birlikte hareket ediyor mu?” diye sorar ve tek bir sayı verir. Regresyon bir adım ileri gider: ilişkiyi **bir denkleme çevirir**. Bu, üç yeni şey kazandırır:

- **Tahmin:** “Kanadı 210 mm olan bir penguen kaç gram gelir?” Korelasyon bunu cevaplayamaz, regresyon cevaplar.
- **Miktar:** “1 mm kanat, kaç gram eder?” Katsayı doğrudan bu sayıdır — ve birimlidir, dolayısıyla yorumlanabilir.
- **Kontrol:** Birden çok değişkeni aynı anda modele koyup “diğerleri sabitken bunun etkisi ne?” diye sorabilirsin. Gözlemsel veride karıştırıcı değişkenleri hesaba katmanın temel yolu budur.

İstatistik ile makine öğrenmesi arasındaki köprü budur. Aynı denklem iki farklı amaçla kullanılır: istatistikçi *katsayıya* bakar (“kanat uzunluğunun etkisi anlamlı mı?”), makine öğrenmecisi *tahmine* bakar (“yeni penguenin ağırlığını ne kadar iyi bilirim?”). Kısım 7 birinci bakışı, Kısım 8–9 ikinci bakışı öğretir — ama model aynı modeldir.

Adı niçin “regresyon”? Francis Galton 1880’lerde uzun boylu babaların oğullarının babalarından *daha kısa*, kısa boylu babaların oğullarının *daha uzun* olduğunu fark etti: değerler ortalamaya doğru “geri çekiliyordu”. Buna *ortalamaya regresyon* dedi ve yöntem adını buradan aldı. Bu olgu bugün hâlâ yanlış yorumlanır: “en kötü performans gösteren şubeye eğitim verdik, düzeldi” cümlesi çoğu zaman eğitimin değil, ortalamaya regresyonun sonucudur.

Model ve En Küçük Kareler

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \quad \varepsilon_i \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2)$$

Kalıntı kareler toplamını $\sum (y_i - \beta_0 - \beta_1 x_i)^2$ minimize eden çözüm:

$$\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} = r \frac{s_y}{s_x}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}.$$

Nedir? Niçin? Nerede?

Model ne diyor? “ y , x ’in doğrusal bir fonksiyonudur, artı biraz rastgele gürültü.” Üç parçası var: β_0 (doğrunun nereden başladığı), β_1 (eğim — asıl ilgilendiğimiz sayı), ϵ (modelin açıklayamadığı her şey).

Niçin kareleri minimize ediyoruz? Üç cevabı var ve üçü de doğru:

- *Pratik:* Mutlak hataların toplamı türevlenebilir değildir ve kapalı çözümü yoktur; kareler toplamının türevi alınabilir ve tek satırlık bir formül verir.
- *İstatistiksel:* Hatalar normal dağılıyorsa, en küçük kareler çözümü *maksimum olabilirlik* çözümüyle birebir aynıdır (Kısım 5). Yani kareler keyfî bir seçim değil, normallik varsayımının doğal sonucudur.
- *Yorumsal:* Kare almak büyük hataları orantısız cezalandırır. Bu istenen bir şeydir — ama aynı zamanda regresyonun aykırı değerlere niçin bu kadar duyarlı olduğunun da sebebidir. Aykırı değer sorunu varsa *kuantil regresyon* veya Huber kaybı gibi sağlam alternatifler kullanılır.

$\hat{\beta}_1 = r s_y/s_x$ **eşitliğinin anlamı.** Eğim, korelasyonun *ölçeklenmiş* hâlidir. r birimsizdi; s_y/s_x çarpanı onu “gram/mm” birimine çevirir. Yani regresyon eğimi ile korelasyon aynı bilginin iki farklı kılığıdır — biri karşılaştırma için, diğeri tahmin için.

$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$ **ne söylüyor?** Regresyon doğrusu *her zaman* $(\bar{x}, \bar{y})$ noktasından geçer. Yani veri bulutunun ağırlık merkezinden geçen, eğimi $r s_y/s_x$ olan doğrudur.

Matris formu $(X^T X)^{-1} X^T y$ **niçin önemli?** Çünkü değişken sayısından bağımsızdır: 1 değişkenle de 100 değişkenle de aynı formül çalışır. statsmodels, scikit-learn ve R’ın içinde çalışan şey budur (gerçekte sayısal kararlılık için matris tersi yerine QR ayrıştırması kullanılır). Ayrıca $X^T X$ ’in tersinin alınamaz hâle gelmesi, ilerideki *çoklu doğrusal bağlantı* sorununun matematiksel tanımıdır.

Nerede kullanılır? Fiyat tahmini (ev, araç, sigorta primi); talep tahmini; doz–yanıt eğrileri; kalibrasyon (sensör okuması → gerçek değer); ekonometrik etki analizi (“asgari ücret artışı istihdamı nasıl etkiledi?”); ve makine öğrenmesindeki hemen her modelin temel taşı.

Türetim

$S(\beta_0, \beta_1) = \sum (y_i - \beta_0 - \beta_1 x_i)^2$ için kısmi türevleri sıfırlarsak *normal denklemler* elde edilir:

$$\frac{\partial S}{\partial \beta_0} = -2 \sum (y_i - \beta_0 - \beta_1 x_i) = 0,$$

$$\frac{\partial S}{\partial \beta_1} = -2 \sum x_i (y_i - \beta_0 - \beta_1 x_i) = 0.$$

Birinciden $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$, yerine koyunca $\hat{\beta}_1$ formülü çıkar. Matris gösteriminde çözüm şudur:

$$\hat{\beta} = (X^T X)^{-1} X^T y.$$

```

1 import numpy as np, pandas as pd, seaborn as sns
2 import statsmodels.api as sm, statsmodels.formula.api as smf
3
4 peng = sns.load_dataset('penguins').dropna()
5 m = smf.ols('body_mass_g ~ flipper_length_mm', data=peng).fit()
6 print(m.summary().tables[1])
7
8 # Elle doğrulama
9 x, y = peng.flipper_length_mm.values, peng.body_mass_g.values
10 b1 = np.sum((x - x.mean())*(y - y.mean()))/np.sum((x - x.mean())**2)

```

```

11 b0 = y.mean() - b1*x.mean()
12 print(f"\nmanuel: b0 = {b0:.4f}    b1 = {b1:.4f}")
13 print(f"R2 = {m.rsquared:.4f}    r^2 = {np.corrcoef(x, y)[0,1]**2:.4f}")

```

```

=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept      -5872.0927      310.285     -18.925      0.000     -6482.472     -5261.713
flipper_length_mm    50.1533        1.540      32.562      0.000         47.123         53.183
=====

manuel: b0 = -5872.0927    b1 = 50.1533
R2 = 0.7621    r^2 = 0.7621

```

Bilgi

Kanat uzunluğu 1 mm arttığında vücut kütlesi ortalama **50,15 g** artıyor. Sabit terim (−5872 g) fiziksel olarak anlamsızdır çünkü $x = 0$ veri aralığının dışındadır — sabit terimi ancak $x = 0$ anlamhyasa yorumla. Ayrıca basit regresyonda $R^2 = r^2$ 'dir: burada 0,7621.

31.1 summary() Çıktısını Okumak

Alan	Anlamı
R-squared	Açıklanan varyans oranı. Yeni değişken eklenince <i>asla</i> düşmez.
Adj. R-squared	Değişken sayısına ceza uygular; model karşılaştırmada bunu kullan.
F-statistic	“Tüm katsayılar sıfır” hipotezinin testi.
coef	Diğer değişkenler sabitken x 'te 1 birim artışın y 'ye etkisi.
std err	Katsayının standart hatası.
P> t	$H_0 : \beta_j = 0$ testinin p -değeri.
[0.025 0.975]	Katsayının %95 güven aralığı.
AIC / BIC	Model karşılaştırma ölçütleri (küçük olan iyi).
Durbin-Watson	Artık otokorelasyonu (≈ 2 olmalı).
Cond. No.	Çoklu doğrusal bağlantı işareti (> 30 şüpheli).

32. Çoklu Regresyon

Niçin Birden Çok Değişken?

Tek değişkenli regresyon şu soruyu cevaplar: “kanat uzunluğu ile ağırlık arasındaki ilişki ne?” Ama bu soru neredeyse her zaman yanıltıcıdır, çünkü *başka şeyler de* değişiyordur. Kanadı uzun penguenler aynı zamanda Gentoo türündendir ve Gentoo’lar zaten ağırdır. Tek değişkenli katsayı, bu iki etkiyi birbirine karıştırır.

Çoklu regresyon bu karışıklığı çözer. Katsayıların yorumu şuna dönüşür: “**diğer tüm değişkenler sabit tutulduğunda, bu değişkendeki 1 birimlik artışın etkisi.**” Aşağıdaki örnekte kanat katsayısı 50,15’ten 15,95’e düşüyor — aradaki 34 gramlık fark aslında *türün* etkisiymiş.

Bu, gözlemsel veriden nedensellik çıkarmaya en çok yaklaşılabildiğin yerdir. Randomize deney yapamadığın durumlarda (sigara–kanser ilişkisi, eğitim–gelir ilişkisi, ilaç–yan etki), bilinen karıştırıcıları modele koyarak onların etkisini “sabitlersin”. Ama dikkat: yalnızca *ölçtüğün* değişkenleri kontrol edebilirsin. Ölçülmemiş bir karıştırıcı varsa katsayın hâlâ yanlıdır — bu yüzden gözlemsel çalışmalar asla deney kadar güçlü kanıt sayılmaz.

Nerede kullanılır?

- **Fiyatlandırma:** Ev fiyatı $\sim$ metrekare + oda + yaş + konum. “Aynı mahallede, aynı yaşta bir evde her ek metrekare kaç TL?”
- **İK analitiği:** Maaş $\sim$ deneyim + eğitim + pozisyon + cinsiyet. Son katsayı, “aynı deneyim ve pozisyonda cinsiyete bağlı ücret farkı” sorusunun cevabıdır — ücret eşitliği davalarında kullanılan analiz budur.
- **Pazarlama:** Satış $\sim$ TV + dijital + radyo harcaması (*marketing mix modeling*); her kanalın marjinal katkısı.
- **Epidemiyoloji:** Hastalık riski $\sim$ risk faktörü + yaş + cinsiyet + eşlik eden hastalıklar.

Dikkat — “sabit tutmak” her zaman anlamlı değildir. Bir modelde hem “boy” hem “kilo” varsa, “boyu sabit tutup kiloyu artırmak” fiziksel olarak farklı bir insan demektir. Katsayıyı yorumlamadan önce, sabit tutulan senaryonun gerçek dünyada mümkün olup olmadığını sor.

```
1 m2 = smf.ols('body_mass_g ~ flipper_length_mm + bill_length_mm + bill_depth_mm'
2           ' + C(species) + C(sex)', data=peng).fit()
3 print(m2.summary().tables[1])
4 print(f"R2 = {m2.rsquared:.4f}    adj = {m2.rsquared_adj:.4f}    "
5       f"AIC = {m2.aic:.1f}    BIC = {m2.bic:.1f}")
```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-1460.9946	571.308	-2.557	0.011	-2584.911	-337.079
C(species)[T.Chinstrap]	-251.4767	81.079	-3.102	0.002	-410.980	-91.973
C(species)[T.Gentoo]	1014.6267	129.561	7.831	0.000	759.746	1269.507
C(sex)[T.Male]	389.8915	47.848	8.148	0.000	295.761	484.022
flipper_length_mm	15.9502	2.910	5.482	0.000	10.226	21.674
bill_length_mm	18.2044	7.106	2.562	0.011	4.225	32.184
bill_depth_mm	67.2176	19.742	3.405	0.001	28.380	106.055
=====						
R2 = 0.8750	adj = 0.8727	AIC = 4721.9	BIC = 4748.6			

Dikkat

Kanat uzunluğunun katsayısı 50,15 $\rightarrow$ 15,95’e düştü! Çünkü basit modelde bu katsayı, kanat uzunluğunun *tür farkıyla karışmış* toplam etkisini taşıyordu. Çoklu modelde katsayılar “*diğer değişkenler sabitken*” yorumlanır. Bu, **karıştırıcı değişken** (confounder) kontrolünün ta kendisidir — ve Kısım 2’deki Simpson paradoksunun regresyon dilindeki karşılığıdır.

32.1 Kukla Değişkenler, Etkileşim ve Polinom

```

1 # Kategorik: C(...) otomatik one-hot yapar, ilk seviye referans olur
2 smf.ols('body_mass_g ~ C(species)', data=peng).fit()
3
4 # Etkileşim: ':' sadece etkileşim, '*' ana etkiler + etkileşim
5 smf.ols('body_mass_g ~ flipper_length_mm * C(sex)', data=peng).fit()
6
7 # Polinom
8 smf.ols('body_mass_g ~ flipper_length_mm + I(flipper_length_mm**2)', data=peng).fit()
9
10 # Log dönüşümü (carpimsal etkiler)
11 smf.ols('np.log(body_mass_g) ~ np.log(flipper_length_mm)', data=peng).fit()

```

Bilgi

Log-log modelde katsayı doğrudan **esnekliktir**: “ x %1 artarsa y % β artar”. Yarı-log modelde (y log, x ham) katsayı %100 β ’lık değişimi verir. Ekonomi ve biyolojide bu yorumlanabilirlik yüzünden log dönüşümü çok yaygındır.

33. Varsayımlar ve Tanı Testleri

Varsayımlar Niçin Önemli — ve Ne Kadar Önemli?

Regresyon katsayıları neredeyse her veride hesaplanır; program hata vermez. Sorun şudur: **katsayının yanındaki p -değeri ve güven aralığı, varsayımlar sağlanmıyorsa yanıltır**. Yani model “çalışıyor” görünürken sana yanlış kesinlik satıyor olabilir.

Her varsayım neyi korur?

- **Doğrusallık** — ilişki gerçekten eğriyse, düz bir doğru uydurmak katsayıyı *yanlı* yapar. Sistemati olarak yanlış tahmin edersin. *Teşhis*: artık-tahmin grafiğinde muz şeklinde bir desen. *Çözüm*: polinom terim, log dönüşümü, spline veya doğrusal olmayan model.
- **Bağımsızlık** — en kritik varsayım. Aynı hastadan tekrarlı ölçüm, aynı okuldan öğrenciler, ardışık günlerin satışları: bunlar bağımsız değildir. İhlal edilirse standart hatalar *ciddi biçimde* küçük çıkar, her şey “anlamlı” görünür. *Çözüm*: karışık etkili modeller, kümelenmiş standart hatalar, zaman serisi modelleri.
- **Sabit varyans (homoskedastisite)** — tahmin büyüdükçe hata da büyüyorsa (gelir modellerinde tipiktir), standart hatalar yanıltır. *Teşhis*: artık grafiğinde huni şekli. *Çözüm*: log dönüşümü ya da sağlam standart hatalar (HC3).
- **Normal artıklar** — *en az kritik olan*. MLT sayesinde $n > 50$ ’de p -değerleri normalliğe pek duyarlı değildir. Yalnızca küçük örnekleme ve tekil tahmin aralıklarında önemlidir.
- **Çoklu bağlantı yok** — tahmini bozmaz ama katsayıları oynaklaştırır; sonraki alt bölümün konusu.

Öncelik sırası: Bağımsızlık > doğrusallık > sabit varyans > normallik. Zamanının çoğunu ilk ikisine harca.

Nerede kullanılır? Her ciddi regresyon raporunda tanı grafikleri zorunludur — akademik hakemlerin ilk isteyeceği şeydir. Üretimdeki bir modelde ise tanı grafikleri, *model bozulmasını* (data drift) erken yakalamanın yoludur: artıklarda yeni bir desen belirdiyse dünya değişmiştir.

Varsayım	İhlal edilirse	Test
Doğrusallık	Katsayılar yanlı	artık grafiği
Bağımsızlık	SE'ler yanlı	durbin_watson
Sabit varyans	SE'ler yanlı	het_breuschpagan
Normal artıklar	Küçük n 'de p 'ler yanlı	jarque_bera, Q-Q
Çoklu bağlantı yok	Katsayılar kararsız	variance_inflation_factor

```

1 from statsmodels.stats.diagnostic import het_breuschpagan
2 from statsmodels.stats.stattools import durbin_watson, jarque_bera
3
4 res = m2.resid
5 bp = het_breuschpagan(res, m2.model.exog)
6 print(f"Breusch-Pagan (sabit varyans): LM = {bp[0]:.3f} p = {bp[1]:.4f}")
7 print(f"Durbin-Watson (otokorelasyon): {durbin_watson(res):.4f}")
8 jb = jarque_bera(res)
9 print(f"Jarque-Bera (normallik): JB = {jb[0]:.3f} p = {jb[1]:.4f}")

```

```

Breusch-Pagan (sabit varyans): LM = 11.906 p = 0.0641
Durbin-Watson (otokorelasyon): 2.1686
Jarque-Bera (normallik): JB = 0.871 p = 0.6469

```

Üç varsayım da sağlanıyor: varyans sabit ($p > 0,05$), otokorelasyon yok ($DW \approx 2$), artıklar normal.

```

1 import matplotlib.pyplot as plt
2 fig, ax = plt.subplots(2, 2, figsize=(11, 8))
3 ax[0,0].scatter(m2.fittedvalues, res, s=12, alpha=.6)
4 ax[0,0].axhline(0, color='r', ls='--'); ax[0,0].set_title('Artık vs Tahmin')
5 sm.qqplot(res, line='45', fit=True, ax=ax[0,1]); ax[0,1].set_title('Q-Q')
6 ax[1,0].scatter(m2.fittedvalues, np.sqrt(np.abs(res/res.std()))), s=12, alpha=.6)
7 ax[1,0].set_title('Scale-Location')
8 sm.graphics.influence_plot(m2, ax=ax[1,1], criterion='cooks')
9 plt.tight_layout(); plt.show()

```

33.1 Çoklu Doğrusal Bağlantı (Multicollinearity)

VIF

$$VIF_j = \frac{1}{1 - R_j^2}$$

R_j^2 : x_j 'nin diğer bağımsız değişkenlerle regresyonundaki R^2 . $VIF > 5$ şüpheli, > 10 ciddi sorundur.

Nedir? Niçin? Nerede?

Sorun nedir? İki değişken neredeyse aynı bilgiyi taşıyorsa (boy–kilo, metrekare–oda sayısı, Gentoo olmak–uzun kanat), model “etkinin hangisinden geldiğini” ayırt edemez. Sonsuz sayıda katsayı kombinasyonu neredeyse aynı tahmini üretir; model bunlar arasında rastgele birini seçer.

Belirtileri:

- Katsayıların standart hataları şişer, p -değerleri büyür — “hiçbir değişken anlamlı değil ama R^2 yüksek” tuhaflığı.
- Katsayıların *işareti* sezgiye aykırı çıkar (“oda sayısı arttıkça fiyat düşüyor”).
- Veriden birkaç gözlem çıkarınca katsayılar çarpıcı biçimde değişir.

Formül ne yapıyor? VIF_j , “ x_j ’yi diğer tüm bağımsız değişkenlerle ne kadar iyi tahmin edebiliyorum?” sorusunu ölçer. $R_j^2 = 0$ ise $VIF = 1$ (tamamen bağımsız). $R_j^2 = 0,9$ ise $VIF = 10$: x_j ’nin bilgisinin %90’ı zaten diğerlerinde var. Adındaki “varyans şişme faktörü” birebir doğrudur — katsayının varyansı tam olarak VIF katı büyür, standart hatası $\sqrt{VIF}$ katı.

Ne zaman umursamalı, ne zaman umursamamalı?

- *Amacın tahminse*: umursama. Çoklu bağlantı tahmin doğruluğunu bozmaz.
- *Amacın yorumsa*: ciddiye al. “Bu değişkenin etkisi şu kadar” diyeceksen, katsayının kararlı olması şart.

Çözümler: (1) Fazla olan değişkeni çıkar — en basit ve genellikle en iyi. (2) Değişkenleri birleştir (boy ve kilo yerine vücut kitle indeksi). (3) PCA ile ilişkisiz bileşenlere geç (yorumlanabilirlik kaybı pahasına). (4) Ridge regresyonu — katsayıları küçülterek kararlı hâle getirir; zaten Ridge’in icat edilme sebebi tam olarak budur.

```
1 from statsmodels.stats.outliers_influence import variance_inflation_factor
2 X = pd.get_dummies(peng[['flipper_length_mm', 'bill_length_mm', 'bill_depth_mm',
3                           'species', 'sex']], drop_first=True).astype(float)
4 X = sm.add_constant(X)
5 vif = pd.DataFrame({'degisken': X.columns,
6                     'VIF': [variance_inflation_factor(X.values, i)
7                             for i in range(X.shape[1])])
8 print(vif.round(2).to_string(index=False))
```

```
    degisken  VIF
    const    1.00
flipper_length_mm  6.69
  bill_length_mm  6.07
  bill_depth_mm  6.08
species_Chinstrap  4.31
  species_Gentoo 15.55
      sex_Male    2.31
```

Dikkat

species_Gentoo için $VIF = 15,6$: Gentoo olmak, uzun kanat ve sığ gaga ile neredeyse eşanlamlı. Çoklu bağlantı *tahmin performansını* bozmaz ama *katsayı yorumunu* güvenilmez kılar (standart hatalar şişer, işaretler oynaklaşır). Çözümler: değişken çıkarma, PCA, ya da Ridge regresyonu.

33.2 Sağlam (Robust) Standart Hatalar

```
1 m_rob = m2.get_robustcov_results(cov_type='HC3') # heteroskedastisiteye dayanıklı
```

```

2 print(pd.DataFrame({'coef': m2.params.round(2),
3                     'SE_normal': m2.bse.round(2),
4                     'SE_HC3': m_rob.bse.round(2)}))

```

	coef	SE_normal	SE_HC3
Intercept	-1460.99	571.31	550.55
C(species) [T.Chinstrap]	-251.48	81.08	77.62
C(species) [T.Gentoo]	1014.63	129.56	121.88
C(sex) [T.Male]	389.89	47.85	45.87
flipper_length_mm	15.95	2.91	2.72
bill_length_mm	18.20	7.11	6.43
bill_depth_mm	67.22	19.74	18.93

İpucu

Sabit varyans şüphesi varsa `cov_type='HC3'` kullanmak neredeyse bedavadır: varsayım sağlanıyorsa sonuç değişmez, sağlanmıyorsa seni yanlış çıkarımdan korur. Kümelenmiş veride (aynı okul, aynı hasta, tekrarlı ölçüm) `cov_type='cluster'` gerekir.

34. Lojistik Regresyon

Formül

$$\log \frac{p}{1-p} = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k \quad \Leftrightarrow \quad p = \frac{1}{1 + e^{-(\beta_0 + \sum \beta_j x_j)}}$$

Katsayıların üstelini almak **bahis oranı** (odds ratio) verir: e^{β_j} , x_j 'de 1 birim artışın bahis oranını kaç katına çıkardığıdır.

Nedir? Niçin? Nerede?

Niçin doğrusal regresyon yetmiyor? Hedef ikiliyse (hayatta kaldı / kalmadı, tıkladı / tıklamadı) doğrusal model iki büyük sorun üretir: (1) $\hat{y}$ negatif ya da 1'den büyük çıkabilir — “%130 hayatta kalma olasılığı” anlamsızdır; (2) hata varyansı sabit olamaz. Lojistik regresyon bu iki sorunu tek hamlede çözer.

Çözüm nasıl çalışıyor — üç adım:

1. *Olasılık* $p \in [0, 1]$ sınırlıdır — doğrusal modelleyemezsin.
2. *Bahis* (odds) $= p/(1-p)$ ise $[0, \infty)$ 'a açılır. “4'e 1 bahis” demek $p = 0,8$ demektir; bahisçilerin dilinden alınmıştır.
3. *Log-bahis* (logit) $= \log \frac{p}{1-p}$ ise $(-\infty, +\infty)$ 'a açılır — işte artık doğrusal modelleyebilirsin.

Yani logit dönüşümü, “sıkışmış” bir büyüklüğü doğrusal modelin çalışabileceği sonsuz eksene açmanın yoludur. Ters çevirince *sigmoid* (S eğrisi) elde edersin: girdi ne kadar uç olursa olsun, çıktı hep 0 ile 1 arasında kalır.

Katsayılar nasıl yorumlanır? Doğrusal regresyondaki gibi “ x 1 artınca y şu kadar artar” diyemezsin, çünkü etki doğrusal değildir (eğrinin ortasında dik, uçlarında yatıktır). Bunun yerine:

- β_j — log-bahisteki değişim. Yorumlanması zordur, işaretine bakılır.
- e^{β_j} — **bahis oranı**. Asıl kullanılan budur. $e^\beta = 2$ demek “bahis iki katına çıkar”; $e^\beta = 0,5$ demek “yarıya iner”. 1 ise etki yok.

Bahis oranı ile risk oranını karıştırma. “Bahis 13 kat arttı” ile “olasılık 13 kat arttı” aynı şey değildir. Taban olasılık düşükken ($p < 0,1$) ikisi birbirine yakındır; yüksekken ciddi biçimde ayrışır. Sağlık haberlerindeki en yaygın çarpıtma budur.

Nerede kullanılır — muhtemelen en çok kullanılan istatistiksel modeldir:

- **Kredi skorlama:** “Bu başvuru sahibi temerrüde düşer mi?” Bankacılık düzenlemeleri yorumlanabilirlik istediği için, derin öğrenme çağında bile lojistik regresyon standart kalmıştır.
- **Tıp:** Hastalık risk skorları (Framingham kalp riski skoru bir lojistik regresyondur); yoğun bakım mortalite tahmini.
- **Pazarlama:** Müşteri kaybı (churn) tahmini, tıklama oranı tahmini, dönüşüm modellemesi.
- **Makine öğrenmesi:** Sınıflandırmanın temel referans modelidir — ve bir sinir ağının son katmanı, tam olarak bir lojistik regresyondur.

Nasıl eğitilir? En küçük karelerle değil, *maksimum olabilirlikle* (Kısım 5). Minimize edilen negatif log-olabilirlik, makine öğrenmesinde *log-loss* veya *binary cross-entropy* adıyla bilinir — aynı fonksiyonun üç farklı adı.

Bir Katsayının Milyonlarca Dolarlık Anlamı

Bir bankanın kredi skorlama modelinde “son 6 ayda gecikmiş ödeme sayısı” değişkeninin katsayısı $\beta = 0,82$, yani $OR = e^{0,82} = 2,27$. Yorumu: her ek gecikmiş ödeme, temerrüt bahsini **2,27 katına** çıkarıyor.

Bu tek sayı doğrudan iş kararına dönüşür: onay eşiği belirlenir, faiz oranı riske göre fiyatlanır, hangi başvuruların insan incelemesine gideceği kararlaştırılır. Modelin *yorumlanabilir* olması burada bir lüks değil yasal zorunluluktur — birçok ülkede banka, reddedilen bir başvuru sahibine “niçin reddedildiniz?” sorusunu cevaplamak zorundadır. Bir rastgele orman bu cevabı veremez, lojistik regresyon verir. Model seçiminin her zaman yalnızca doğruluk meselesi olmadığını en net örneği budur.

```

1 tit = sns.load_dataset('titanic').dropna(
2     subset=['age', 'embarked', 'fare', 'sex', 'pclass', 'survived'])
3
4 lg = smf.logit('survived ~ C(pclass) + C(sex) + age + fare + sibsp',
5     data=tit).fit(disps=0)
6 print(lg.summary().tables[1])

```

	coef	std err	z	P> z	[0.025	0.975]
Intercept	4.1636	0.502	8.291	0.000	3.179	5.148
C(pclass)[T.2]	-1.3008	0.320	-4.068	0.000	-1.928	-0.674
C(pclass)[T.3]	-2.5189	0.335	-7.528	0.000	-3.175	-1.863
C(sex)[T.male]	-2.6075	0.216	-12.091	0.000	-3.030	-2.185
age	-0.0443	0.008	-5.357	0.000	-0.061	-0.028
fare	0.0017	0.002	0.709	0.478	-0.003	0.007
sibsp	-0.3917	0.123	-3.183	0.001	-0.633	-0.150

```

1 odds = pd.DataFrame({'coef': lg.params, 'odds_ratio': np.exp(lg.params)})
2 odds[['ci_lo', 'ci_hi']] = np.exp(lg.conf_int())
3 print(odds.round(4))
4 print(f"\nPseudo R2 = {lg.prsquared:.4f}    n = {int(lg.nobs)}")
5 print(f"dogruluk = {(lg.predict() > 0.5).astype(int) == tit.survived}.mean():.4f")

```

	coef	odds_ratio	ci_lo	ci_hi
Intercept	4.1636	64.3055	24.0322	172.0690
C(pclass)[T.2]	-1.3008	0.2723	0.1455	0.5096
C(pclass)[T.3]	-2.5189	0.0806	0.0418	0.1552
C(sex)[T.male]	-2.6075	0.0737	0.0483	0.1125
age	-0.0443	0.9566	0.9413	0.9723
fare	0.0017	1.0017	0.9969	1.0066
sibsp	-0.3917	0.6759	0.5310	0.8603

Pseudo R2 = 0.3386 n = 712
dogruluk = 0.8076

Katsayıları Yorumlamak

- sex[male]: OR = 0,074 — erkek olmanın hayatta kalma bahsi, kadınının %7,4'ü. Yani kadın olmak bahsi $\approx 13,6$ kat artırıyor.
- pclass[3]: OR = 0,081 — 3. sınıf, 1. sınıfa göre çok dezavantajlı.
- age: OR = 0,957 — her yıl bahsi %4,3 azaltıyor; 10 yaş farkı $0,957^{10} = 0,64$ demek.
- fare: $p = 0,478$ — sınıf zaten modelde olduğu için bilet ücreti ek bilgi taşıyor.

35. Genelleştirilmiş Doğrusal Modeller (GLM)

GLM Nedir — Tek Bir Çatı

Şu ana kadar iki model gördük: sürekli hedef için OLS, ikili hedef için lojistik. GLM, bunların ikisini de kapsayan **tek bir genel çerçevedir**. Her GLM üç parçadan oluşur:

1. *Doğrusal tahmin edici*: $\eta = \beta_0 + \beta_1 x_1 + \dots$ — her modelde aynı.
2. *Bağ fonksiyonu (link)*: η 'yı hedefin ortalamasına bağlar. OLS'te kimlik ($\mu = \eta$), lojistikte logit, Poisson'da logaritma.
3. *Dağılım ailesi*: Hedefin hangi dağılımdan geldiği — Gauss, binom, Poisson, gamma...

Niçin bu genelleme değerli? Çünkü “hedefim ne tür bir sayı?” sorusunu cevaplayınca doğru model kendiliğinden belirlenir:

- *Sayım verisi* (günlük arıza, aylık ziyaret, kopma sayısı) → Poisson. OLS kullanırsan negatif tahminler üretir ve varyans yapısını yanlış modeller.
- *Oran/ikili* → binom + logit.
- *Pozitif ve sağa çarpık* (sigorta hasarı, bekleme süresi, harcama tutarı) → gamma + log bağ. Sigortacılıkta prim hesabının standart modelidir.
- *Aşırı yayılmış sayım* → negatif binom.

Log bağ niçin bu kadar kullanışlı? Katsayılar *çarpımsal* yorum kazanır: e^β , “o değişken 1 birim artınca beklenen sayı kaç katına çıkar?” sorusunu cevaplar. Aşağıdaki örnekte `wool[T.B]` katsayısı $-0,206$, yani $e^{-0,206} = 0,81$: B tipi yün kullanmak kopma sayısını %19 azaltıyor. Bu, üretim hattında doğrudan uygulanabilir bir bulgudur.

Aşırı yayılım niçin bu kadar önemli? Poisson, $\text{Var} = \mathbb{E}$ olduğunu varsayar. Gerçek sayım verilerinde varyans neredeyse her zaman daha büyüktür (olaylar birbirini tetikler, birimler heterojendir). Bu varsayım tutmazsa model *çok fazla kesinlik* raporlar: standart hatalar küçük çıkar, p -değerleri sahte anlamlılık üretir. Aşağıdaki örnek bunu net biçimde gösteriyor — aynı katsayılar, 5 kat büyük standart hatalar ve kaybolan “anlamlılık”. Teşhis tek satırdır: `deviance/df` oranı 1'e yakın mı?

Yanıt tipi	Aile / bağ	Örnek
Sürekli	Gaussian / identity	Fiyat, kütle (OLS).
İkili	Binomial / logit	Hayatta kalma, tıklama.
Sayım	Poisson / log	Arıza sayısı, ziyaret sayısı.
Aşırı yayılmış sayım	NegativeBinomial / log	Varyans > ortalama olan sayımlar.
Pozitif çarpık	Gamma / log	Sigorta hasarı, süre.

```

1 # Gerçek veri: warpbreaks (dokuma tezgahında iplik kopma sayıları, n=54)
2 wb = sm.datasets.get_rdataset('warpbreaks').data
3 po = smf.glm('breaks ~ C(wool) + C(tension)', data=wb,
4             family=sm.families.Poisson()).fit()
5 print(po.summary().tables[1])
6 print(f"deviance/df = {po.deviance/po.df_resid:.4f}    (1'e yakın olmalı)")

```

	coef	std err	z	P> z	[0.025	0.975]
Intercept	3.1735	0.056	57.002	0.000	3.064	3.283

```

C(wool)[T.B]      -0.2060      0.052      -3.994      0.000      -0.307      -0.105
C(tension)[T.L]   0.5185      0.064      8.107      0.000      0.393      0.644
C(tension)[T.M]   0.1972      0.068      2.885      0.004      0.063      0.331
=====
deviance/df = 4.2078      (1'e yakın olmalı)

```

Dikkat

deviance/df = 4,21 $\gg$ 1: **aşırı yayılım** (overdispersion) var. Poisson, $\text{Var} = \mathbb{E}$ varsayar; gerçek varyans daha büyük. Sonuç: standart hatalar olduğundan küçük çıkar ve *sahte anlamlılık* üretilir. Çözüm negatif binom regresyondur.

```

1 nb = smf.glm('breaks ~ C(wool) + C(tension)', data=wb,
2             family=sm.families.NegativeBinomial(alpha=1.0)).fit()
3 print(nb.summary().tables[1])
4 print(f"NB deviance/df = {nb.deviance/nb.df_resid:.4f}")
5 print(f"Poisson AIC = {po.aic:.1f}    NB AIC = {nb.aic:.1f}")

```

```

=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
Intercept      3.1591      0.278      11.369      0.000      2.614      3.704
C(wool)[T.B]   -0.1823      0.277      -0.658      0.511      -0.726      0.361
C(tension)[T.L] 0.5102      0.340      1.503      0.133      -0.155      1.176
C(tension)[T.M] 0.2168      0.340      0.637      0.524      -0.450      0.884
=====
NB deviance/df = 0.1433
Poisson AIC = 493.1    NB AIC = 475.5

```

Bilgi

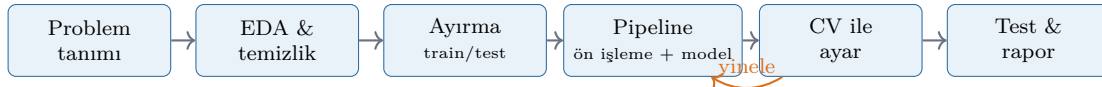
Katsayılar neredeyse aynı kaldı ama standart hatalar **5 kat** büyüdü ve “çok anlamlı” bulgular anlamsızlaştı. AIC de negatif binomu destekliyor ($475,5 < 493,1$). Bu, yanlış dağılım varsayımının nasıl sahte keşifler ürettiğinin ders kitabı örneğidir.

KISIM 8

Makine Öğrenmesinin Temelleri

36. Problem Tipleri ve İş Akışı

Tip	Girdi → Çıktı	Örnek
Denetimli — regresyon	$X \rightarrow y$ (sürekli)	Ev fiyatı, talep tahmini.
Denetimli — sınıflandırma	$X \rightarrow y$ (kategorik)	Tümör iyi/kötü huylu, spam.
Denetimsiz — kümeleme	$X \rightarrow$ gruplar	Müşteri segmentasyonu.
Denetimsiz — boyut indirgeme	$X \rightarrow Z$ ($\dim Z \ll \dim X$)	Görselleştirme, gürültü azaltma.
Denetimsiz — anomali	$X \rightarrow$ normal/anormal	Dolandırıcılık, arıza tespiti.



37. Eğitim/Test Ayrımı ve Çapraz Doğrulama

Dikkat

Altın kural: test seti, model geliştirme sürecinin *hiçbir* adımında kullanılmaz — ölçekleme parametrelerini, öznitelik seçimini, hiperparametreleri, hatta “hangi modeli deneyeyim” kararını bile etkilemez. Test setine bakarak alınan her karar, o setin tarafsızlığını yok eder.

```

1 import numpy as np, pandas as pd
2 from sklearn.datasets import load_breast_cancer
3 from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate
4 from sklearn.pipeline import make_pipeline
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.linear_model import LogisticRegression
7
8 bc = load_breast_cancer(as_frame=True)
9 X, y = bc.data, bc.target
10 print(bc.data.shape, np.bincount(y), bc.target_names)      # (569, 30) [212 357]
  
```

```
11
12 Xtr, Xte, ytr, yte = train_test_split(
13     X, y, test_size=0.25, random_state=42, stratify=y)    # stratify: sınıf oranı korunur
14 print(f"pozitif oran: egitim={ytr.mean():.3f} test={yte.mean():.3f}")
```

```
(569, 30) [212 357] ['malignant' 'benign']
pozitif oran: egitim=0.627 test=0.629
```

```
1 lr_ham = LogisticRegression(max_iter=5000, random_state=42).fit(Xtr, ytr)
2 pipe   = make_pipeline(StandardScaler(),
3                        LogisticRegression(max_iter=5000, random_state=42)).fit(Xtr, ytr)
4 print(f"ölçeklemesiz test doğrulugu = {lr_ham.score(Xte, yte):.4f}")
5 print(f"ölçekli test doğrulugu = {pipe.score(Xte, yte):.4f}")
```

```
ölçeklemesiz test doğrulugu = 0.9580
ölçekli test doğrulugu = 0.9860
```

Bilgi

Sadece ölçekleme ekleyerek doğruluk %95,8'den %98,6'ya çıktı. Mesafe ve gradyan temelli her yöntem (lojistik regresyon, SVM, k -NN, sinir ağları, PCA) ölçeklemeye *muhtaçtır*. Ağaç temelli yöntemler (karar ağacı, Random Forest, Gradient Boosting) ise ölçeklemeden etkilenmez.

37.1 Çapraz Doğrulama

Nedir? Niçin? Nerede?

Tek bir eğitim/test ayrımı niçin yetmiyor? Çünkü sonucun, hangi %20'nin teste düştüğüne bağlıdır. `random_state` değerini değiştirdiğinde doğruluğun %94'ten %97'ye zıplayabilir. Hangisi gerçek? İkisi de değil — ikisi de tek bir örneklemin gürültüsü.

Çapraz doğrulama ne yapıyor? Veriyi k eşit parçaya böler. Sırayla her parçayı test, kalan $k - 1$ parçayı eğitim olarak kullanır. Sonunda k skorun ortalamasını alır. Böylece *her gözlem tam bir kez* test edilmiş olur — ve elde ettiğin skor, tek bir şanslı/şanssız bölmeye değil, verinin tamamına dayanır.

Bu, doğrudan Kısım 5'teki fikrin uygulamasıdır: tek bir tahmin yerine *tahminin dağılımına* bakarsın. k skorun standart sapması, model performansının belirsizliğini verir — ve bu sayıyı raporlamak, ortalamayı raporlamak kadar önemlidir. “%97,4 ± 1,7” cümlesi, “%97,4” cümlesinden çok daha dürüsttür.

k kaç olmalı? 5 veya 10 standarttır. Küçük k : hızlı ama eğitim kümesi küçüldüğü için skorlar biraz karamsar çıkar. Büyük k : daha doğru ama k kat pahalı. n çok küçükse (< 50) `LeaveOneOut` ($k = n$) kullanılır.

Doğru stratejiyi seçmek — en kritik karar. Yandaki tablo bir kontrol listesidir:

- **Sınıflandırma yapıyorsan** her zaman `StratifiedKFold`. Sıradan `KFold`, dengesiz veride bir katmanda hiç pozitif örnek bırakmayabilir.
- **Aynı birimden birden çok satır varsa** (aynı hasta, aynı kullanıcı, aynı mağaza) mutlaka `GroupKFold`. Aksi hâlde aynı hastanın bir kaydı eğitimde, diğeri testte olur — model hastayı tanır, hastalığı değil. Bu, veri sızıntısının en sinsi biçimidir.
- **Zaman boyutu varsa** `TimeSeriesSplit`. Rastgele bölmek, modelin geleceği görmesine izin verir; ölçtüğün skor gerçek hayatta asla tekrarlanmaz.

Nerede kullanılır? Model karşılaştırma; hiperparametre arama (`GridSearchCV` içeride bunu çağdırır); öznitelik seçiminin faydasını ölçme; ve en önemlisi — bir modelin üretime çıkmadan önceki tek dürüst performans tahmini.

Strateji	Ne zaman?
<code>KFold</code>	Genel amaçlı regresyon.
<code>StratifiedKFold</code>	Sınıflandırma — sınıf oranlarını korur (varsayılan yap).
<code>GroupKFold</code>	Aynı hasta/kullanıcı/mağaza birden çok satırdaysa.
<code>TimeSeriesSplit</code>	Zaman serisi — geleceği görmeyi engeller.
<code>RepeatedStratifiedKFold</code>	Küçük veride kararlı kestirim.
<code>LeaveOneOut</code>	Çok küçük veri ($n < 50$); pahalı ve yüksek varyanslı.

```

1 cv = StratifiedKFold(5, shuffle=True, random_state=42)
2 res = cross_validate(pipe, X, y, cv=cv,
3                       scoring=['accuracy', 'roc_auc', 'f1', 'precision', 'recall'])
4 for k in ['test_accuracy', 'test_roc_auc', 'test_f1', 'test_precision', 'test_recall']:
5     print(f"{k:>16}: {res[k].mean():.4f}")

```

```

test_accuracy: 0.9737
test_roc_auc: 0.9953
test_f1: 0.9794
test_precision: 0.9683
test_recall: 0.9916

```

38. Veri Sızıntısı: En Pahalı Hata

Sızıntı Nedir ve Niçin Bu Kadar Tehlikeli?

Veri sızıntısı, modelin eğitim sırasında *tahmin anında sahip olamayacağı* bir bilgiye erişmesidir. Sonuç yıkıcıdır: model laboratuvarında muhteşem skorlar verir, üretimde çöker. Ve fark etmesi zordur — çünkü hiçbir hata mesajı almazsın, aksine *çok iyi* sonuçlar alırsın.

En yaygın dört biçimi:

1. **Ön işlemeyi bölmeden önce yapmak.** StandardScaler'ı tüm veriye uygulayıp sonra bölersen, ölçekleyici test kümesinin ortalamasını ve sapmasını görmüş olur. Aynı şey SimpleImputer, PCA, öznitelik seçimi ve hedef kodlama (target encoding) için de geçerlidir — sonuncusu en tehlikelidir. *Çözüm:* her şeyi Pipeline içine koy; çapraz doğrulama her katmanda ön işlemeyi baştan uydurur.
2. **Geleceği kullanmak.** “Müşteri ayrılacak mı?” modelinde iptal_tarihi sütununu öznitelik olarak vermek. Model %100 doğruluk verir — ve tamamen işe yaramazdır.
3. **Aynı birimin hem eğitimde hem testte olması.** Aynı hastanın iki taraması, aynı kullanıcının iki oturumu. GroupKFold ile önlenir.
4. **Test kümesine bakarak karar vermek.** Test skoruna göre model seçip aynı test skorunu raporlamak. Test kümesi *bir kez* kullanılır; ayar için ayrı bir doğrulama kümesi (ya da iç içe çapraz doğrulama) gerekir.

Tehhis işaretleri: Skorların “çok iyi” olması (%99,9 doğruluk — gerçek problemler nadiren bu kadar kolaydır); beklenmedik bir özniteliğin öneminin çok yüksek çıkması; üretimde performansın çakılması. **Kural: Sonuç gerçek olamayacak kadar iyiye, gerçek değildir.**

Neden “en pahalı hata”? Çünkü diğer hatalar (yanlış model, kötü hiperparametre) seni yalnızca *daha kötü* bir modele götürür ve bunu ölçebilirsin. Sızıntı ise ölçüm aracının kendisini bozar — performansı doğru ölçemediğin için hatalı modeli üretime alırsın, ve maliyeti orada öğrenirsin.

Tanım

Veri sızıntısı (data leakage), eğitim sırasında modele gerçek kullanımda erişemeyeceği bilginin sızmasıdır. Sonuç: harika doğrulama skorları, felaket bir üretim performansı.

```

1 from sklearn.feature_selection import SelectKBest, f_classif
2 from sklearn.pipeline import Pipeline
3 from sklearn.model_selection import cross_val_score
4
5 # Etkiyi çok net görmek için: TAMAMEN rastgele veri (gerçek bilgi = SIFIR)
6 rng = np.random.default_rng(0)
7 Xr = rng.normal(size=(100, 5000))
8 yr = rng.integers(0, 2, 100)
9
10 # YANLIŞ: özellik seçimini TUM veride yap, sonra CV
11 sec = SelectKBest(f_classif, k=20).fit(Xr, yr)
12 X_sizinti = sec.transform(Xr)
13 print("sızıntılı CV =",
14       f"{cross_val_score(LogisticRegression(max_iter=2000), X_sizinti, yr, cv=5).mean():.4f}")
15
16 # DOĞRU: seçim pipeline içinde, her katlamada yeniden
17 pipe_dogru = Pipeline([('sel', SelectKBest(f_classif, k=20)),
18                        ('clf', LogisticRegression(max_iter=2000))])

```

```
19 print("dogru      CV =", f"{cross_val_score(pipe_dogru, Xr, yr, cv=5).mean():.4f}")
```

```
sizintili CV = 0.8500
dogru     CV = 0.4800
```

Dikkat

Veride *hiçbir* gerçek sinyal yokken sızıntı %85 doğruluk üretiyor; doğru yöntem ise %48 (yani yazı-tura) diyor. Bu, yayımlanmış pek çok “mucize model”in gerçek açıklamasıdır.

Sızıntı türü	Nasıl önlenir?
Ön işleme sızıntısı	Ölçekleme/doldurma/kodlamayı Pipeline içine koy.
Öznitelik seçimi sızıntısı	Seçimi de pipeline’a al; her katlamada yeniden yap.
Hedef sızıntısı	Hedefin sonucunu içeren sütunları at (örn. “iptal tarihi” ile “iptal etti mi”).
Zamansal sızıntı	Zaman serisinde TimeSeriesSplit; geleceği kullanma.
Grup sızıntısı	Aynı kişi hem eğitimde hem testte olmasın → GroupKFold.
Yinelenen satırlar	Ayırmadan önce drop_duplicates().

39. Yanlılık–Varyans Dengesi

Hata Ayırıştırması

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{yanlılık}^2} + \underbrace{\text{Var}(\hat{f}(x))}_{\text{varyans}} + \underbrace{\sigma^2}_{\text{indirgenemez}}$$

Basit modeller yüksek yanlılık (eksik öğrenme), karmaşık modeller yüksek varyans (aşırı öğrenme) gösterir.

Nedir? Niçin? Nerede?

Bu denklem makine öğrenmesinin en önemli tek denklemidir. Model hatanın nereden geldiğini üçe ayırır ve her biri farklı bir tedavi gerektirir:

- **Yanlılık (bias)** — modelin *yanlış varsayımı*. Gerçek ilişki eğriyken düz doğru uydurmak gibi. Ne kadar veri verirsene ver düzelmez, çünkü sorun veride değil modelin kendisindedir. Belirtisi: *hem eğitim hem test hatası yüksek* — **eksik öğrenme** (underfitting).
- **Varyans** — modelin *aşırı hassasiyeti*. Eğitim verisindeki gürültüyü de ezberler; birkaç örneği değiştirsen model tamamen başkalaşır. Belirtisi: *eğitim hatası çok düşük, test hatası yüksek* — **aşırı öğrenme** (overfitting).
- **İndirgenemez hata (σ^2)** — verinin kendi gürültüsü. Aynı öz niteliklere sahip iki ev farklı fiyata satılır. Bu, hiçbir modelin aşamayacağı tabandır. Bunu bilmek önemlidir: “%3 hataya inelim” hedefi, taban %5 ise imkânsız bir hedeftir.

Niçin “denge”? Çünkü ikisini aynı anda küçültemezsin. Modeli karmaşıktırdıkça yanlılık düşer ama varyans yükselir. Toplam hata bu yüzden U şeklinde bir eğri çizer — en iyi model, en karmaşık model değil, bu eğrinin dibindeki modeldir.

Teşhis çok basit: Eğitim ve test skorlarını karşılaştır.

- Eğitim %70, test %68 → yüksek yanlılık. *Çözüm:* daha karmaşık model, daha çok öz nitelik, daha az düzenleme. Daha çok veri işe yaramaz.
- Eğitim %99, test %72 → yüksek varyans. *Çözüm:* daha çok veri, düzenleme (Ridge/Lasso, dropout), daha basit model, topluluk yöntemleri.

Bu tek karşılaştırma, “modelim niçin kötü ve ne yapmalıyım?” sorusunun pratikteki en hızlı cevabıdır.

Nerede karşına çıkar? Aslında her yerde — ve daha önce de çıktı: Kısım 5’teki Bessel düzeltmesi tartışması (yanlı ama düşük varyanslı tahmin edici), Ridge regresyonunun kasıtlı olarak yanlılık ekleyip varyans düşürmesi, rastgele ormanın çok sayıda yüksek varyanslı ağacı ortalayarak varyansı düşürmesi, boosting’in zayıf (yüksek yanlılıklı) öğrencileri birleştirerek yanlılığı düşürmesi. Kısım 9’daki tüm algoritmalar, bu dengede farklı bir noktada durur.

Sınava Çalışmanın İki Yanlış Yolu

Denge kavramını sezmenin en kolay yolu bir öğrenci benzetmesidir:

- **Yüksek yanlılık:** Öğrenci konuyu hiç çalışmamış, her soruya “B” işaretliyor. Deneme sınavında da gerçek sınavda da aynı düşük notu alır — tutarlı biçimde kötü.
- **Yüksek varyans:** Öğrenci geçen yılın sorularını *cevaplarıyla birlikte ezberlemiş*. Deneme sınavında 100 alıyor. Ama gerçek sınavda sorular biraz değişince çakılıyor — çünkü konuyu değil, o soruların cevaplarını öğrendi.
- **Denge:** Öğrenci konuyu anlamış. Deneme sınavında 85, gerçek sınavda 82 alıyor. En yüksek deneme notu onun değil, ama gerçek sınavdaki en yüksek not onun.

Eğitim skoru, deneme sınavı notudur — ve hiç kimse deneme notuna göre işe alınmaz. Model seçerken bakılması gereken tek şey test (veya çapraz doğrulama) skorudur.

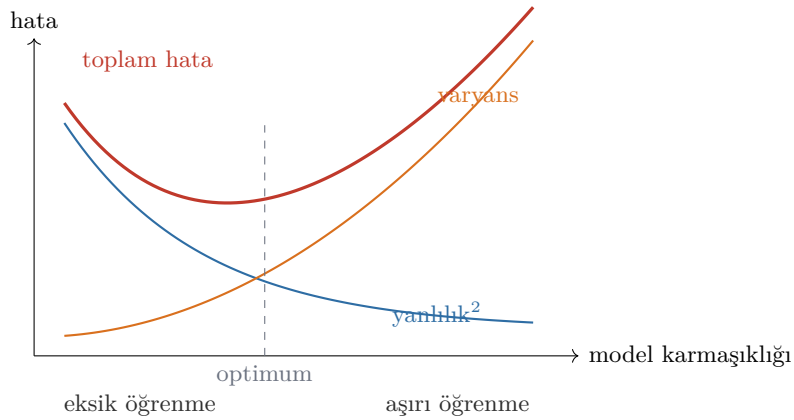
```
1 from sklearn.preprocessing import PolynomialFeatures
2 from sklearn.linear_model import LinearRegression
3
4 rng = np.random.default_rng(0)
```

```

5 Xp = rng.uniform(-3, 3, (60, 1))
6 yp = 0.5*Xp[:,0]**2 - Xp[:,0] + 2 + rng.normal(0, 1.2, 60) # gercek: 2. derece
7 Xtr2, Xte2, ytr2, yte2 = train_test_split(Xp, yp, test_size=0.35, random_state=0)
8
9 print("derece | egitim MSE | test MSE")
10 for d in [1, 2, 3, 5, 9, 15]:
11     m = make_pipeline(PolynomialFeatures(d), LinearRegression()).fit(Xtr2, ytr2)
12     tr = np.mean((m.predict(Xtr2) - ytr2)**2)
13     te = np.mean((m.predict(Xte2) - yte2)**2)
14     print(f"{d:>6} | {tr:10.3f} | {te:8.3f}")

```

derece	egitim MSE	test MSE
1	2.211	4.003
2	1.382	1.825
3	1.373	1.816
5	1.174	1.689
9	0.842	2.748
15	1.241	7.890



Bilgi

Tablo eğrinin sayısal karşılığı: derece 1'de *eksik öğrenme* (hem eğitim hem test hatası yüksek), derece 3-5'te optimum, derece 15'te *aşırı öğrenme* (eğitim hatası düşük, test hatası 7,89'a fırlıyor). Eğitim ile test hatası arasındaki *makas*, aşırı öğrenmenin göstergesidir.

39.1 Öğrenme Eğrileri: Daha Fazla Veri İşe Yarar mı?

```

1 from sklearn.model_selection import learning_curve
2 import matplotlib.pyplot as plt
3
4 boyut, tr_sk, te_sk = learning_curve(
5     pipe, X, y, cv=cv, train_sizes=np.linspace(0.1, 1.0, 8),
6     scoring='accuracy', n_jobs=-1)
7
8 plt.plot(boyut, tr_sk.mean(1), 'o-', label='egitim')
9 plt.plot(boyut, te_sk.mean(1), 's-', label='dogrulama')
10 plt.fill_between(boyut, te_sk.mean(1)-te_sk.std(1), te_sk.mean(1)+te_sk.std(1), alpha=.2)
11 plt.xlabel('egitim ornegi sayisi'); plt.ylabel('dogruluk'); plt.legend(); plt.show()

```

İpucu

Öğrenme eğrisi okuma: (1) İki eğri de düşük ve birleşmişse *yüksek yanlılık* → daha karmaşık model, daha iyi öznelilik. (2) Arada büyük makas varsa *yüksek varyans* → daha çok veri, düzenleme, daha basit model. (3) Doğrulama eğrisi düzleşmişse **daha fazla veri toplamak para kaybıdır.**

40. Ön İşleme ve Pipeline

```

1 from sklearn.compose import ColumnTransformer
2 from sklearn.impute import SimpleImputer
3 from sklearn.preprocessing import OneHotEncoder, StandardScaler
4 from sklearn.pipeline import Pipeline
5 import seaborn as sns
6
7 tit = sns.load_dataset('titanic')
8 sayisal = ['age', 'fare', 'sibsp', 'parch']
9 kategorik = ['sex', 'embarked', 'class', 'who', 'alone']
10
11 on_isleme = ColumnTransformer([
12     ('num', Pipeline([('imp', SimpleImputer(strategy='median')),
13                       ('sc', StandardScaler())])), sayisal,
14     ('cat', Pipeline([('imp', SimpleImputer(strategy='most_frequent')),
15                       ('oh', OneHotEncoder(handle_unknown='ignore',
16                                           drop='first'))])), kategorik,
17 ])
18
19 model = Pipeline([('prep', on_isleme),
20                  ('clf', LogisticRegression(max_iter=2000, random_state=42))])

```

Dönüştürücü	Ne yapar / ne zaman?
StandardScaler	$(x - \mu)/\sigma$. Varsayılan tercih.
MinMaxScaler	[0, 1] aralığına sıkıştırır; sinir ağlarında yaygın.
RobustScaler	Medyan ve IQR kullanır; aykırı değer varsa.
PowerTransformer	Yeo-Johnson/Box-Cox; çarpıklığı giderir.
QuantileTransformer	Dağılımı normale/düzyüne zorlar.
OneHotEncoder	Kategorik → ikili sütunlar. drop='first' eşdoğrusallığı önler.
OrdinalEncoder	Sıralı kategoriler için (ağaç modellerinde de uygun).
TargetEncoder	Yüksek kardinaliteli kategoriler; sızıntıya dikkat.
SimpleImputer / KNNImputer	Eksik değer doldurma.
PolynomialFeatures	Etkileşim ve polinom terimleri.

41. Değerlendirme Metrikleri

41.1 Sınıflandırma

Niçin Tek Bir Metrik Yetmiyor?

“Modelim %99 doğru” cümlesi çoğu zaman hiçbir şey ifade etmez. Bir hastalık toplumda binde bir görülüyorsa, *herkese “sağlıklı” diyen* bir model %99,9 doğruluk alır — ve tek bir hastayı bile bulamaz. Doğruluk, sınıflar dengesizken tamamen yanıltıcıdır (Kısım 3’teki taban oranı yanılgısının makine öğrenmesindeki karşılığı).

Önce dört sayıyı tanı — her metrik bunlardan türer:

- **TP** (doğru pozitif): Hasta dedin, gerçekten hasta.
- **TN** (doğru negatif): Sağlıklı dedin, gerçekten sağlıklı.
- **FP** (yanlış pozitif): Hasta dedin, aslında sağlıklı — *yanlış alarm*, Tip I hata.
- **FN** (yanlış negatif): Sağlıklı dedin, aslında hasta — *kaçırma*, Tip II hata.

İki temel metrik, iki farklı soru:

- **Kesinlik (precision)** = $TP/(TP + FP)$: “Pozitif dediklerimin kaçını gerçekten pozitif?” Alarm verdiğinde ne kadar güvenilirsin. *Yanlış alarm pahalıysa* bunu optimize et — spam filtresi (önemli e-postayı silmek felaket), otomatik kredi reddi, hedefli pahalı pazarlama kampanyası.
- **Duyarlılık (recall)** = $TP/(TP + FN)$: “Gerçek pozitiflerin kaçını yakaladım?” Kaçırma oranının ne. *Kaçırmak pahalıysa* bunu optimize et — kanser taraması, dolandırıcılık tespiti, güvenlik kamerası, üretim hattında kusurlu parça tespiti.

İkisi **birbiriyle çelişir**: eşiği düşürünce daha çok pozitif dersin, duyarlılık artar ama kesinlik düşer. Bu tam olarak Kısım 6’daki α - β dengesidir — aynı matematik, farklı isimler.

F1 niçin harmonik ortalama? Kısım 2’de gördüğümüz gibi harmonik ortalama küçük değere yapışır. Kesinlik 0,99, duyarlılık 0,02 olan bir model $F1 = 0,039$ alır — “bir tarafı berbat model berbatır”. Aritmetik ortalama alsaydık 0,5 çıkardı ve model iyi görünürdü.

ROC-AUC ne ölçer? Eşikten *bağımsız* olarak, modelin rastgele bir pozitif rastgele bir negatiften daha yüksek puanlama olasılığı. 0,5 = madenî para atmak, 1,0 = kusursuz sıralama. Model karşılaştırmak için idealdir çünkü eşik seçimini denklemden çıkarır.

Ne zaman PR-AUC? Sınıflar ciddi biçimde dengesizse. ROC eğrisi, çoğunluk sınıfı çok büyükse yanıltıcı biçimde iyi görünür (paydadaki TN devasadır, FP oranını bastırır). PR eğrisi TN’yi hiç kullanmaz, bu yüzden azınlık sınıfına odaklanır. *Kural*: pozitif sınıf %10’un altındaysa PR-AUC raporla.

Log loss niçin ayrı bir metrik? Diğerleri yalnızca kararın doğru olup olmadığına bakar; log loss *ne kadar emin olduğuna* da bakar. “%51 ihtimalle hasta” ile “%99 ihtimalle hasta” aynı kararı verir ama çok farklı bilgilerdir. Olasılığı bir sonraki adımda kullanacaksan (risk fiyatlandırma, beklenen değer hesabı, eşik optimizasyonu) log loss ve kalibrasyon (Kısım 11) kritik hâle gelir.

Metrik	Formül	Ne zaman?
Doğruluk	$\frac{TP + TN}{N}$	Dengeli sınıflar.
Kesinlik	$\frac{TP}{TP + FP}$	Yanlış alarmanın maliyeti yüksekse.
Duyarlılık	$\frac{TP}{TP + FN}$	Kaçırmanın maliyeti yüksekse (tıp!).
F1	$\frac{2PR}{P + R}$	Kesinlik-duyarlılık dengesi.
ROC-AUC	eğri altı alan	Eşikten bağımsız sıralama kalitesi.
PR-AUC	eğri altı alan	Dengesiz veride ROC'tan iyi.
Log loss	$-\frac{1}{N} \sum [y \log \hat{p} + (1 - y) \log(1 - \hat{p})]$	Olasılık kalitesi (kalibrasyon).

```

1 from sklearn.metrics import (confusion_matrix, classification_report,
2                               roc_auc_score, average_precision_score, log_loss)
3
4 yhat = pipe.predict(Xte)
5 yprob = pipe.predict_proba(Xte)[:, 1]
6
7 cm = confusion_matrix(yte, yhat)
8 tn, fp, fn, tp = cm.ravel()
9 print(f"TN={tn} FP={fp} FN={fn} TP={tp}")
10 print(f"roc_auc = {roc_auc_score(yte, yprob):.4f}")
11 print(f"pr_auc = {average_precision_score(yte, yprob):.4f}")
12 print(f"log_loss = {log_loss(yte, yprob):.4f}")
13 print(classification_report(yte, yhat, target_names=bc.target_names))

```

```

TN=52 FP=1 FN=1 TP=89
roc_auc = 0.9977
pr_auc = 0.9986
log_loss = 0.0679

```

	precision	recall	f1-score	support
malignant	0.98	0.98	0.98	53
benign	0.99	0.99	0.99	90
accuracy			0.99	143
macro avg	0.99	0.99	0.99	143
weighted avg	0.99	0.99	0.99	143

Dikkat

Bu problemde malignant (kötü huylu) sınıfı için **1 yanlış negatif** var: bir kanser vakası kaçırıldı. Tıbbi tanıda bu, yanlış alarmdan çok daha pahalıdır. Doğru yaklaşım, karar eşiğini 0,5'te bırakmak yerine duyarlılığı artıracak biçimde düşürmektir (Kısım 11).

41.2 Regresyon

Hangi Hata Metriği, Niçin?

Hepsi “tahminim gerçekten ne kadar uzak?” sorusunu ölçer, ama *hataları nasıl cezalandırdıkları* farklıdır — ve bu fark, modelinin neyi optimize edeceğini belirler.

- **MSE / RMSE — büyük hatalar orantısız kötüdür.** Kare aldığı için 10 birimlik bir hata, 1 birimlik on hatadan *10 kat* daha ağır sayılır. *Ne zaman doğru:* Tek bir büyük hata felaketse — ilaç dozu, uçuş rotası, elektrik şebekesi yük tahmini, stok planlaması. Ayrıca matematiksel olarak uysal olduğu için (türevlenebilir) neredeyse tüm modeller varsayılan olarak MSE optimize eder.
- **MAE — her hata kendi büyüklüğü kadar kötüdür.** Aykırı değerlerden etkilenmez. *Ne zaman doğru:* Veride silemediğin uç gözlemler varsa; “ortalama olarak kaç TL yanıliyorum?” sorusu iş için anlamlıysa. Not: MSE’yi minimize eden tahmin *ortalamadır*, MAE’yi minimize eden *medyandır* — yani metrik seçimi, modelin neyi tahmin ettiğini değiştirir.
- **RMSE mi MAE mi?** RMSE her zaman MAE’den büyük veya eşittir; aradaki fark ne kadar açıksa, hata dağılımında o kadar çok uç değer var demektir. İkisini birlikte raporlamak bu yüzden bilgilendiricidir.
- **R^2 — “modelim hiçbir şey bilmemekten ne kadar iyi?”** Payda, her zaman ortalamayı tahmin eden aptal modelin hatasıdır. $R^2 = 0,76$ demek “o aptal modele göre hatanın %76’sını yok ettim” demektir. R^2 negatif de olabilir — modelin ortalamadan kötüyse. Birimsiz olduğu için farklı problemler arasında karşılaştırılabilir; ama *aynı* problemde model seçerken RMSE daha somuttur.
- **MAPE — yüzde cinsinden hata.** İş birimlerine anlatması en kolay metriktir (“%8 sapmayla tahmin ediyoruz”). *Tuzağı:* y sıfıra yaklaşıncı patlar, ve düşük tahminleri yüksek tahminlerden daha az cezalandırır (asimetriktir). Sıfıra yakın değerler varsa MAPE yerine sMAPE veya MAE kullan.

Pratik kural: Metriği modelden *önce* seç ve iş sorusuna göre seç. “Hangi metrik daha iyi çıkıyorsa onu raporlarım” yaklaşımı, sonuçları kendi lehine seçmenin bir biçimidir.

Yanlış Metrik, Yanlış Model

Bir e-ticaret şirketi günlük talep tahmini yapıyor. Ürünlerin %95’i günde 0–5 adet satıyor, %5’i ise kampanya günlerinde 500–1000 adet.

RMSE ile optimize edilen model, o %5’lik kampanya gününü kaçırmamaya odaklanır — çünkü oradaki hatanın karesi devasadır. Sonuç: normal günlerde sürekli fazla stok tahmini yapar, depo dolar.

MAE ile optimize edilen model ise %95’lik çoğunluğa odaklanır, kampanya günlerini kaçıır — ve o günlerde stoksuz kalınır, asıl para orada kaybedilir.

Doğru çözüm metriği değiştirmek değil, **problemi bölmektir**: normal günler ve kampanya günleri için ayrı modeller. Metrik tartışması çoğu zaman gizli bir “problem yanlış kurulmuş” sinyalidir.

Metrik	Formül	Özellik
MSE	$\frac{1}{n} \sum (y - \hat{y})^2$	Büyük hataları çok cezalandırır.
RMSE	$\sqrt{\text{MSE}}$	y ile aynı birim.
MAE	$\frac{1}{n} \sum y - \hat{y} $	Aykırı değere dayanıklı.
R^2	$1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}}$	Açıklanan varyans oranı.
MAPE	$\frac{1}{n} \sum \left \frac{y - \hat{y}}{y} \right $	Yüzde hata; $y \approx 0$ ’da patlar.

Dikkat

scikit-learn'de skorlar *büyük daha iyi* kuralına uyar; bu yüzden `scoring='neg_mean_squared_error'` gibi negatif isimler görürsün. `cross_val_score` çıktısının işaretini çevirmeyi unutma.

KISIM 9

Denetimli Öğrenme Algoritmaları

42. Algoritma Haritası

Algoritma	Güçlü yanı	Zayıf yanı
Lojistik regresyon	Yorumlanabilir, hızlı, kalibre olasılık	Yalnız doğrusal sınır
k -NN	Varsayımsız, basit	Yavaş tahmin, boyut laneti, ölçekleme şart
Naive Bayes	Çok hızlı, az veriyle çalışır	Bağımsızlık varsayımı gerçekçi değil
SVM	Yüksek boyutta güçlü, kernel esnekliği	Büyük veride yavaş, olasılık vermez
Karar ağacı	Yorumlanabilir, ölçekleme gerekmez	Kararsız, kolay aşırı öğrenir
Random Forest	Sağlam, ayar gerektirmez	Yorumlanması zor, büyük bellek
Gradient Boosting	Genelde en yüksek doğruluk	Ayar hassas, aşırı öğrenebilir
Sinir ağı	Karmaşık örüntüler	Çok veri ister, kara kutu

42.1 Hepsini Aynı Veride Karşılaştırmak

```

1 import numpy as np, pandas as pd, time
2 from sklearn.datasets import load_breast_cancer
3 from sklearn.model_selection import cross_val_score, StratifiedKFold
4 from sklearn.pipeline import make_pipeline
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.linear_model import LogisticRegression
7 from sklearn.neighbors import KNeighborsClassifier
8 from sklearn.naive_bayes import GaussianNB
9 from sklearn.svm import SVC
10 from sklearn.tree import DecisionTreeClassifier
11 from sklearn.ensemble import (RandomForestClassifier, ExtraTreesClassifier,
12                               AdaBoostClassifier, GradientBoostingClassifier,
13                               HistGradientBoostingClassifier)
14
15 bc = load_breast_cancer(as_frame=True); X, y = bc.data, bc.target
16 cv = StratifiedKFold(5, shuffle=True, random_state=42)
17

```

```

18 modeller = {
19     'LogisticRegression': make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000,
20         random_state=42)),
21     'kNN (k=5)': make_pipeline(StandardScaler(), KNeighborsClassifier(5)),
22     'GaussianNB': make_pipeline(StandardScaler(), GaussianNB()),
23     'SVM (rbf)': make_pipeline(StandardScaler(), SVC(random_state=42)),
24     'DecisionTree': DecisionTreeClassifier(random_state=42),
25     'RandomForest': RandomForestClassifier(n_estimators=300, random_state=42, n_jobs
26         =-1),
27     'ExtraTrees': ExtraTreesClassifier(n_estimators=300, random_state=42, n_jobs=-1),
28     'AdaBoost': AdaBoostClassifier(random_state=42),
29     'GradientBoosting': GradientBoostingClassifier(random_state=42),
30     'HistGradientBoosting': HistGradientBoostingClassifier(random_state=42),
31 }
32 satirlar = []
33 for ad, m in modeller.items():
34     t0 = time.perf_counter()
35     acc = cross_val_score(m, X, y, cv=cv, scoring='accuracy', n_jobs=-1)
36     auc = cross_val_score(m, X, y, cv=cv, scoring='roc_auc', n_jobs=-1)
37     satirlar.append({'model': ad, 'accuracy': acc.mean(), 'std': acc.std(),
38         'roc_auc': auc.mean(), 'sure_s': time.perf_counter()-t0})
39 print(pd.DataFrame(satirlar).sort_values('roc_auc', ascending=False).round(4))

```

model	accuracy	std	roc_auc	sure_s
LogisticRegression	0.9737	0.0166	0.9953	1.4124
SVM (rbf)	0.9772	0.0163	0.9945	0.0460
ExtraTrees	0.9649	0.0157	0.9941	0.4697
AdaBoost	0.9525	0.0219	0.9933	0.2171
GradientBoosting	0.9491	0.0237	0.9927	0.4894
HistGradientBoosting	0.9666	0.0129	0.9925	0.2062
RandomForest	0.9526	0.0131	0.9895	0.6589
GaussianNB	0.9297	0.0199	0.9860	0.0452
kNN (k=5)	0.9631	0.0179	0.9849	1.0836
DecisionTree	0.9104	0.0279	0.9000	0.0651

Bilgi

Sürpriz: en basit model olan **lojistik regresyon** en yüksek ROC-AUC'yi veriyor. Bu veri seti doğrusal olarak ayrılabilir olduğu için karmaşık modeller ek değer katmıyor. **Ders:** her zaman basit bir temel çizgi (baseline) kur; gradient boosting'e atlamadan önce onu yenmen gerektiğini bil.

43. k-En Yakın Komşu

Formül

Tahmin, en yakın k komşunun oyu (sınıflandırma) ya da ortalamasıdır (regresyon). Eğitim yoktur — *tembel öğrenme*. Uzaklık genelde Öklid'dir: $d(x, x') = \sqrt{\sum_j (x_j - x'_j)^2}$.

Nedir? Niçin? Nerede?

Fikir tek cümlede: “Bana benzeyenlere bak, onların cevabını ver.” Yeni bir gözlem geldiğinde, eğitim verisindeki en yakın k komşuyu bulur ve onların çoğunluk sınıfını (ya da ortalamasını) döndürür. Hiçbir denklem uydurmaz, hiçbir parametre öğrenmez — bu yüzden *tembel öğrenme* denir: bütün işi tahmin anına erteler.

Niçin öğrenmesi gerekmiyor? Çünkü modeli, verinin kendisidir. Eğitim aşaması sadece veriyi hafızaya almaktır ($O(1)$); asıl maliyet tahmin anında ortaya çıkar, çünkü her yeni nokta için tüm eğitim kümesine uzaklık hesaplanır. Bu, diğer algoritmaların tam tersidir — ve k -NN’in milyonlarca satırlık veride niçin kullanılmadığını açıklar.

k neyi kontrol eder? Doğrudan yanlışlık–varyans düğmesidir. $k = 1$: karar sınırı her gözleme uyar, gürültüyü ezberler (yüksek varyans). k çok büyük: her şey ortalanır, ince yapı kaybolur (yüksek yanlışlık). Tabloda $k = 3$ optimum çıkıyor. Pratik başlangıç: $k \approx \sqrt{n}$, ve sınıflandırmada beraberliği önlemek için tek sayı seç.

Ölçekleme niçin zorunlu? Uzaklık hesabı birimlere duyarlıdır. “Maaş” (30.000–90.000) ve “yaş” (20–65) özniteliklerini birlikte kullanırsan, maaştaki 1000 TL’lik fark yaştaki 45 yıllık farktan büyük görünür — yaş fiilen yok sayılır. StandardScaler olmadan k -NN neredeyse her zaman yanlışdır.

Boyut laneti niçin k -NN’i özellikle vurur? Yüksek boyutta tüm noktalar birbirine yaklaşık eşit uzaklıkta olur; “en yakın komşu” kavramı anlamını yitirir. 100 boyutta k -NN genellikle rastgeleden az iyidir. Çözüm: önce PCA ile boyut indirgemek veya öznitelik seçimi yapmak.

Nerede kullanılır?

- **Öneri sistemleri:** “Senin gibi kullanıcılar şunu beğendi” — işbirlikçi filtrelemenin en basit hâli birebir k -NN’dir.
- **Görüntü/yüz eşleştirme:** Gömme vektörleri (embedding) çıkarıldıktan sonra en yakın komşu araması. Modern vektör veritabanları (FAISS, pgvector) endüstriyel ölçekte k -NN motorlarıdır.
- **Eksik veri doldurma:** KNNImputer, eksik değeri benzer satırların ortalamasıyla doldurur.
- **Hızlı temel model (baseline):** Yeni bir problemde “bu veri öğrenilebilir mi?” sorusunun en hızlı testi.

```
1 for k in [1, 3, 5, 11, 21, 51]:
2     s = cross_val_score(make_pipeline(StandardScaler(), KNeighborsClassifier(k)),
3                           X, y, cv=cv)
4     print(f"k = {k:>3}: {s.mean():.4f}")
```

```
k = 1: 0.9508
k = 3: 0.9666
k = 5: 0.9631
k = 11: 0.9631
k = 21: 0.9526
k = 51: 0.9490
```

Dikkat

$k = 1$ eğitim verisini ezberler (aşırı öğrenme), k çok büyükse her şeyi ortalar (eksik öğrenme). k tam olarak yanlışlık–varyans düğmesidir. Ayrıca k -NN ölçeklemesiz kullanılamaz: birimi büyük olan öznitelik uzaklığı domine eder. **Boyut laneti:** özellik sayısı arttıkça tüm noktalar birbirine eşit uzaklıkta görünmeye başlar.

44. Karar Ağaçları

Bölme Ölçütleri

Bir düğümü, çocuk düğümlerin *saflığını* en çok artıran özellik-eşik çiftiyle böl:

$$\text{Gini} = 1 - \sum_c p_c^2, \quad \text{Entropi} = - \sum_c p_c \log_2 p_c, \quad \text{Regresyonda: MSE azalması.}$$

Nedir? Niçin? Nerede?

Ağaç ne yapıyor? Veriyi ardışık evet/hayır sorularıyla giderek daha *saf* parçalara böler. “Saf” demek, bir yaprakta çoğunlukla tek bir sınıfın kalması demektir. Aşağıdaki çıktıda iki soru — “en kötü yarıçap 16,8’den küçük mü?” ve “içbükey nokta sayısı 0,14’ten küçük mü?” — %90 doğruluk veriyor.

Gini ve entropi ne ölçüyor? İkisi de *safsızlığı* ölçer. Bir düğümde tek sınıf varsa ikisi de 0; sınıflar yarı yarıyaysa ikisi de en büyük değerini alır (Gini 0,5, entropi 1). Ağaç her adımda tüm olası bölmeleri dener ve safsızlığı en çok düşüreni seçer — buna *bilgi kazancı* denir.

Gini mi entropi mi? Pratikte neredeyse hiç fark etmezler; aynı ağaçları üretirler. Gini biraz daha hızlıdır (logaritma yok), bu yüzden varsayılan odur. Entropi bilgi kuramından gelir ve “bu bölme kaç bit belirsizlik azalttı?” diye okunabilir.

Regresyonda MSE azalması — tanıdık geldi mi? Bu, Kısım 3’teki *varyans ayrıştırmasının* ta kendisidir: ağaç, gruplar arası varyansı en çok büyüten bölmeyi arar. ANOVA, R^2 ve karar ağacı bölmesi aynı fikrin üç yüzüdür.

Ağaçların ayırt edici üstünlükleri:

- **Yorumlanabilirlik:** Sonucu bir akış şeması olarak çizip alan uzmanıyla tartışabilirsin. Hiçbir başka model bunu bu kadar doğrudan yapamaz.
- **Ölçekleme gerektirmez:** Eşik karşılaştırması yaptığı için birimler önemsizdir.
- **Doğrusal olmayan ilişkileri ve etkileşimleri kendiliğinden yakalar:** “Eğer erkekse *ve* 3. sınıftaysa” kuralı, regresyonda elle eklemen gereken bir etkileşim terimidir.
- **Karma veri tipleriyle çalışır:** Kategorik ve sayısal öznitelikler bir arada.

En büyük zayıflığı: kararsızlık. Veriden birkaç örneği değiştiren ağaç bambaşka çıkabilir — yüksek varyanslı bir modeldir. Kısım 9’daki topluluk yöntemleri (rastgele orman, gradyan artırma) tam olarak bu zayıflığı kapatmak için icat edildi: yüzlerce oynak ağacın ortalaması, tek bir ağaçtan çok daha kararlıdır.

Nerede kullanılır? Kredi onay kuralları; tıbbi triyaj protokolleri (acil serviste hangi hasta önce görülecek); müşteri segmentasyonu kuralları; sigortada risk sınıflandırması; ve en önemlisi — gradyan artırma kütüphanelerinin (XGBoost, LightGBM) yapı taşı olarak, tablo verisindeki en güçlü modellerin temeli.

```
1 from sklearn.tree import DecisionTreeClassifier, export_text
2 for d in [1, 2, 3, 5, 10, None]:
3     s = cross_val_score(DecisionTreeClassifier(max_depth=d, random_state=42), X, y, cv=cv)
4     print(f"max_depth = {str(d):>4}: {s.mean():.4f}")
5
6 dt = DecisionTreeClassifier(max_depth=2, random_state=42).fit(X, y)
7 print(export_text(dt, feature_names=list(X.columns), max_depth=2))
```

```
max_depth = 1: 0.8876
```

```

max_depth = 2: 0.9052
max_depth = 3: 0.9245
max_depth = 5: 0.9280
max_depth = 10: 0.9104
max_depth = None: 0.9104

|--- worst radius <= 16.80
| |--- worst concave points <= 0.14
| | |--- class: 1
| |--- worst concave points > 0.14
| | |--- class: 0
|--- worst radius > 16.80
| |--- mean texture <= 16.11
| | |--- class: 1
| |--- mean texture > 16.11
| | |--- class: 0

```

Bilgi

Sadece iki soruyla %90 doğruluk: “en kötü yarıçap 16,8’den küçük mü?” ve “en kötü içbükey nokta sayısı 0,14’ten küçük mü?” Karar ağaçlarının büyük değeri budur — bir doktora gösterip tartışabileceğin kurallar üretir. Derinlik arttıkça doğruluk düşüyor (aşırı öğrenme), bu yüzden `max_depth`, `min_samples_leaf`, `ccp_alpha` ile budama şarttır.

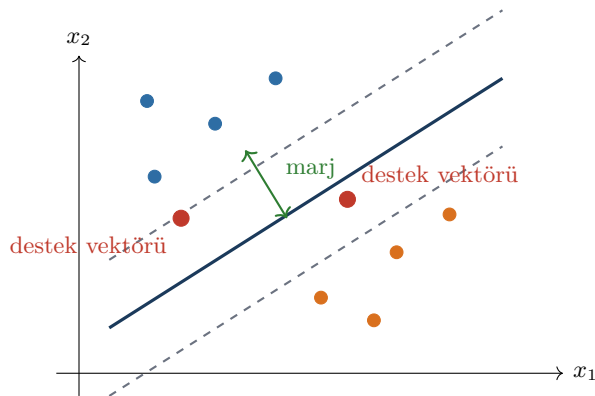
45. Destek Vektör Makineleri (SVM)

Maksimum Marj

SVM, iki sınıfı ayıran hiperdüzlemler arasından *marj* (sınıflara en yakın noktaların uzaklığını) en büyük olanı seçer:

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i \quad \text{öyle ki} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

C , marj genişliği ile hata toleransı arasındaki dengeyi kurar: büyük $C \rightarrow$ dar marj, az hata (aşırı öğrenme riski).



Kernel Hilesi

Doğrusal olarak ayrılamayan veri, yüksek boyutlu bir uzaya taşınırsa ayrılabilir hâle gelir. SVM bu dönüşümü *açıkça hesaplamadan*, yalnızca iç çarpımları değiştirerek yapar:

$$K(x, x') = \phi(x)^\top \phi(x') \quad \begin{cases} \text{doğrusal} : & x^\top x' \\ \text{polinom} : & (\gamma x^\top x' + r)^d \\ \text{RBF} : & \exp(-\gamma \|x - x'\|^2) \end{cases}$$

Nedir? Niçin? Nerede?

Problem neydi? Bazı veriler düz bir çizgiyle ayrılamaz — klasik örnek, bir sınıfın diğerinin *etrafını çevrelediği* halka biçimindeki veridir. Çözüm fikri şaşırtıcı derecede basit: veriyi daha yüksek boyutlu bir uzaya taşı, orada düzlemle ayrılabilir hâle gelsin. İki boyutta iç içe iki halka, üçüncü bir eksen (mesela $z = x^2 + y^2$) eklendiğinde bir düzlemle rahatça ayrılır.

“Hile” nerede? Bu dönüşümü açıkça hesaplamak pahalıdır — bazen sonsuz boyutludur (RBF kernel tam olarak öyledir). Kernel hilesi şunu fark eder: **SVM’in matematiğinde noktaların kendisi hiç geçmez, yalnızca iç çarpımları geçer.** O hâlde noktaları taşımaya gerek yok; *iç çarpımı nasıl hesapladığını* değiştirmek yeter. $K(x, x')$ fonksiyonu, “o yüksek boyutlu uzayda bu iki nokta ne kadar benzer olurdu?” sorusunu doğrudan cevaplar — oraya hiç gitmeden.

Hangi kernel ne zaman?

- **Doğrusal:** Öznitelik sayısı gözlem sayısından fazlaysa (metin sınıflandırma, genomik). Zaten yeterince boyut var, eklemeye gerek yok. Hızlıdır ve katsayıları yorumlanabilir.
- **RBF (Gauss):** Varsayılan tercih. “Yakın noktalar benzerdir” fikrini kodlar; γ “ne kadar yakın?” sorusunu ayarlar. Küçük $\gamma \rightarrow$ geniş etki alanı, yumuşak sınır; büyük $\gamma \rightarrow$ her nokta kendi adacığını yaratır, aşırı öğrenme.
- **Polinom:** Öznitelikler arası çarpımsal etkileşimlerin önemli olduğu problemlerde.

C parametresi ne yapar? Marj genişliği ile hata toleransı arasındaki dengeyi kurar. Küçük C : “geniş marj istiyorum, birkaç yanlış sınıflandırmaya razıyım” — basit, düzenlenilmiş model. Büyük C : “her eğitim örneğini doğru sınıflandır” — karmaşık sınır, aşırı öğrenme riski. Yani C de bir yanlışlık–varyans düşmesidir ve γ ile *birlikte* aranmalıdır.

Nerede kullanılır? Metin ve doküman sınıflandırma; biyoinformatikte gen ifadesi verisi (az örnek, çok öznitelik — SVM’in en güçlü olduğu durum); el yazısı tanıma (derin öğrenme öncesi standarttı); yüz tanıma; küçük ve orta ölçekli, yüksek boyutlu her problem. Büyük veride (>100.000 satır) eğitim maliyeti nedeniyle yerini gradyan artırmaya bırakır.

```
1 from sklearn.svm import SVC
2 for k in ['linear', 'poly', 'rbf', 'sigmoid']:
3     s = cross_val_score(make_pipeline(StandardScaler(), SVC(kernel=k, random_state=42)),
4                         X, y, cv=cv, scoring='roc_auc', n_jobs=-1)
5     print(f"{k:>8}: roc_auc = {s.mean():.4f}")
6
7 for C in [0.01, 0.1, 1, 10, 100, 1000]:
8     s = cross_val_score(make_pipeline(StandardScaler(), SVC(C=C, random_state=42)),
9                         X, y, cv=cv, scoring='accuracy', n_jobs=-1)
10    print(f"C = {C:>6}: dogruluk = {s.mean():.4f}")
```

```
linear: roc_auc = 0.9942
poly: roc_auc = 0.9946
rbf: roc_auc = 0.9945
```

```
sigmoid: roc_auc = 0.9903
```

```
C = 0.01: dogruluk = 0.6274
C = 0.1: dogruluk = 0.9473
C = 1: dogruluk = 0.9772
C = 10: dogruluk = 0.9789
C = 100: dogruluk = 0.9508
C = 1000: dogruluk = 0.9508
```

Bilgi

$C = 0,01$ 'de model neredeyse hiçbir şey öğrenmiyor (%62,7 — taban oranın biraz üstü); $C = 10$ 'da optimum; $C \geq 100$ 'de aşırı öğrenip düşüyor. C ve γ birlikte aranmalıdır (GridSearchCV) çünkü etkileri birbirine bağlıdır.

```
1 m = make_pipeline(StandardScaler(), SVC(kernel='linear', random_state=42)).fit(X, y)
2 print("destek vektörü sayısı:", m.named_steps['svc'].n_support_, "/", len(y))
3 # [21 19] / 569 -> 569 ornekten sadece 40'i sinirin belirlenmesinde rol oynuyor
```

İpucu

SVM'in zarafeti budur: karar sınırı yalnızca *destek vektörleri* tarafından belirlenir; diğer 529 örneği silsen model değişmez. Bu, SVM'i aykırı değerlere görece dayanıklı yapar — ama örneklem büyüdükçe eğitim maliyeti $O(n^2)$ – $O(n^3)$ 'e çıktığı için $n > 100$ bin'de LinearSVC veya SGDClassifier tercih edilir.

46. Naive Bayes ve Metin Sınıflandırma

Naive Bayes

Bayes teoremi + “öznitelikler sınıf verildiğinde bağımsızdır” varsayımı:

$$\mathbb{P}(y \mid x_1, \dots, x_k) \propto \mathbb{P}(y) \prod_{j=1}^k \mathbb{P}(x_j \mid y).$$

Varsayım neredeyse hiç doğru değildir — ama sınıflandırma için *sıralama* doğru olduğu sürece bu önemli değildir.

Nedir? Niçin? Nerede?

Doğrudan Kısım 3'teki Bayes teoreminin uygulamasıdır. Sorduğu soru: “Bu e-postada şu kelimeler geçiyor; spam olma olasılığı ne?” Bayes ile ters çevirirsin: $\mathbb{P}(\text{spam} \mid \text{kelimeler}) \propto \mathbb{P}(\text{kelimeler} \mid \text{spam}) \cdot \mathbb{P}(\text{spam})$.

“Naive” (saf) tam olarak neyi varsayıyor? Kelimelerin birbirinden bağımsız olduğunu. Yani “New” ile “York” kelimelerinin bir arada görülmesini, iki ayrı tesadüf sayar. Bu *açıkça yanlıştır* — dil böyle çalışmaz. Ama varsayım sayesinde $\mathbb{P}(\text{tüm kelimeler} \mid y)$ gibi imkânsız bir hesap, tek tek kelime olasılıklarının çarpımına iner. 10.000 kelimelik bir sözlükte bu, hesaplanamaz bir problemi saniyeler sürecektir bir işe çevirir.

Yanlışı varsayımla niçin iyi çalışıyor? Çünkü sınıflandırmada *doğru olasılık* değil, *doğru sıralama* gerekir. Bağımsızlık ihlali olasılıkları bozar (Naive Bayes genellikle aşırı özgüvenlidir — 0,99 veya 0,01 der), ama hangi sınıfın daha yüksek olduğunu genellikle doğru belirler. *Sonuç:* sınıflandırma için mükemmel, olasılık tahmini için kötü. Olasılığa ihtiyacın varsa kalibre etmen gerekir (Kısım 11).

Üstünlükleri:

- **Çok hızlı:** Eğitim tek geçişte biter — sadece sayım yapar. Milyonlarca dokümanı dakikalar içinde işler.
- **Az veriyle çalışır:** Her parametreyi ayrı ayrı tahmin ettiği için, karmaşık modellerin veri açlığından muaftır.
- **Yüksek boyuta doğal uyum:** 50.000 kelimelik bir sözlük onun için sorun değil — aksine, ideal ortamdır.
- **Artımlı öğrenme:** Yeni veri geldikçe sayaçları güncellemek yeter; baştan eğitmek gerekmez.

Nerede kullanılır? Spam filtreleri (tarihsel olarak ilk büyük başarısı); duygu analizi; haber/doküman kategorilendirme; tıbbi ön tanı sistemleri; ve her metin probleminde *referans model* olarak — daha karmaşık bir modelin gerçekten daha iyi olduğunu Naive Bayes'i yenerek kanıtlaman gerekir.

İki pratik ayrıntı: (1) *Laplace düzeltmesi:* eğitimde hiç görülmemiş bir kelime olasılığı sıfır yapar ve tüm çarpımı sıfırlar; her sayaca 1 eklenerek bu önlenir (alpha parametresi). (2) *Logaritma:* binlerce küçük olasılığın çarpımı sayısal olarak sıfıra yuvarlanır, bu yüzden pratikte log-olasılıklar toplanır — Kısım 5'teki MLE'de gördüğümüz aynı numara.

Varyant	Ne zaman?
GaussianNB	Sürekli öznitelikler (her sınıfta normal varsayılır).
MultinomialNB	Sayım verisi — metin sınıflandırmanın klasığı.
ComplementNB	Dengesiz metin verisinde MultinomialNB'den iyi.
BernoulliNB	İkili öznitelikler (kelime var/yok).

20 Newsgroups

1990'ların Usenet haber gruplarından toplanmış gerçek forum mesajları. Dört kategori seçiyoruz: otomobil, tıp, uzay ve Ortadoğu siyaseti. Başlıklar ve imzalar temizleniyor ki model gerçekten *metinden* öğrensinsin.

```

1 from sklearn.datasets import fetch_20newsgroups
2 from sklearn.feature_extraction.text import TfidfVectorizer
3 from sklearn.naive_bayes import MultinomialNB

```

```

4 from sklearn.svm import LinearSVC
5 from sklearn.metrics import accuracy_score
6
7 kat = ['sci.med', 'sci.space', 'rec.autos', 'talk.politics.mideast']
8 tr = fetch_20newsgroups(subset='train', categories=kat,
9                          remove=('headers', 'footers', 'quotes'), random_state=42)
10 te = fetch_20newsgroups(subset='test', categories=kat,
11                          remove=('headers', 'footers', 'quotes'), random_state=42)
12 print("egitim:", len(tr.data), " test:", len(te.data), " sinif:", tr.target_names)
13
14 for ad, clf in [('MultinomialNB', MultinomialNB()),
15                 ('LinearSVC', LinearSVC()),
16                 ('LogisticRegression', LogisticRegression(max_iter=2000))]:
17     m = make_pipeline(TfidfVectorizer(stop_words='english', min_df=2), clf)
18     m.fit(tr.data, tr.target)
19     print(f"{ad:>20}: test dogrulugu = {accuracy_score(te.target, m.predict(te.data)):.4f}")

```

```

egitim: 2345 test: 1562 sinif: ['rec.autos', 'sci.med', 'sci.space', 'talk.politics.mideast']
MultinomialNB: test dogrulugu = 0.8892
LinearSVC: test dogrulugu = 0.8630
LogisticRegression: test dogrulugu = 0.8796

```

```

1 # Modelin ogrendigi kelimeler: her sinif icin en yuksek olasilikli terimler
2 import numpy as np
3 m = make_pipeline(TfidfVectorizer(stop_words='english', min_df=2),
4                  MultinomialNB()).fit(tr.data, tr.target)
5 v, nb = m.named_steps['tfidfvectorizer'], m.named_steps['multinomialnb']
6 isim = np.array(v.get_feature_names_out())
7 for i, s in enumerate(tr.target_names):
8     print(f"{s:>22}: {' ', ' '.join(isim[np.argsort(nb.feature_log_prob_[i])[-8:]][::-1])}")

```

```

rec.autos: car, cars, like, just, engine, good, new, dealer
sci.med: msg, edu, gordon, geb, pitt, banks, skepticism, n3jxp
sci.space: space, nasa, like, launch, orbit, moon, just, earth
talk.politics.mideast: israel, israeli, jews, people, armenian, armenians, turkish, arab

```

Bilgi

Otomobil, uzay ve siyaset kategorilerinde öğrenilen kelimeler tam beklenen gibi. Ama sci.med'de gordon, geb, pitt, n3jxp gibi anlamsız terimler var: bunlar tek bir üretken kullanıcının imzasından geliyor. Model “tıp”ı değil “o kullanıcıyı” öğrenmiş — bu bir **sahte öznitelik** (spurious feature) örneğidir ve test kümesinde de aynı kullanıcı olduğu için skor yüksek çıkar. Gerçek kullanımda çöker. *Modelin ne öğrendiğine bakmadan skoruna güvenme.*

47. Öznitelik Seçimi

Yaklaşım	Araç	Nasıl çalışır?
Filtre	SelectKBest	İstatistiksel skor (F , χ^2 , karşılıklı bilgi); modelden bağımsız, çok hızlı.
Sarmalayıcı	RFECV	Modeli tekrar tekrar eğitip en zayıf öz niteliği eler; pahalı ama etkili.
Gömülü	Lasso, ağaç önemleri	Model kendi seçer; düzenlileştirme katsayısını sıfırlar.

```

1 from sklearn.feature_selection import RFECV, SelectKBest, f_classif
2
3 sel = RFECV(LogisticRegression(max_iter=5000), step=1, cv=cv,
4             scoring='roc_auc', n_jobs=-1).fit(StandardScaler().fit_transform(X), y)
5 print(f"RFECV: secilen oznitelik sayisi = {sel.n_features_} / {X.shape[1]}")
6
7 for k in [5, 10, 20, 30]:
8     p = make_pipeline(StandardScaler(), SelectKBest(f_classif, k=k),
9                     LogisticRegression(max_iter=5000))
10    print(f"SelectKBest k={k:>2}: roc_auc = "
11          f"{cross_val_score(p, X, y, cv=cv, scoring='roc_auc', n_jobs=-1).mean():.4f}")

```

```

RFECV: secilen oznitelik sayisi = 23 / 30
SelectKBest k= 5: roc_auc = 0.9890
SelectKBest k=10: roc_auc = 0.9893
SelectKBest k=20: roc_auc = 0.9963
SelectKBest k=30: roc_auc = 0.9953

```

İpucu

20 öz nitelik, 30'un tamamından daha iyi (0,9963 vs 0,9953): gereksiz öz nitelikler gürültü katar. Ama fark küçüktür — öz nitelik seçiminin asıl faydası genelde skor değil, **yorumlanabilirlik, hız ve veri toplama maliyetidir**. Ve seçimi *daima* pipeline içinde yap (Kısım 8'deki sızıntı demosunu hatırla).

48. Topluluk Yöntemleri (Ensembles)

Niçin Birçok Zayıf Model, Bir Güçlü Modelden İyi?

Temel fikir şaşırtıcı derecede eskidir ve istatistikten gelir: **bağımsız hataların ortalaması, tek bir hatadan küçüktür**. B tane bağımsız tahminin ortalamasını alırsan, varyans B kat düşer ($\text{Var}(\bar{X}) = \sigma^2/B$ — Kısım 4'teki standart hata formülünün ta kendisi). Topluluk yöntemleri bu tek matematiksel gerçeği farklı biçimlerde sömürür.

İki temel strateji — ve hangi hatayı hedefledikleri:

- **Bagging (paralel) — varyansı düşürür.** Her modeli verinin farklı bir bootstrap örneğinde *bağımsız* olarak eğit, sonra ortalala. Tek bir karar ağacı yüksek varyanslıdır (veriyi biraz değiştirsen bambaşka çıkar); 500 ağacın ortalaması kararlıdır. *Rastgele orman* bir adım daha atar: her bölmede özniteliklerin de rastgele bir alt kümesine bakar, böylece ağaçlar birbirine daha az benzer — ortalamanın işe yaraması için hataların *ilişkisiz* olması gerektiğini hatırla.
- **Boosting (sıralı) — yanlışlığı düşürür.** Modelleri sırayla eğit; her yeni model, öncekilerin *yanlış yaptığı* örneklere odaklanır. Tek tek zayıf olan modeller (genelde çok sığ ağaçlar) birikerek güçlü bir model oluşturur. Gradyan artırma, bunu “kalıntıların gradyanına uy” biçiminde formüle eder.

Pratik sonucu: Rastgele orman ayarsız da iyi çalışır, aşırı öğrenmeye dirençlidir, paralelleşir — güvenli varsayılan tercihtir. Gradyan artırma daha yüksek doğruluk verir ama ayar ister (öğrenme hızı, ağaç sayısı, derinlik) ve fazla iterasyonda aşırı öğrenir. **Tablo verisinde bugün en iyi sonucu neredeyse her zaman gradyan artırma verir** — Kaggle yarışmalarının çoğunu XG-Boost/LightGBM kazanır.

Bedeli: Yorumlanabilirlik. 500 ağacı bir insana açıklayamazsın. Bu yüzden Kısım 11'deki yorumlama araçları (permütasyon önemi, SHAP) topluluk modelleriyle birlikte kullanılır.

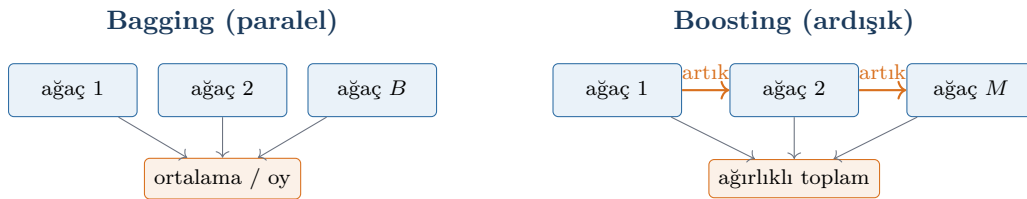
Nerede kullanılır? Kredi risk skorlama, talep tahmini, tıklama oranı tahmini, sigorta fiyatlaması, arıza öngörüsü — kısacası satır-sütun biçimli her ciddi tahmin probleminde.

Topluluk Fikrinin Kökeni: Öküzün Ağırlığı

1906'da Francis Galton bir panayırda ilginç bir gözlem yaptı: bir öküzün ağırlığını tahmin etme yarışmasına 787 kişi katıldı. Kimse doğru bilemedi; tahminler 500 kg ile 1200 kg arasında dağılmıştı.

Ama Galton tüm tahminlerin *medyanını* aldığında sonuç 1207 pound çıktı — öküzün gerçek ağırlığı 1198 pound'du. %0,8 hata. Hiçbir uzman, kalabalığın ortalaması kadar isabetli değildi.

Sebepten olarak topluluk yöntemlerinin çalışma sebebidir: her tahminde bir miktar bilgi ve bir miktar bağımsız hata vardır. Ortalama alınca hatalar birbirini götürür, bilgi birikir. **Ama kritik şart: hataların bağımsız olması.** Herkes aynı yanlış ipucuna bakıyorsa (veya tüm ağaçlar aynı özniteliği kullanıyorsa) ortalama hiçbir şey düzeltmez — rastgele ormanın öznitelikleri de rastgeleleştirmesinin sebebi budur.



Yöntem	Fikir	Azalttığı
Bagging	Bootstrap örneklerinde bağımsız modeller	Varyans
Random Forest	Bagging + her bölmede rastgele öznelik alt kümesi	Varyans (ağaçları decorrelate eder)
Boosting	Her model öncekinin hatasına odaklanır	Yanlılık (ve varyans)
Stacking	Model tahminlerini bir üst model birleştirir	Her ikisi

```

1 rf = RandomForestClassifier(n_estimators=300, random_state=42, n_jobs=-1).fit(X, y)
2 imp = pd.Series(rf.feature_importances_, index=X.columns).sort_values(ascending=False)
3 print(imp.head(8).round(4))

```

```

worst perimeter      0.1476
worst area           0.1373
worst concave points 0.1209
mean concave points  0.0962
worst radius         0.0716
mean perimeter       0.0508
mean concavity       0.0486
mean radius          0.0478

```

Dikkat

`feature_importances_` (Gini önemi) **yanlıdır**: yüksek kardinaliteli ve sürekli değişkenleri şişirir, korelasyonlu özneliklerin önemini keyfi biçimde birine yükler. Güvenilir alternatif *permütasyon önemi* veya SHAP değerleridir (Kısım 11).

48.1 Voting ve Stacking

```

1 from sklearn.ensemble import VotingClassifier, StackingClassifier
2
3 temel = [('lr', make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))),
4          ('rf', RandomForestClassifier(n_estimators=300, random_state=42)),
5          ('gb', GradientBoostingClassifier(random_state=42))]
6
7 vote = VotingClassifier(temel, voting='soft')
8 stack = StackingClassifier(temel, final_estimator=LogisticRegression(max_iter=1000), cv=5)
9
10 for ad, m in [('VotingSoft', vote), ('Stacking', stack)]:
11     print(f"{ad}: roc_auc = {cross_val_score(m, X, y, cv=cv, scoring='roc_auc', n_jobs=-1).mean():.4f}")

```

```

VotingSoft: roc_auc = 0.9948
Stacking: roc_auc = 0.9952

```

Bilgi

Topluluklar burada tek başına lojistik regresyonu (0,9953) geçemedi. Topluluklar, temel modeller *farklı hatalar* yaptığında işe yarar; hepsi aynı yönde yanılıyorsa birleştirmek fayda etmez. Yine de gerçek dünyada (özellikle tablo verisinde) gradient boosting ve stacking genelde kazanır — bu veri seti fazla “kolay”.

49. Hiperparametre Ayarı

Parametre ile Hiperparametre Farkı

Parametreler modelin veriden *öğrendiği* sayılardır — regresyon katsayıları, sinir ağı ağırlıkları. Bunları sen belirlemezsin, `fit()` belirler. **Hiperparametreler** ise eğitimden *önce* senin belirlemen gereken ayarlardır — ağaç derinliği, k komşu sayısı, SVM’in C ’si, öğrenme hızı, düzenleme gücü.

Niçin bu kadar önemli? Çünkü hemen hepsi doğrudan *yanlılık–varyans* düğmesidir. k -NN’de k , ağaçta `max_depth`, Ridge’de α , SVM’de C — hepsi aynı şeyi kontrol eder: modelin ne kadar esnek olacağını. Yanlış ayarlanmış bir hiperparametre, doğru seçilmiş bir algoritmayı işe yaramaz hâle getirir; SVM örneğinde $C = 0,01$ ile %62, $C = 10$ ile %98 doğruluk almıştık — aynı algoritma. **Üç arama stratejisi:**

- **Izgara araması (GridSearchCV):** Tüm kombinasyonları dener. Kesin ama üstel pahalıdır — 3 parametrenin her biri 5 değer $\rightarrow 125 \text{ model} \times 5 \text{ katman} = 625$ eğitim. Az sayıda parametre varsa uygundur.
- **Rastgele arama (RandomizedSearchCV):** Rastgele kombinasyonlar dener. Şaşırtıcı biçimde ızgaradan *daha verimlidir*, çünkü genellikle parametrelerin yalnızca birkaçı gerçekten önemlidir ve rastgele arama o birkaçı daha çok farklı değerde dener. Varsayılan tercihin bu olsun.
- **Bayesçi optimizasyon (Optuna, skopt):** Önceki denemelerden öğrenerek nereye bakacağına karar verir. Pahalı modellerde (derin öğrenme, büyük veri) en verimli yöntemdir — ve mantığı doğrudan Kısım 6’daki Bayesçi güncellemedir.

En kritik kural — test kümesine dokunma. Hiperparametre araması, *eğitim* verisi üzerindeki çapraz doğrulama ile yapılır. Test kümesine bakarak ayar yaparsan, test skorun artık dürüst bir tahmin değildir — farkında olmadan test kümesine aşırı uydurmuş olursun. Hem model seçip hem performans ölçmen gerekiyorsa *iç içe (nested) çapraz doğrulama* kullanılır; aşağıdaki kod tam olarak bunu gösteriyor.

Pratik öncelik: Sonsuz parametre aramak yerine, önce hangilerinin önemli olduğunu bil. Gradyan artırmada `learning_rate` ve `n_estimators`; rastgele ormanda `max_features` ve `min_samples_leaf`; SVM’de C ve γ . Geri kalanı genellikle varsayılanda bırakılabilir. Ve unutma: **daha iyi öznitelik, daha iyi hiperparametreden neredeyse her zaman daha değerlidir.**

```
1 from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
2
3 grid = GridSearchCV(
4     make_pipeline(StandardScaler(), SVC(random_state=42)),
5     {'svc__C': [0.1, 1, 10, 100], 'svc__gamma': ['scale', 0.01, 0.1, 1]},
6     cv=cv, scoring='roc_auc', n_jobs=-1)
7 grid.fit(X, y)
8 print("en iyi parametreler:", grid.best_params_)
9 print(f"en iyi CV roc_auc = {grid.best_score_:.4f}")
```

```
en iyi parametreler: {'svc__C': 10, 'svc__gamma': 0.01}
en iyi CV roc_auc = 0.9960
```

Yöntem	Ne zaman?
GridSearchCV	Az sayıda parametre, küçük ızgara.
RandomizedSearchCV	Çok parametre; aynı bütçeyle genelde daha iyi sonuç.
HalvingGridSearchCV	Ardışık eleme; büyük ızgaralarda hızlı.
optuna / hyperopt	Bayeşçi optimizasyon; pahalı modellerde en verimli.

Dikkat

Yuvalanmış (nested) CV: hiperparametreyi CV ile seçip aynı CV skorunu “model performansı” diye raporlamak iyimser yanlışlık üretir. Dürüst kestirim için dış döngü (performans) ile iç döngü (ayar) ayrılmalıdır: `cross_val_score(GridSearchCV(...), X, y, cv=dis_cv)`.

```
1 # Yuvalanmis CV: durust performans kestirimi
2 from sklearn.model_selection import cross_val_score as cvs
3 ic_cv = StratifiedKFold(4, shuffle=True, random_state=1)
4 dis_cv = StratifiedKFold(5, shuffle=True, random_state=2)
5 arayici = GridSearchCV(make_pipeline(StandardScaler(), SVC(random_state=42)),
6                        {'svc__C': [0.1, 1, 10, 100]}, cv=ic_cv, scoring='roc_auc')
7 print(f"yuvalanmis CV roc_auc = {cvs(arayici, X, y, cv=dis_cv, scoring='roc_auc').mean():.4f}")
```

49.1 Boosting Kütüphaneleri

Kütüphane	Özellik
HistGradientBoosting	scikit-learn içinde; hızlı, eksik değeri kendi işler.
XGBoost	Olgun, düzenlileştirme güçlü, GPU desteği.
LightGBM	Çok hızlı, büyük veride tercih; yaprak-bazlı büyüme.
CatBoost	Kategorik değişkenleri doğrudan işler; ayarsız iyi çalışır.

```
1 # Tipik XGBoost/LightGBM baslangic parametreleri
2 params = dict(
3     n_estimators=1000, learning_rate=0.05, max_depth=6,
4     subsample=0.8, colsample_bytree=0.8,
5     reg_lambda=1.0, early_stopping_rounds=50, random_state=42)
6 # Kural: learning_rate kucultursen n_estimators buyut (ters orantili).
```

KISIM 10

Denetimsiz Öğrenme

50. K-Means Kümeleme

Amaç Fonksiyonu

Küme içi kareler toplamını (inertia) minimize et:

$$J = \sum_{i=1}^n \min_c \|x_i - \mu_c\|^2.$$

Lloyd algoritması iki adımı yineler: (1) her noktayı en yakın merkeze ata, (2) merkezleri atanan noktaların ortalaması yap.

Nedir? Niçin? Nerede?

Kümeleme hangi soruyu cevaplar? Şimdiye kadarki her modelde bir “doğru cevap” (etiket) vardı. Kümelemede yoktur — elinde sadece veri var ve soru şu: “*Bu veride doğal olarak kaç grup var ve hangi gözlemler birbirine benziyor?*” Etiket olmadan öğrendiği için **denetimsiz** öğrenme denir.

J formülü ne diyor? Her noktanın, ait olduğu kümenin merkezine uzaklığının karesi — hepsi toplanır. Küçük J demek, “noktalar kendi merkezlerine sıkıca yapışmış” demektir. Algoritma bu toplamı en küçük yapan küme atamasını arar.

Lloyd algoritması niçin işe yarıyor? İki adım da J 'yi asla artırmaz: (1) her noktayı en yakın merkeze atamak, o noktanın katkısı küçültür; (2) merkezi ortalamaya taşımak, o kümenin kareler toplamını küçültür (Kısım 2: ortalama, karesel sapmayı minimize eden noktadır). Sürekli azalan ve alttan sınırlı bir dizi olduğu için mutlaka durur. Ama **yerel** bir minimuma takılabilir — bu yüzden `n_init` ile farklı başlangıçlardan defalarca çalıştırılır.

k 'yı kim seçiyor? Sen. Ve bu, K-Means'in en büyük zorluğudur. Yardımcı araçlar: *dirsek yöntemi* (J 'nin düşüş hızının kırıldığı nokta), *siluet skoru* (noktalar kendi kümesine mi yoksa komşusuna mı daha yakın?), ve — en önemlisi — *alan bilgisi*. “Pazarlama ekibi 4 segmentle çalışabilir” cümlesi çoğu zaman siluet skorundan daha belirleyicidir.

Varsayımları (ve ne zaman kırıldığı):

- Kümeler *küresel* ve benzer büyüklüktedir. Uzun, ince veya iç içe kümelerde başarısız olur — o zaman DBSCAN veya spektral kümeleme gerekir.
- Uzaklık temellidir, dolayısıyla **ölçekleme zorunludur**.
- Her nokta bir kümeye atanır; aykırı değerler bir kümeye zorla girer ve merkezi çeker. Aykırı değer varsa K-Medoids daha güvenlidir.

Nerede kullanılır?

- **Müşteri segmentasyonu:** RFM (recency–frequency–monetary) verisiyle müşterileri gruplara ayırmak — pazarlamanın en yaygın veri bilimi uygulaması.
- **Görüntü sıkıştırma:** Milyonlarca rengi 16 renge indirmek — her piksel en yakın küme merkezine yuvarlanır (renk nicemleme).
- **Belge/haber gruplama:** Benzer içerikleri otomatik toplamak.
- **Anomali tespiti:** Hiçbir merkeze yakın olmayan noktalar şüpheli.
- **Öznitelik mühendisliği:** Küme numarasını yeni bir kategorik öznitelik olarak denetimli modele vermek.

Önemli uyarı: K-Means *her zaman k küme bulur* — veri tamamen homojen olsa bile. “Küme buldum” demek “gerçekten grup var” demek değildir. Bulunan kümelerin anlamlı olup olmadığını her zaman alan bilgisiyle doğrula.

```
1 import numpy as np, pandas as pd
2 from sklearn.datasets import load_wine
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.cluster import KMeans
5 from sklearn.metrics import (silhouette_score, adjusted_rand_score,
6                               calinski_harabasz_score, davies_bouldin_score)
7
8 wine = load_wine(as_frame=True); Xw, yw = wine.data, wine.target
9 Xs = StandardScaler().fit_transform(Xw)          # ÖLÇEKLEME SART
10
11 for k in range(2, 8):
```

```

12 km = KMeans(k, n_init=10, random_state=42).fit(Xs)
13 print(f"k={k}: inertia={km.inertia_:8.1f} silhouette={silhouette_score(Xs, km.labels_)
14       :.4f}")
15       f" CH={calinski_harabasz_score(Xs, km.labels_):7.1f}"
16       f" DB={davies_bouldin_score(Xs, km.labels_):.4f}")

```

```

k=2: inertia= 1658.8 silhouette=0.2593 CH= 69.5 DB=1.5260
k=3: inertia= 1277.9 silhouette=0.2849 CH= 70.9 DB=1.3892
k=4: inertia= 1175.4 silhouette=0.2602 CH= 56.2 DB=1.7969
k=5: inertia= 1109.5 silhouette=0.2016 CH= 47.0 DB=1.8083
k=6: inertia= 1046.0 silhouette=0.2372 CH= 41.7 DB=1.5544
k=7: inertia= 981.6 silhouette=0.2036 CH= 38.7 DB=1.6606

```

Ölçüt	Yorum	Yön
Inertia (elbow)	Küme içi yayılım	Küçük iyi; ama k ile hep düşer → dirsek ara
Silhouette	Kendi kümesine yakınlık / diğerlerine uzaklık	$[-1, 1]$, büyük iyi
Calinski–Harabasz	Gruplar arası / grup içi varyans	Büyük iyi
Davies–Bouldin	Küme benzerliği	Küçük iyi
ARI	Gerçek etiketlerle uyum (varsa)	$[-1, 1]$, büyük iyi

Üç iç ölçüt de $k = 3$ 'ü işaret ediyor — ve şarap veri setinde gerçekten 3 üzüm çeşidi var.

```

1 km3 = KMeans(3, n_init=10, random_state=42).fit(Xs)
2 print(f"ARI (gerçek sınıflarla) = {adjusted_rand_score(yw, km3.labels_):.4f}")
3 print(pd.crosstab(yw, km3.labels_))

```

```

ARI (gerçek sınıflarla) = 0.8975
col_0    0    1    2
target
0         0    0  59
1        65    3    3
2         0  48    0

```

Bilgi

Etiketleri hiç görmeden, yalnızca kimyasal ölçümlerden üç üzüm çeşidinin 178 örneğinin 172'si doğru gruplandı (ARI = 0,90). Kümeleme etiket üretmez, *yapı* bulur; etiketle eşleştirme senin yorumundur.

Dikkat

K-Means'in varsayımları: küresel, benzer boyutlu, benzer yoğunluklu kümeler. Uzamış, iç içe geçmiş veya farklı yoğunluktaki kümelerde başarısız olur. Ayrıca n_init varsayılını yetersizse kötü yerel minimuma takılır — her zaman $n_init=10$ veya daha fazlasını ver.

51. Diğer Kümeleme Yöntemleri

Yöntem	Fikir	Avantaj / kısıt
K-Means	Merkezlere uzaklık	Hızlı; k verilmeli, küresel kümeler
Hiyerarşik	Ardışık birleştirme	Dendrogram; k sonradan seçilir, $O(n^2)$
DBSCAN	Yoğunluk temelli	Rastgele şekil, gürültü tespiti; eps hassas
GMM	Olasılıksal karışım	Elips kümeler, yumuşak atama, BIC ile k
Spektral	Grafik Laplacian	İç içe geçmiş kümeler; pahalı

```

1 from sklearn.cluster import AgglomerativeClustering, DBSCAN
2 from sklearn.mixture import GaussianMixture
3
4 ag = AgglomerativeClustering(3).fit(Xs)
5 print(f"Agglomerative ARI = {adjusted_rand_score(yw, ag.labels_):.4f}")
6
7 for eps in [1.5, 2.0, 2.5, 3.0]:
8     db = DBSCAN(eps=eps, min_samples=5).fit(Xs)
9     nk = len(set(db.labels_)) - (1 if -1 in db.labels_ else 0)
10    print(f"DBSCAN eps={eps}: kume={nk}   gurultu={db.labels_==-1}.sum():3d}   "
11          f"ARI={adjusted_rand_score(yw, db.labels_):.4f}")
12
13 for k in range(1, 7):
14     g = GaussianMixture(k, random_state=42, n_init=3).fit(Xs)
15     print(f"GMM k={k}: BIC={g.bic(Xs):8.1f}   AIC={g.aic(Xs):8.1f}")

```

```

Agglomerative ARI = 0.7899
DBSCAN eps=1.5: kume=0   gurultu=178   ARI=0.0000
DBSCAN eps=2.0: kume=5   gurultu= 85   ARI=0.2205
DBSCAN eps=2.5: kume=1   gurultu= 24   ARI=0.0050
DBSCAN eps=3.0: kume=1   gurultu= 11   ARI=-0.0059
GMM k=1: BIC= 5741.3   AIC= 5410.4
GMM k=2: BIC= 5629.6   AIC= 4964.6
GMM k=3: BIC= 5806.3   AIC= 4807.2
GMM k=4: BIC= 5806.8   AIC= 4473.7
GMM k=5: BIC= 6360.4   AIC= 4693.1
GMM k=6: BIC= 6752.7   AIC= 4751.3

```

Dikkat

Bu 13 boyutlu veride **DBSCAN tamamen başarısız**: yüksek boyutta yoğunluk kavramı anlamını yitirir (boyut laneti) ve eps için işe yarar bir değer yoktur. DBSCAN'i düşük boyutlu, yoğunluk farkı belirgin verilerde kullan — ya da önce PCA ile boyut indir.

Bilgi

GMM'de BIC $k = 2$ 'yi, AIC ise daha büyük k 'ları seçiyor. BIC daha muhafazakârdır (karmaşıklık log n ile cezalandırır), AIC daha cömerttir. Küme sayısı gibi *yapısal* kararlarda BIC tercih edilir.

52. Boyut İndirgeme: PCA

PCA

Veriyi, varyansı en çok koruyan dik eksenlere yansıtır. Bu eksenler kovaryans matrisinin özvektörleridir; açıklanan varyans oranları da özdeğerlerdir:

$$\Sigma = \frac{1}{n-1} X^T X, \quad \Sigma v_j = \lambda_j v_j, \quad \text{açıklanan oran}_j = \frac{\lambda_j}{\sum_k \lambda_k}.$$

Nedir? Niçin? Nerede?

Hangi problemi çözüyor? 30 öznitelikli bir veriyi göremezsin, ve öznitelikler birbiriyle ilişkiliyse çoğu *aynı bilgiyi tekrar tekrar* taşıyordur. PCA, bu tekrarı sıkıştırır: ilişkili birçok değişkeni, birbiriyle ilişkisiz az sayıda “bileşene” indirir.

Sezgi — gölge benzetmesi. Elinde üç boyutlu bir nesne var ve onu duvara yansıtacaksın. Hangi açıdan tutarsan gölge en çok bilgi taşır? Nesnenin en *uzun* olduğu yönden. PCA tam olarak bunu yapar: verinin en çok yayıldığı yönü bulur (birinci bileşen), sonra ona *dik* olan yönlerden en çok yayılanı (ikinci bileşen), ve böyle devam eder.

Niçin “en çok varyans” iyi bir ölçüt? Çünkü varyans = bilgi. Bütün gözlemlerde aynı olan bir değişken hiçbir şey ayırt etmez; çok değişen bir değişken çok şey ayırt eder. En az varyanslı yönleri atmak, en az bilgi kaybetmek demektir.

Özvektör–özdeğer bağlantısı nereden geliyor? “Varyansı en büyük yapan yön” problemi, kovaryans matrisinin en büyük özdeğerine karşılık gelen özvektörü bulmakla *aynı problem*dir. Özdeğer λ_j , o yöndeki varyansın kendisidir — bu yüzden “açıklanan oran” basitçe λ_j ’nin toplama bölümüdür.

Kaç bileşen tutmalı? Üç ölçüt: (1) kümülatif açıklanan varyans %90–95’i geçene kadar; (2) scree grafiğinde dirsek noktası; (3) sonraki modelin çapraz doğrulama skoru — en dürüst ölçüt budur.

Nerede kullanılır?

- **Görselleştirme:** 50 boyutlu veriyi 2 boyuta indirip gruplaşmayı gözle görmek.
- **Gürültü azaltma:** Küçük özdeğerli bileşenler genellikle gürültüdür; atınca sinyal temizlenir. Görüntü ve sinyal işlemede standarttır.
- **Çoklu bağlantı çözümü:** Bileşenler tanımı gereği ilişkisizdir, dolayısıyla regresyondaki VIF sorununu bitirir (*temel bileşen regresyonu*).
- **Hız:** 10.000 öznitelikli veriyi 100 bileşene indirip modeli 100 kat hızlandırmak.
- **Yüz tanıma:** Klasik “eigenfaces” yöntemi doğrudan PCA’dır.
- **Finans:** Faiz oranı eğrisinin %95’i üç bileşenle açıklanır: seviye, eğim ve eğrilik. Tüm tahvil risk yönetimi bu ayrıştırma üzerine kuruludur.

Bedeli nedir? *Yorumlanabilirlik.* “PC1” artık “metrekare” gibi anlamlı bir şey değil, tüm değişkenlerin ağırlıklı bir karışımıdır. Bileşenleri anlamamanın tek yolu *yüklemelere* (loadings) bakmaktır — hangi değişkenler o bileşene çok katkı yapıyor? Aşağıdaki kod tam olarak bunu yapıyor.

```
1 from sklearn.decomposition import PCA
2 p = PCA().fit(Xs)
3 print("aciklanan varyans (ilk 6):", np.round(p.explained_variance_ratio_[:6], 4))
4 print("kumulatif          :", np.round(np.cumsum(p.explained_variance_ratio_)[:6], 4))
5 n95 = np.searchsorted(np.cumsum(p.explained_variance_ratio_), 0.95) + 1
6 print(f"%95 varyans icin bilesen sayisi = {n95}")
```

```
aciklanan varyans (ilk 6): [0.362  0.1921 0.1112 0.0707 0.0656 0.0494]
```

```
kumulatif      : [0.362  0.5541 0.6653 0.736  0.8016 0.851 ]
%95 varyans icin bileşen sayısı = 10
```

```
1 # Yuklemeler (loadings): bileşenleri YORUMLAMANIN tek yolu
2 p2 = PCA(2).fit(Xs)
3 yuk = pd.DataFrame(p2.components_.T, index=Xw.columns, columns=['PC1', 'PC2'])
4 print(yuk.round(3))
```

	PC1	PC2
alcohol	0.144	0.484
malic_acid	-0.245	0.225
ash	-0.002	0.316
alcalinity_of_ash	-0.239	-0.011
magnesium	0.142	0.300
total_phenols	0.395	0.065
flavanoids	0.423	-0.003
nonflavanoid_phenols	-0.299	0.029
proanthocyanins	0.313	0.039
color_intensity	-0.089	0.530
hue	0.297	-0.279
od280/od315_of_diluted_wines	0.376	-0.164
proline	0.287	0.365

Bilgi

PC1'e en çok katkı veren değişkenler flavanoids (0,42), total_phenols (0,40) ve od280 (0,38) — yani PC1 bir *“fenolik zenginlik”* eksenidir. PC2 ise color_intensity (0,53) ve alcohol (0,48) ile yüklü: *“gövde/reng”* eksenidir. Bileşenler ancak yüklemeler okunarak yorumlanır.

Dikkat

PCA **ölçeklemeye bağımlıdır**: ölçeklemezsen en büyük birimli değişken (burada proline, binlerce birim) tüm varyansı domine eder ve PCA anlamsızlaşır. Ayrıca PCA *denetimsizdir*: en çok varyans taşıyan yön, sınıfları en iyi ayıran yön olmak zorunda değildir — o iş için LDA kullanılır.

52.1 t-SNE ve UMAP: Görselleştirme İçin

```
1 from sklearn.manifold import TSNE
2 from sklearn.datasets import load_digits
3
4 digits = load_digits()
5 Z = TSNE(n_components=2, perplexity=30, init='pca', random_state=42).fit_transform(digits.
6     data)
7
8 import matplotlib.pyplot as plt
9 plt.figure(figsize=(7, 6))
10 sc = plt.scatter(Z[:,0], Z[:,1], c=digits.target, cmap='tab10', s=8)
11 plt.colorbar(sc); plt.title('digits: t-SNE'); plt.show()
```

Dikkat

t-SNE ve UMAP yalnızca **görselleştirme** içindir: (1) kümeler arası *uzaklıklar* anlamlı değildir, (2) küme *boyutları* anlamlı değildir, (3) perplexity'yi değiştirince görüntü değişir, (4) transform yoktur — yeni veriye uygulanamaz. Boyut indirgemeyi bir modelin ön işleme adımı yapacaksan PCA kullan.

53. Anomali Tespiti

```

1 from sklearn.ensemble import IsolationForest
2 from sklearn.neighbors import LocalOutlierFactor
3 from sklearn.datasets import load_breast_cancer
4
5 bc = load_breast_cancer()
6 Xb = StandardScaler().fit_transform(bc.data)
7
8 iso = IsolationForest(contamination=0.05, random_state=42).fit(Xb)
9 pred = iso.predict(Xb) # -1: anomali
10 print(f"IsolationForest anomali = {(pred==-1).sum()} / {len(pred)}")
11 print(f"anomalilerin gerçek sınıf dağılımı = {np.bincount(bc.target[pred==-1])}")
12
13 lof = LocalOutlierFactor(n_neighbors=20, contamination=0.05).fit_predict(Xb)
14 print(f"LOF anomali = {(lof==-1).sum()}")

```

```

IsolationForest anomali = 29 / 569
anomalilerin gerçek sınıf dağılımı = [22  7]
LOF anomali = 29

```

Bilgi

Tespit edilen 29 anomalinin 22'si kötü huylu tümör (sınıf 0) — etiketler hiç kullanılmadan. Anomali tespiti, etiketli veri bulmanın zor olduğu alanlarda (dolandırıcılık, arıza, siber saldırı) tam olarak bu yüzden kullanılır: “normal”i öğrenip sapanları işaretler.

Yöntem	Ne zaman?
IsolationForest	Genel amaçlı, yüksek boyut, hızlı. Varsayılan tercih.
LocalOutlierFactor	Yerel yoğunluk farkları önemliyse.
OneClassSVM	Küçük veri, karmaşık sınır; ölçeklemeye duyarlı.
EllipticEnvelope	Veri yaklaşık normal ve tek tepeliyse.

KISIM 11

İleri Konular

54. Dengesiz Veri

Dolandırıcılık, nadir hastalık, makine arızası — gerçek problemlerin çoğunda ilgilendiğimiz sınıf %1–5 orandadır. Bu, “doğruluk” metriğini işe yaramaz hâle getirir.

Dengesizlik Niçin Bir Sorun?

Bir model, kaybı minimize etmeye çalışır. Pozitif sınıf %2 ise, “hepsine negatif de” stratejisi %98 doğruluk verir — ve model bunu bulmakta hiç zorlanmaz. Sorun modelin kötü olması değil; **ona sorduğun sorunun yanlış olmasıdır**.

Bu, Kısım 3’teki *taban oranı yanlıgısının* makine öğrenmesindeki karşılığıdır ve aynı sonucu doğurur: nadir bir olayda, iyi bir sınıflandırıcı bile çok sayıda yanlış pozitif üretir.

Dört müdahale yolu — ve hangisi ne zaman:

- **Metriği değiştir (her zaman yap).** Doğruluk yerine PR-AUC, duyarlılık, F1 veya doğru-dan iş maliyeti. Bu, en ucuz ve en etkili adımdır — çoğu “dengesiz veri sorunu”, aslında “yanlış metrik sorunudur”.
- **Sınıf ağırlığı (`class_weight='balanced'`).** Azınlık sınıfının hatalarını kayıp fonksiyonunda daha ağır say. Veriye dokunmadığı için en temiz çözümdür; ilk deneyeceğin şey olsun.
- **Yeniden örnekleme (SMOTE, alt/üst örnekleme).** Azınlık sınıfını çoğalt veya çoğunluğu azalt. *Kritik uyarı:* bu işlem yalnızca **eğitim** katmanında yapılmalıdır. Bölmeden önce SMOTE uygularsan sentetik örnekler test kümesine sızar ve skorların anlamsızlaşır — `imblearn.pipeline.Pipeline` bunu doğru yapar.
- **Karar eşiğini ayarla.** Genellikle en yüksek getirili ve en çok ihmal edilen adım. Model zaten iyi olasılıklar üretiyor olabilir; sorun 0,5 eşiğinin bu problem için yanlış olmasıdır. Bir sonraki alt bölümün konusu budur.

Önemli nüans: Dengesizlik *her zaman* bir sorun değildir. Eğer modelin ürettiği olasılıklar iyiye ve sen sıralamayla ilgileniyorsan (“en riskli 100 işlemi incele”), hiçbir müdahaleye gerek yoktur. Dahası, yeniden örnekleme olasılıkları bozar — kalibrasyon önemliyse ondan kaçın.

Nerede karşına çıkar? Dolandırıcılık tespiti (%0,1 pozitif), tıbbi tarama, üretimde kusur tespiti, müşteri kaybı tahmini, reklam tıklama tahmini (%0,5 CTR), ağ saldırısı tespiti — yani gerçek dünyadaki problemlerin çoğunda.

```
1 import numpy as np, pandas as pd
```

```

2 from sklearn.datasets import make_classification
3 from sklearn.model_selection import train_test_split
4 from sklearn.dummy import DummyClassifier
5 from sklearn.linear_model import LogisticRegression
6 from sklearn.pipeline import make_pipeline
7 from sklearn.preprocessing import StandardScaler
8 from sklearn.metrics import (accuracy_score, precision_score, recall_score,
9                               f1_score, roc_auc_score, average_precision_score)
10
11 X, y = make_classification(n_samples=10_000, n_features=20, n_informative=5,
12                           weights=[0.98, 0.02], random_state=42)
13 print("sinif dagilimi:", np.bincount(y), f"pozitif oran = {y.mean():.4f}")
14 Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=.3, random_state=42, stratify=y)
15
16 def rapor(ad, m):
17     yh, yp = m.predict(Xte), m.predict_proba(Xte)[: , 1]
18     print(f"{ad:>26}: acc={accuracy_score(yte,yh):.4f} "
19           f"prec={precision_score(yte,yh,zero_division=0):.4f} rec={recall_score(yte,yh):.4f} "
20           f"f1={f1_score(yte,yh):.4f} roc={roc_auc_score(yte,yp):.4f} "
21           f"pr_auc={average_precision_score(yte,yp):.4f}")
22
23 rapor("hep-cogunluk (dummy)", DummyClassifier(strategy='most_frequent').fit(Xtr, ytr))
24 rapor("lojistik (duz)",
25       make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000, random_state=42)).
26       fit(Xtr, ytr))
27 rapor("lojistik (class_weight)",
28       make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000,
29                                                           class_weight='balanced', random_state=42)).fit(Xtr, ytr))

```

```

sinif dagilimi: [9744 256] pozitif oran = 0.0256
hep-cogunluk (dummy): acc=0.9743 prec=0.0000 rec=0.0000 f1=0.0000 roc=0.5000 pr_auc=0.0257
lojistik (duz): acc=0.9743 prec=0.0000 rec=0.0000 f1=0.0000 roc=0.7590 pr_auc=0.1349
lojistik (class_weight): acc=0.6860 prec=0.0555 rec=0.7013 f1=0.1029 roc=0.7481 pr_auc=0.1045

```

Dikkat

“Her şeye negatif de” diyen aptal model **%97,43** doğruluk alıyor — ve eğitilmiş lojistik regresyon da tam olarak bunu yapıyor (duyarlılık = 0). Doğruluk burada tamamen anlamsızdır. Kullanılacak metrikler: **PR-AUC**, duyarlılık, kesinlik ve iş maliyetine dayalı özel bir skor.

Strateji	Açıklama
<code>class_weight='balanced'</code>	Kayıp fonksiyonunda azınlığa daha büyük ağırlık. İlk denenecek.
Eşik ayarı	Karar eşiğini 0,5'ten indir; model yeniden eğitilmez.
Rastgele alt örnekleme	Çoğunluğu azalt; hızlı ama bilgi kaybı.
SMOTE	Azınlık için sentetik örnek üret (imbalanced-learn); <i>yalnız eğitim katlamasında.</i>
Maliyet duyarlı öğrenme	FP ve FN maliyetlerini doğrudan modele ver.
Anomali tespiti	Azınlık <%1 ise sınıflandırma yerine anomali yaklaşımı.

54.1 Eşik Ayarı: Bedava Kazanç

```
1 from sklearn.metrics import precision_recall_curve
2 m1 = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000, random_state=42)).fit
   (Xtr, ytr)
3 yp = m1.predict_proba(Xte)[: , 1]
4
5 prec, rec, thr = precision_recall_curve(yte, yp)
6 f1s = 2*prec*rec/(prec + rec + 1e-12)
7 i = np.nanargmax(f1s)
8 print(f"varsayılan esik 0.50 -> f1 = {f1_score(yte, (yp > 0.5).astype(int)):.4f}")
9 print(f"en iyi esik {thr[i]:.4f} -> f1 = {f1s[i]:.4f} "
10       f"(kesinlik={prec[i]:.4f}, duyarlılık={rec[i]:.4f})")
```

```
varsayılan esik 0.50 -> f1 = 0.0000
en iyi esik 0.0820 -> f1 = 0.2379 (kesinlik=0.1800, duyarlılık=0.3506)
```

İpucu

Eşiği 0,5'te bırakmak bir *varsayılan*, bir karar değildir. Doğru eşik, hataların **maliyetinden** çıkar: kanser taramasında FN maliyeti FP'nin 100 katıysa eşik çok düşük olmalıdır. Eşiği eğitim/doğrulama verisiyle seç, test verisiyle *asla*.

55. Model Yorumlama

55.1 Permütasyon Önemi

Formül

Bir öznitelğin değerlerini rastgele karıştır; skor ne kadar düşerse o öznitelik o kadar önemlidir. Model bağımsızdır, test verisinde hesaplanır ve Gini öneminin yanlılıklarından muaftır.

Nedir? Niçin? Nerede?

Fikir — Kısım 6’daki permütasyon testinin ta kendisi. Mantık aynı: eğer bir öznelik gerçekten işe yarıyorsa, değerlerini karıştırıp anlamsızlaştırdığında model kötüleşmelidir. Kötüleşmiyorsa o öznelik zaten kullanılmıyordu.

Niçin test verisinde hesaplanır? Çünkü sorduğun soru “model bu özneliğe ne kadar *bel bağlıyor?*” değil, “bu öznelik *gerçek* performansa ne kadar katkı yapıyor?”dur. Eğitim verisinde hesaplırsan, ezberlenmiş gürültüyü de “önemli” sayarsın.

Ağacın kendi (Gini) önemine niçin güvenilmez? Üç sistematik yanlılığı vardır: (1) çok değerli özneliklere (sürekli değişkenler, yüksek kardinaliteli kategoriler) haksız avantaj verir — daha çok bölme fırsatı buldukları için; (2) eğitim verisinde hesaplanır, dolayısıyla aşırı öğrenmeyi yansıtır; (3) yalnızca ağaç tabanlı modellerde vardır. Permütasyon önemi bu üçünü de çözer ve *her* modelde çalışır — sinir ağı, SVM, hatta bir kara kutu API.

Bilinmesi gereken zayıflığı: Korelasyonlu öznelikler. İki öznelik aynı bilgiyi taşıyorsa, birini karıştırdığında model diğerine dayanır ve skor düşmez — *ikisi de önemsiz görünür*, oysa birlikte kritiktirler. Çözüm: önce korelasyonlu grupları belirle, sonra grubu *birlikte* karıştır.

Nerede kullanılır?

- **Öznelik seçimi:** Önemi sıfır (veya negatif) çıkanları elemek — veri toplama maliyetini düşürür.
- **Model doğrulama:** Beklenmedik bir özneliğin çok önemli çıkması, genellikle *veri sızıntısı* işaretidir (Kısım 8). “Müşteri numarası” önemli çıkıyorsa bir şeyler yanlışdır.
- **Paydaş iletişimi:** “Model en çok şu üç faktöre bakıyor” cümlesi, bir kara kutuyu tartışılabilir hâle getirir.
- **Düzenleyici uyum:** Kredi ve sigorta gibi alanlarda modelin neye dayandığını gösterme zorunluluğu.

Pratik ayrıntı: `n_repeats` ile birden çok kez karıştır ve standart sapmaya bak. Aşağıdaki çıktıda birçok özneliğin `perm_std` değeri kendi `perm` değerinden büyük — yani o önemler gürültüden ayırt edilemiyor. Tek bir sıralamaya bakıp karar vermek yerine, bu belirsizliği de raporlamak gerekir.

```
1 from sklearn.datasets import load_breast_cancer
2 from sklearn.ensemble import RandomForestClassifier
3 from sklearn.inspection import permutation_importance
4
5 bc = load_breast_cancer(as_frame=True); Xb, yb = bc.data, bc.target
6 Xbtr, Xbte, ybtr, ybte = train_test_split(Xb, yb, test_size=.3, random_state=42, stratify=yb
7 )
8 rf = RandomForestClassifier(n_estimators=300, random_state=42, n_jobs=-1).fit(Xbtr, ybtr)
9
10 pi = permutation_importance(rf, Xbte, ybte, n_repeats=20, random_state=42, n_jobs=-1)
11 s = pd.DataFrame({'gini': rf.feature_importances_,
12                  'perm': pi.importances_mean,
13                  'perm_std': pi.importances_std}, index=Xb.columns)
14 print(s.sort_values('perm', ascending=False).head(8).round(4))
```

	gini	perm	perm_std
worst texture	0.0163	0.0094	0.0034
mean texture	0.0096	0.0047	0.0035
mean concavity	0.0400	0.0023	0.0029
area error	0.0206	0.0020	0.0028
perimeter error	0.0075	0.0020	0.0028
mean concave points	0.0803	0.0009	0.0046
worst concave points	0.0371	0.0009	0.0046

worst symmetry	0.0137	0.0006	0.0018
----------------	--------	--------	--------

Dikkat

İki sıralama *çok* farklı: Gini'ye göre en önemli olan mean concave points (0,080), permütasyona göre neredeyse önemsiz (0,0009). Nedeni **korelasyonlu öznitelikler**: bir tanesini bozarsan model kardeşini kullanır ve skor düşmez. Ders: korelasyonlu özniteliklerde *tekil* önem yanıltıcıdır; gruplar hâlinde permütasyon veya SHAP kullan.

55.2 Kısmi Bağımlılık ve SHAP

```
1 from sklearn.inspection import PartialDependenceDisplay
2 import matplotlib.pyplot as plt
3
4 PartialDependenceDisplay.from_estimator(
5     rf, Xbte, features=['worst texture', 'mean radius',
6                        ('worst texture', 'mean radius')], kind='average')
7 plt.tight_layout(); plt.show()

1 # SHAP: her tahmin için özniteliklerin katkısı (oyun teorisi temelli)
2 import shap
3 exp = shap.TreeExplainer(rf)
4 sv = exp.shap_values(Xbte)
5
6 shap.summary_plot(sv[...], Xbte) # küresel önem + yön
7 shap.plots.waterfall(shap.Explanation( # tek bir tahminin açıklaması
8     values=sv[0, :, 1], base_values=exp.expected_value[1],
9     data=Xbte.iloc[0], feature_names=list(Xb.columns)))
```

Araç	Ne söyler?
Katsayılar	Doğrusal modelde etkinin yönü ve büyüklüğü (yorumlanabilirliğin altın standardı).
Permütasyon önemi	Küresel: hangi öznitelik olmadan skor düşer?
Kısmi bağımlılık (PDP)	Küresel: öznitelik değiştikçe tahmin nasıl değişir?
ICE	Bireysel: her gözlem için ayrı PDP eğrisi.
SHAP	Hem küresel hem yerel : bu tahmine hangi öznitelik ne kadar katkı verdi?
LIME	Yerel: tek tahmin çevresinde basit model uydurur.

56. Olasılık Kalibrasyonu

Bir model “%80 olasılıkla pozitif” dediğinde, bu tür tahminlerin gerçekten %80'i pozitif çıkmalıdır. Aksi hâlde *sıralama* doğru olsa bile *olasılıklar* yanlıştır — ve karar eşiği, beklenen maliyet hesabı, risk skorları hep bozulur.

```
1 from sklearn.calibration import CalibratedClassifierCV, calibration_curve
2 from sklearn.metrics import brier_score_loss
3
```

```

4 lr = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000)).fit(Xbtr, ybtr)
5 cal = CalibratedClassifierCV(RandomForestClassifier(n_estimators=300, random_state=42),
6                             cv=5, method='isotonic').fit(Xbtr, ybtr)
7
8 for ad, p in [('RandomForest', rf.predict_proba(Xbte)[: ,1]),
9              ('LogisticRegression', lr.predict_proba(Xbte)[: ,1]),
10             ('RF + isotonic', cal.predict_proba(Xbte)[: ,1])]:
11     print(f"{ad:>20}: Brier = {brier_score_loss(ybte, p):.4f} "
12           f"roc_auc = {roc_auc_score(ybte, p):.4f}")

```

```

RandomForest: Brier = 0.0358 roc_auc = 0.9920
LogisticRegression: Brier = 0.0170 roc_auc = 0.9981
RF + isotonic: Brier = 0.0506 roc_auc = 0.9829

```

Bilgi

Lojistik regresyon doğal olarak kalibredir (Brier = 0,017); Random Forest daha kötüdür. Burada isotonic kalibrasyon işe *yaramadı* — $n = 398$ eğitim örneği isotonic için az, aşırı öğreniyor. Küçük veride method='sigmoid' (Platt ölçekleme) daha güvenlidir. Kalibrasyonu her zaman ölç (*Brier skoru* + kalibrasyon eğrisi), varsayma.

57. Zaman Serisi Verisi

Zaman Serisi Niçin Ayrı Bir Dünya?

Bu rehberdeki hemen her yöntem tek bir varsayıma dayanıyordu: **gözlemler bağımsızdır**. Zaman serisinde bu varsayım kökünden yanlıştır — bugünün satışı dünün satışına benzer, bu haftanın trafiği geçen haftaninkine. Bağımlılık burada bir sorun değil, *tam olarak modellemek istediğin şeydir*.

Bağımsızlık gittiği için standart araç kutusu da gider: rastgele eğitim/test ayrımı geçersizdir, standart çapraz doğrulama geçersizdir, *t*-testinin ve regresyonun standart hataları yanlıştır (otokorelasyon onları sistematik olarak küçültür ve sahte anlamlılık üretir).

Üç temel kavram:

- **Durağanlık (stationarity)**. Serinin istatistiksel özellikleri (ortalama, varyans) zamanla değişmiyorsa durağandır. Neredeyse tüm zaman serisi modelleri durağanlık ister — çünkü değişen bir şeyi tahmin etmenin yolu, önce onu değişmez hâle getirmektir. ADF testi bunu sınar. Durağan değilse: *fark alma* (bugün – dün) trendi, *logaritma* artan varyansı, *mevsimsel fark* yıllık deseni giderir. Aşağıdaki kodda ham seri durağan değil, log-farkı durağan.
- **Ayrıştırma (STL)**. Bir seri üç parçadan oluşur: *trend* (uzun vadeli yön), *mevsimsellik* (tekrarlayan desen) ve *artık* (geri kalan). Bu ayrıştırma tek başına bile çok bilgi verir — “satışlar düştü” mü yoksa “her yıl bu ayda düşer” mi?
- **Otokorelasyon (ACF/PACF)**. Serinin kendi geçmişiyile korelasyonu. ARIMA’nın *p* ve *q* derecelerini bu grafiklerden okursun; ayrıca artıklarda otokorelasyon kalmışsa model eksik demektir.

Doğrulama nasıl yapılır? *İleri doğru zincirleme* (walk-forward): ilk 24 ayla eğit, 25. ayı tahmin et; ilk 25 ayla eğit, 26. ayı tahmin et... `TimeSeriesSplit` tam olarak bunu yapar. Ölüğün skor böylece gerçek kullanım senaryosunu taklit eder.

Nerede kullanılır? Talep ve stok planlama; enerji tüketimi tahmini; finansal fiyat ve volatilité modellemesi; sunucu kapasite planlama; hastalık yayılımı tahmini; sensör verisinde anomali tespiti.

En sık yapılan üç hata: (1) rastgele bölme; (2) tahmin anında mevcut olmayacak bir öznitelik kullanmak (“o günün hava durumu” — ileriye dönük tahminde bilinmez); (3) durağanlığı kontrol etmeden regresyon kurup *sahte regresyon* elde etmek — iki ilgisiz ama ikisi de trendli seri arasında $R^2 = 0,95$ çıkması çok kolaydır.

Dikkat

Zaman serisinde **rastgele** eğitim/test ayrımı ölümcül bir hatadır: geleceği kullanarak geçmiş tahmin edersin. Her zaman *kronolojik* ayr ve `TimeSeriesSplit` kullan.

```
1 import statsmodels.api as sm, pandas as pd, numpy as np
2 from statsmodels.tsa.stattools import adfuller
3 from statsmodels.tsa.seasonal import STL
4 from statsmodels.tsa.arima.model import ARIMA
5
6 ap = sm.datasets.get_rdataset('AirPassengers').data # 1949-1960 aylık yolcu sayısı
7 y = pd.Series(ap['value'].values,
8               index=pd.date_range('1949-01-01', periods=len(ap), freq='MS'))
9
10 print(f"ADF (ham) : p = {adfuller(y)[1]:.4f}") # durağan değil
11 print(f"ADF (log-fark) : p = {adfuller(np.log(y).diff().dropna())[1]:.4f}")
12
13 stl = STL(np.log(y), period=12).fit()
14 print(f"STL: trend std={stl.trend.std():.4f} mevsim std={stl.seasonal.std():.4f} "
15       f"artık std={stl.resid.std():.4f}")
```

```
ADF (ham) : p = 0.9919
ADF (log-fark) : p = 0.0711
STL: trend std=0.4213 mevsim std=0.1291 artik std=0.0203
```

```
1 tr, te = y[:-24], y[-24:] # son 24 ay test
2 m = ARIMA(np.log(tr), order=(1,1,1), seasonal_order=(1,1,1,12)).fit()
3 fc = np.exp(m.forecast(24))
4
5 def mape(gercek, tahmin):
6     return np.mean(np.abs((gercek - tahmin)/gercek))*100
7
8 print(f"SARIMA(1,1,1)(1,1,1,12) MAPE = {mape(te.values, fc.values):.2f}% AIC={m.aic:.1f}")
9 print(f"naive (son deger) MAPE = {mape(te.values, np.repeat(tr.values[-1], 24)):.2f}%"
10 )
11 print(f"mevsimsel naive MAPE = {mape(te.values, np.array(tr.values[-12:].tolist()*2)
12 ):.2f}%")
```

```
SARIMA(1,1,1)(1,1,1,12) MAPE = 7.81% AIC=-385.4
naive (son deger) MAPE = 23.58%
mevsimsel naive MAPE = 15.52%
```

Bilgi

Her zaman naive temel çizgiyle karşılaştır. SARIMA %7,8 MAPE veriyor; “geçen yılın aynı ayı” demek %15,5. Model, temel çizgiyi iki kat geçiyor — işte anlamlı ilerleme budur. Çoğu “derin öğrenmeyle zaman serisi” çalışması mevsimsel naive’i geçemez.

```
1 # ML ile zaman serisi: gecikme (lag) özellikleri + TimeSeriesSplit
2 from sklearn.model_selection import TimeSeriesSplit
3 df = pd.DataFrame({'y': y})
4 for L in [1, 2, 3, 12]:
5     df[f'lag{L}'] = df.y.shift(L)
6 df['ay'] = df.index.month
7 df['trend'] = np.arange(len(df))
8 df = df.dropna()
9
10 tscv = TimeSeriesSplit(n_splits=5, test_size=12)
11 # ... cross_val_score(model, df.drop(columns='y'), df.y, cv=tscv)
```

58. Sinir Ağlarına Giriş

Çok Katmanlı Algılayıcı

$$h^{(1)} = \phi(W^{(1)}x + b^{(1)}), \quad h^{(2)} = \phi(W^{(2)}h^{(1)} + b^{(2)}), \quad \hat{y} = \text{softmax}(W^{(3)}h^{(2)} + b^{(3)})$$

ϕ doğrusal olmayan aktivasyondur (ReLU). Ağırlıklar, kaybın gradyanı *geri yayılım* ile hesaplanarak güncellenir.

Nedir? Niçin? Nerede?

Formülü sökelim. Her katman iki iş yapar: önce bir *doğrusal dönüşüm* ($Wx + b$ — yani bir çoklu regresyon), sonra bir *doğrusal olmayan sıkıştırma* (ϕ). Bu ikiliyi üst üste yığarsın. Yani bir sinir ağı, kabaca **birbirine bağlanmış çok sayıda regresyondur**.

ϕ **niçin zorunlu?** Aktivasyon olmasaydı, kaç katman koyarsan koy sonuç yine tek bir doğrusal dönüşüm olurdu ($W_2(W_1x) = (W_2W_1)x$). Derinliğin hiçbir anlamı kalmazdı. **Doğrusal olmayan aktivasyon, derinliği anlamlı kılan tek şeydir.** ReLU'nun ($\max(0, z)$) popülerliği sadeliğinden gelir: türevi 0 ya da 1'dir, dolayısıyla gradyanlar derin ağlarda sönmez.

Son katman niçin softmax? Çünkü çıktının bir olasılık dağılımı olması gerekir — Kısım 3'teki Kolmogorov aksiyomlarını hatırla: negatif olmama (üstel olarak) ve toplamın 1 olması (normalize ederek). İkili sınıflandırmada softmax yerine sigmoid kullanılır ve o durumda son katman *tam olarak bir lojistik regresyondur*.

Geri yayılım nedir? Sadece zincir kuralıdır. Kayıp fonksiyonunun her ağırlığa göre türevini, çıkıştan girişe doğru geri hesaplar. Sonra her ağırlık, gradyanın ters yönünde küçük bir adım atar (gradyan inişi). Milyonlarca parametrelili bir ağı eğitmeyi mümkün kılan tek şey, bu türevlerin tek geçişte hesaplanabilmesidir.

Niçin işe yarıyor? *Evrensel yaklaşım teoremi*: yeterince geniş tek gizli katmanlı bir ağ, herhangi bir sürekli fonksiyonu istenen hassasiyette yaklaşık verebilir. Ama bu teorem “*öğrenilebilir*” demiyor — pratikte derinlik, genişlikten daha verimlidir; her katman bir öncekinin çıkardığı öz niteliklerin üzerine daha soyut öz nitelikler kurar. Görüntüde: kenarlar $\rightarrow$ dokular $\rightarrow$ parçalar $\rightarrow$ nesneler.

Ne zaman sinir ağı, ne zaman değil?

- **Kullan:** Görüntü, ses, metin, video — yani yapısı gereği *ham* ve yüksek boyutlu veri. Bol veri (on binler ve üstü) varsa. Öz nitelik mühendisliğini modele devretmek istiyorsan.
- **Kullanma:** Tablo verisinde. Bu rehberdeki gibi satır-sütun biçimli, birkaç bin satırlık verilerde gradyan artırma (XGBoost, LightGBM) neredeyse her zaman daha iyi, daha hızlı ve daha az ayar gerektirir. Yorumlanabilirlik zorunluysa da uygun değildir.

Ayarlanması gereken şeyler: katman sayısı ve genişliği, öğrenme hızı (`learning_rate` — en kritik hiperparametre), düzenleleştirme (`alpha`, `dropout`), yığın boyutu, erken durdurma. Sinir ağlarının ayarının zor olması, tablo verisinde niçin ilk tercih olmadığına pratik sebebidir.

Ölçekleme zorunludur: Girdiler farklı ölçeklerdeyse gradyanlar dengesizleşir ve eğitim yakınsamaz. Aşağıdaki kodda `Xd = dg.data/16.0` satırı tam olarak bunun içindir.

```
1 from sklearn.neural_network import MLPClassifier
2 from sklearn.datasets import load_digits
3
4 dg = load_digits()
5 Xd, yd = dg.data/16.0, dg.target          # [0,1] araligina olcekle
6 Xdtr, Xdte, ydtr, ydte = train_test_split(Xd, yd, test_size=.25,
7                                           random_state=42, stratify=yd)
8
9 for hl in [(32,), (128,), (128, 64)]:
10     mlp = MLPClassifier(hidden_layer_sizes=hl, max_iter=600,
11                        early_stopping=True, random_state=42).fit(Xdtr, ydtr)
12     print(f"katman {str(hl):>10}: test dogrulugu = {mlp.score(Xdte, ydte):.4f} "
13           f"(epoch = {mlp.n_iter_})")
```

```
katman      (32,): test dogrulugu = 0.8956 (epoch = 44)
katman      (128,): test dogrulugu = 0.9511 (epoch = 49)
katman      (128, 64): test dogrulugu = 0.9756 (epoch = 54)
```

Bileşen	Pratik seçim
Aktivasyon	Gizli katmanlarda ReLU; çıkışta sigmoid (ikili) / softmax (çok sınıf).
Kayıp	İkili çapraz entropi, kategorik çapraz entropi, MSE (regresyon).
Optimizasyon	Adam (varsayılan); büyük veride SGD + momentum.
Düzenleştirme	Dropout, weight decay (L_2), erken durdurma, veri artırma.
Ölçekleme	Zorunlu: ölçeklenmemiş girdide ağ öğrenemez.
Kütüphane	Prototip için MLPClassifier; ciddi iş için PyTorch/Keras.

Dikkat

Tablo (tabular) verisinde sinir ağları genellikle gradient boosting'i **yenemez**. Derin öğrenmenin üstünlüğü görüntü, ses, metin gibi yapılandırılmamış verilerdedir. Elinde 1000 satırlık bir CSV varsa doğru cevap büyük ihtimalle LightGBM'dir, transformer değil.

KISIM 12

Uçtan Uca Projeler

59. Proje 1 — Titanic: Sınıflandırma Hattı

Gerçek Veri

891 yolcunun yaşı, cinsiyeti, bilet sınıfı ve aile bilgileri veriliyor; hedef hayatta kalıp kalmadığını tahmin etmek. Eksik veri (age'de 177, deck'te 688), kategorik değişkenler ve dengesiz olmayan ama asimetrik bir hedef içerir — gerçek bir sınıflandırma probleminin bütün unsurları.

59.1 Adım 1: Keşif

```
1 import numpy as np, pandas as pd, seaborn as sns
2 tit = sns.load_dataset('titanic')
3 print("sekil:", tit.shape)
4 print(tit.isna().sum()[tit.isna().sum() > 0])
5 print("\nhayatta kalma oranı:", round(tit.survived.mean(), 4))
6 print(tit.groupby('sex')['survived'].mean().round(3))
7 print(tit.groupby('pclass')['survived'].mean().round(3))
```

```
sekil: (891, 15)
age          177
embarked     2
deck         688
embark_town  2

hayatta kalma oranı: 0.3838
sex
female    0.742
male      0.189
pclass
1         0.630
2         0.473
3         0.242
```

59.2 Adım 2: Öznitelik Mühendisliği ve Pipeline

```
1 from sklearn.compose import ColumnTransformer
2 from sklearn.impute import SimpleImputer
3 from sklearn.preprocessing import StandardScaler, OneHotEncoder
4 from sklearn.pipeline import Pipeline
```

```

5 from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
6
7 df = tit.copy()
8 df['aile_buyuklugu'] = df.sibsp + df.parch + 1          # alan bilgisi ile yeni ozellik
9 df['yalniz'] = (df.aile_buyuklugu == 1).astype(int)
10
11 sayisal = ['age', 'fare', 'sibsp', 'parch', 'aile_buyuklugu']
12 kategorik = ['sex', 'embarked', 'pclass', 'who', 'yalniz']
13 X, y = df[sayisal + kategorik], df.survived
14
15 prep = ColumnTransformer([
16     ('num', Pipeline([('imp', SimpleImputer(strategy='median')),
17                       ('sc', StandardScaler())]), sayisal),
18     ('cat', Pipeline([('imp', SimpleImputer(strategy='most_frequent')),
19                       ('oh', OneHotEncoder(handle_unknown='ignore', drop='first'))]),
20     kategorik)])
21
22 Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=.2, random_state=42, stratify=y)
23 cv = StratifiedKFold(5, shuffle=True, random_state=42)

```

59.3 Adım 3: Model Karşılaştırma

```

1 from sklearn.linear_model import LogisticRegression
2 from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
3
4 for ad, clf in {'LogisticRegression': LogisticRegression(max_iter=2000, random_state=42),
5                'RandomForest': RandomForestClassifier(n_estimators=400, random_state=42,
6                n_jobs=-1),
7                'HistGB': HistGradientBoostingClassifier(random_state=42)}.items():
8     pipe = Pipeline([('prep', prep), ('clf', clf)])
9     acc = cross_val_score(pipe, Xtr, ytr, cv=cv, scoring='accuracy', n_jobs=-1)
10    auc = cross_val_score(pipe, Xtr, ytr, cv=cv, scoring='roc_auc', n_jobs=-1)
11    print(f"{ad}>20}: CV acc = {acc.mean():.4f} +- {acc.std():.4f}    CV roc_auc = {auc.mean():.4f}")

```

```

LogisticRegression: CV acc = 0.8174 +- 0.0270    CV roc_auc = 0.8632
RandomForest: CV acc = 0.8034 +- 0.0131    CV roc_auc = 0.8680
HistGB: CV acc = 0.8216 +- 0.0225    CV roc_auc = 0.8837

```

59.4 Adım 4: Ayar ve Nihai Test

```

1 from sklearn.model_selection import GridSearchCV
2 from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
3
4 pipe = Pipeline([('prep', prep), ('clf', RandomForestClassifier(random_state=42, n_jobs=-1))])
5 grid = GridSearchCV(pipe, {'clf__n_estimators': [200, 400],
6                           'clf__max_depth': [4, 6, 8, None],
7                           'clf__min_samples_leaf': [1, 3, 5]},
8                       cv=cv, scoring='roc_auc', n_jobs=-1).fit(Xtr, ytr)
9 print("en iyi:", grid.best_params_, f"    CV roc_auc = {grid.best_score_:.4f}")
10
11 best = grid.best_estimator_
12 yh, yp = best.predict(Xte), best.predict_proba(Xte)[:, 1]
13 print(f"\nTEST dogruluk = {(yh == yte).mean():.4f}    TEST roc_auc = {roc_auc_score(yte, yp):.4f}")

```

```

14 print(confusion_matrix(yte, yh))
15 print(classification_report(yte, yh, target_names=['oldu', 'hayatta']))

```

```

en iyi: {'clf__max_depth': None, 'clf__min_samples_leaf': 5, 'clf__n_estimators': 200} CV roc_auc = 0.8823

```

```

TEST doğruluk = 0.8212 TEST roc_auc = 0.8472

```

```

[[103  7]
 [ 25 44]]

```

	precision	recall	f1-score	support
oldu	0.80	0.94	0.87	110
hayatta	0.86	0.64	0.73	69
accuracy			0.82	179
macro avg	0.83	0.79	0.80	179
weighted avg	0.83	0.82	0.81	179

Dikkat

CV skoru 0,8823, test skoru 0,8472 — *beklenen* bir düşüş. Hiperparametre araması CV skorunu bir miktar iyimser yapar (seçim yanlılığı). Raporlanacak sayı **test skorudur**. Bu farkın büyük olması (örn. > 0,10) aşırı ayar (overtuning) işaretidir.

```

1 from sklearn.inspection import permutation_importance
2 pi = permutation_importance(best, Xte, yte, n_repeats=20, random_state=42, n_jobs=-1)
3 print(pd.Series(pi.importances_mean, index=X.columns).sort_values(ascending=False).round(4))

```

who	0.1318
pclass	0.0397
sex	0.0360
fare	0.0101
age	0.0098
aile_buyuklugu	0.0059
embarked	0.0020
sibsp	0.0011
parch	0.0008
yalniz	0.0006

Bilgi

En önemli değişken who (man/woman/child) — “kadınlar ve çocuklar önce” kuralının veriye yansması. sex ile büyük ölçüde örtüştüğü için ikisi önemi paylaşıyor. Kısım 3’te olasılık hesabıyla, Kısım 7’de lojistik regresyonla vardığımız sonuca üçüncü kez, bu kez ML yoluyla ulaştık: **iyi bir analizde farklı yöntemler aynı hikâyeyi anlatır**.

60. Proje 2 — California Housing: Regresyon

Gerçek Veri

1990 ABD nüfus sayımından 20 640 California bölgesi; her satır bir blok grubu. Hedef: medyan ev değeri (100 000 \$ birimiyle). Özellikler medyan gelir, ev yaşı, ortalama oda sayısı, nüfus, enlem ve boylam.

```

1 from sklearn.datasets import fetch_california_housing
2 cal = fetch_california_housing(as_frame=True)
3 X, y = cal.data, cal.target
4 print(X.assign(target=y).corr()['target'].drop('target').sort_values(ascending=False).round
      (3))

```

```

MedInc      0.688
AveRooms    0.152
HouseAge    0.106
AveOccup   -0.024
Population -0.025
Longitude  -0.046
AveBedrms  -0.047
Latitude   -0.144

```

```

1 from sklearn.model_selection import KFold
2 from sklearn.linear_model import LinearRegression, Ridge, Lasso
3 from sklearn.ensemble import RandomForestRegressor, HistGradientBoostingRegressor
4 from sklearn.dummy import DummyRegressor
5 from sklearn.pipeline import make_pipeline
6
7 Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=.2, random_state=42)
8 cv = KFold(5, shuffle=True, random_state=42)
9
10 for ad, m in {'Dummy (ortalama)': DummyRegressor(),
11              'LinearRegression': make_pipeline(StandardScaler(), LinearRegression()),
12              'Ridge(alpha=1)': make_pipeline(StandardScaler(), Ridge(1.0)),
13              'Lasso(alpha=0.01)': make_pipeline(StandardScaler(), Lasso(0.01, max_iter
14              =10000)),
15              'RandomForest': RandomForestRegressor(n_estimators=200, random_state=42,
16              n_jobs=-1),
17              'HistGB': HistGradientBoostingRegressor(random_state=42)}.items():
18     rmse = -cross_val_score(m, Xtr, ytr, cv=cv, scoring='neg_root_mean_squared_error',
19                             n_jobs=-1)
20     r2 = cross_val_score(m, Xtr, ytr, cv=cv, scoring='r2', n_jobs=-1)
21     print(f"{ad:>20}: CV RMSE = {rmse.mean():.4f}    CV R2 = {r2.mean():.4f}")

```

```

Dummy (ortalama): CV RMSE = 1.1562    CV R2 = -0.0002
LinearRegression: CV RMSE = 0.7205    CV R2 = 0.6115
Ridge(alpha=1): CV RMSE = 0.7205    CV R2 = 0.6115
Lasso(alpha=0.01): CV RMSE = 0.7239    CV R2 = 0.6078
RandomForest: CV RMSE = 0.5097    CV R2 = 0.8056
HistGB: CV RMSE = 0.4736    CV R2 = 0.8322

```

```

1 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
2 best = HistGradientBoostingRegressor(random_state=42, max_iter=500,
3                                     learning_rate=0.08).fit(Xtr, ytr)
4 pred = best.predict(Xte)
5 print(f"TEST: RMSE={np.sqrt(mean_squared_error(yte,pred)):.4f}    "
6       f"MAE={mean_absolute_error(yte,pred):.4f}    R2={r2_score(yte,pred):.4f}")
7
8 pi = permutation_importance(best, Xte, yte, n_repeats=10, random_state=42, n_jobs=-1)
9 print(pd.Series(pi.importances_mean, index=X.columns).sort_values(ascending=False).round(4))

```

```

TEST: RMSE=0.4595 MAE=0.3069 R2=0.8389
Latitude      0.8090

```

Longitude	0.6706
MedInc	0.5204
AveOccup	0.1578
AveRooms	0.0463
HouseAge	0.0432
Population	0.0084
AveBedrms	0.0056

Bilgi

Doğrusal modeller $R^2 = 0,61$ 'de takılıyor, ağaç toplulukları 0,84'e çıkıyor. Fark tamamen **coğrafyadan** geliyor: enlem–boylam ile fiyat arasındaki ilişki son derece doğrusal değildir (sahil şeridi pahalı, iç kesimler ucuz) ve doğrusal model bunu yakalayamaz. Korelasyon tablosunda enlemin korelasyonu yalnızca $-0,14$ iken permütasyon önemi en yüksek — *korelasyon yalnızca doğrusal ilişkiyi görür* dersinin somut bedeli.

61. Proje 3 — Meme Kanseri: Eşik ve Maliyet Analizi

Gerçek Veri

569 tümör biyopsisinden hesaplanan 30 morfolojik ölçüm (yarıçap, doku, içbükeylik...). Hedef: kötü huylu (malignant) mu, iyi huylu (benign) mi? Burada asıl soru “model ne kadar doğru” değil, “**hangi eşikte karar vermeliyiz**”dir.

```

1 from sklearn.datasets import load_breast_cancer
2 from sklearn.metrics import confusion_matrix, roc_auc_score
3
4 bc = load_breast_cancer(as_frame=True)
5 X, y = bc.data, 1 - bc.target          # pozitif sınıf = KOTU HUYLU
6 Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=.3, random_state=42, stratify=y)
7
8 m = make_pipeline(StandardScaler(),
9                    LogisticRegression(max_iter=5000, random_state=42)).fit(Xtr, ytr)
10 p = m.predict_proba(Xte)[: , 1]
11 print(f"roc_auc = {roc_auc_score(yte, p):.4f}\n")
12
13 print("esik | TP  FN  FP  TN | duyarlilik kesinlik | maliyet(FN=100, FP=5)")
14 for t in [0.9, 0.7, 0.5, 0.3, 0.2, 0.1, 0.05]:
15     tn, fp, fn, tp = confusion_matrix(yte, (p >= t).astype(int)).ravel()
16     print(f"{t:4.2f} | {tp:3d} {fn:3d} {fp:3d} {tn:3d} | "
17           f"{tp/(tp+fn):.4f}    {tp/(tp+fp):.4f} | {100*fn + 5*fp:5d}")

```

roc_auc = 0.9975

esik	TP	FN	FP	TN	duyarlilik	kesinlik	maliyet(FN=100, FP=5)
0.90	51	13	0	107	0.7969	1.0000	1300
0.70	59	5	0	107	0.9219	1.0000	500
0.50	60	4	1	106	0.9375	0.9836	405
0.30	62	2	1	106	0.9688	0.9841	205
0.20	63	1	3	104	0.9844	0.9545	115
0.10	63	1	7	100	0.9844	0.9000	135
0.05	64	0	14	93	1.0000	0.8205	70

Dikkat

Varsayılan 0,5 eşiği **4 kanser vakasını kaçırıyor**. Eşiği 0,05'e indirince hiçbir vaka kaçmıyor; bedeli 14 yanlış alarm (ek biyopsi). Kaçırılan bir kanserin maliyetini yanlış alarmın 20 katı sayarsak toplam maliyet 405 → 70'e düşüyor. **Karar eşiği bir istatistik değil, bir iş kararıdır** — ve mühendisin değil, alan uzmanının vermesi gereken bir karardır.

```
1 # Maliyeti minimize eden esigi otomatik bul
2 ts = np.linspace(0.01, 0.99, 99)
3 maliyet = [100*confusion_matrix(yte, (p>=t).astype(int))[1,0]
4           + 5*confusion_matrix(yte, (p>=t).astype(int))[0,1] for t in ts]
5 i = int(np.argmin(maliyet))
6 print(f"en dusuk maliyetli esik = {ts[i]:.2f}    maliyet = {maliyet[i]}")
```

62. Proje 4 — Bölge Segmentasyonu (Kümeleme)

```
1 from sklearn.cluster import KMeans
2 from sklearn.metrics import silhouette_score
3
4 cal = fetch_california_housing(as_frame=True)
5 df = cal.frame.copy()
6 ozellikler = ['MedInc', 'HouseAge', 'AveRooms', 'AveOccup', 'Latitude', 'Longitude']
7 Xs = StandardScaler().fit_transform(df[ozellikler])
8
9 for k in range(2, 8):
10     km = KMeans(k, n_init=10, random_state=42).fit(Xs)
11     print(f"k={k}: inertia={km.inertia_:9.0f} "
12           f"silhouette={silhouette_score(Xs, km.labels_, sample_size=5000, random_state=42)
13           :.4f}")
14 km = KMeans(5, n_init=10, random_state=42).fit(Xs)
15 df['segment'] = km.labels_
16 print(df.groupby('segment').agg(
17     n=('MedHouseVal', 'size'), gelir=('MedInc', 'mean'), yas=('HouseAge', 'mean'),
18     oda=('AveRooms', 'mean'), doluluk=('AveOccup', 'mean'),
19     enlem=('Latitude', 'mean'), boylam=('Longitude', 'mean'),
20     deger=('MedHouseVal', 'mean')).round(2))
```

segment	n	gelir	yas	oda	doluluk	enlem	boylam	deger
0	3869	3.40	41.32	5.11	2.77	37.90	-121.94	2.03
1	8998	3.14	31.21	4.69	3.08	33.96	-118.02	1.86
2	3	6.67	42.33	5.80	781.84	38.02	-121.06	1.85
3	3073	6.54	20.75	7.45	2.97	34.10	-118.15	3.07
4	4697	3.92	18.42	5.79	2.86	37.96	-121.52	1.84

Bilgi

Segment yorumları: **0** eski ve orta gelirli kuzey bölgeleri; **1** güney California'nın kalabalık orta kesimi; **3** yüksek gelirli, geniş evli, pahalı güney bölgeleri (ortalama değer 3,07 = 307 bin \$); **4** yeni yapılaşmış kuzey banliyöleri. **Segment 2** ise yalnızca 3 bölgeden oluşuyor ve ortalama doluluk 781 kişi/hane — bunlar aykırı değer (muhtemelen yurt veya kışla). K-Means aykırı değerlere duyarlıdır: kümeleme öncesi temizlik veya RobustScaler gerekir.

63. Proje 5 — A/B Testi: Tasarımdan Karara

Senaryo

Bir e-ticaret sitesinde “Sepete Ekle” düğmesinin rengi değiştiriliyor. Mevcut dönüşüm oranı %10; yönetim en az *1 puanlık* mutlak artışı (yani %11) tespit edebilmek istiyor. Deney öncesinde kaç kullanıcı gerekir, sonrasında karar nasıl verilir?

63.1 Adım 1: Tasarım — Örneklem Büyüklüğü

```
1 import numpy as np
2 from statsmodels.stats.power import zt_ind_solve_power
3
4 p0, mde = 0.10, 0.01
5 h = 2*np.arcsin(np.sqrt(p0 + mde)) - 2*np.arcsin(np.sqrt(p0))      # Cohen'in h'si
6 n = zt_ind_solve_power(effect_size=h, alpha=0.05, power=0.8, alternative='two-sided')
7 print(f"Cohen h = {h:.4f}    grup basına gereken n = {np.ceil(n):.0f}")
```

Cohen h = 0.0326 grup basına gereken n = 14745

Dikkat

Örneklem büyüklüğü **deneyden önce** hesaplanır ve deney o sayıya ulaşana kadar sonuca bakılmaz. “Anlamlı çıkana kadar her gün kontrol etmek” (*peeking*) Tip I hatayı %5’ten %20–30’a çıkarır. Erken durdurmak istiyorsan ardışık test yöntemleri (sequential testing) gerekir.

63.2 Adım 2: Analiz

```
1 from scipy import stats as st
2 from statsmodels.stats.proportion import proportions_ztest
3
4 rng = np.random.default_rng(42)
5 nA = nB = 17_000
6 xA = rng.binomial(nA, 0.100)      # kontrol
7 xB = rng.binomial(nB, 0.112)      # varyant
8
9 pA, pB = xA/nA, xB/nB
10 print(f"A: {xA}/{nA} = {pA:.4f}    B: {xB}/{nB} = {pB:.4f}")
11 print(f"mutlak fark = {pB-pA:.4f}    göreli = {(pB-pA)/pA:.1%}")
12
13 z, p = proportions_ztest([xB, xA], [nB, nA])
14 print(f"z = {z:.4f}    p = {p:.3e}")
15
16 se = np.sqrt(pA*(1-pA)/nA + pB*(1-pB)/nB)
17 print(f"fark %95 GA = [{(pB-pA) - 1.96*se:.4f}, {(pB-pA) + 1.96*se:.4f}])")
```

A: 1631/17000 = 0.0959 B: 1880/17000 = 0.1106
mutlak fark = +0.0146 göreli = +15.3%
z = 4.4376 p = 9.095e-06
fark %95 GA = [0.0082, 0.0211]

63.3 Adım 3: Bayesçi Bakış — Karar İçin Daha Kullanışlı

```

1 postA = st.beta(1 + xA, 1 + nA - xA)          # Beta(1,1) onsel ile eslenik sonsal
2 postB = st.beta(1 + xB, 1 + nB - xB)
3 sA, sB = postA.rvs(200_000, random_state=1), postB.rvs(200_000, random_state=2)
4
5 print(f"P(B > A)                                = {(sB > sA).mean():.4f}")
6 print(f"beklenen kayıp (B secilirse) = {np.maximum(sA - sB, 0).mean():.6f}")

```

```

P(B > A)                                = 1.0000
beklenen kayıp (B secilirse) = 0.000000

```

Bilgi

Sıklıkçı sonuç “ $p < 0,001$, H_0 reddedilir” der; Bayesçi sonuç “B’nin daha iyi olma olasılığı $> \%99,99$, yanlış seçim yaparsak beklenen kayıp pratikte sıfır” der. **İkincisi karar vericinin duymak istediği cümledir.** Sonuç: varyant B yayına alınır — ama önce güven aralığının alt sınırına bak: en kötü ihtimalle +0,8 puan, en iyi ihtimalle +2,1 puan kazanç.

Dikkat

A/B testinde sık yapılan hatalar: (1) çoklu metrik bakıp düzeltme yapmamak, (2) kullanıcı değil oturum bazında randomizasyon (aynı kullanıcı iki grupta), (3) yenilik etkisi (novelty effect) yüzünden çok kısa süre, (4) alt gruplarda “anamlı” sonuç arayarak p -hacking, (5) istatistiksel anlamlılığı *pratik* anlamlılıkla karıştırmak.

KISIM 13

Referans

64. Dağılım Tablosu

Dağılım	scipy	$\mathbb{E}[X]$	Kullanım
Bernoulli	<code>bernoulli(p)</code>	p	Tek ikili deneme.
Binom	<code>binom(n,p)</code>	np	Başarı sayısı.
Poisson	<code>poisson(mu)</code>	λ	Nadir olay sayımı.
Geometrik	<code>geom(p)</code>	$1/p$	İlk başarıya kadar.
Hipergeometrik	<code>hypergeom(M,n,N)</code>	nK/N	İadesiz çekiliş.
Düzgün	<code>uniform(a,b-a)</code>	$(a+b)/2$	Bilgisizlik.
Normal	<code>norm(mu,sigma)</code>	μ	MLT limiti, hatalar.
Üstel	<code>expon(scale=1/lam)</code>	$1/\lambda$	Bekleme süresi.
Gamma	<code>gamma(a,scale)</code>	$a \cdot \text{scale}$	Toplam bekleme.
Beta	<code>beta(a,b)</code>	$\frac{a}{a+b}$	Oran, Bayesçi önsel.
Ki-kare	<code>chi2(df)</code>	k	Varyans, uyum iyiliği.
Student t	<code>t(df)</code>	0	Küçük örneklem ortalaması.
F	<code>f(dfn,dfd)</code>	$\frac{d_2}{d_2-2}$	Varyans oranı, ANOVA.
Log-normal	<code>lognorm(s,scale)</code>	$e^{\mu+s^2/2}$	Gelir, fiyat, süre.

65. Hangi Testi Seçmeliyim?

Soru	Durum	Test
Ortalama bir değere eşit mi?	Tek grup	ttest_1samp
İki grup ortalaması farklı mı?	Bağımsız	ttest_ind(equal_var=False)
İki ölçüm farklı mı?	Eşleştirilmiş	ttest_rel
≥ 3 grup farklı mı?	Bağımsız	f_oneway
Hangi gruplar farklı?	ANOVA sonrası	pairwise_tukeyhsd
İki kategorik ilişkili mi?	Kontenjans tablosu	chi2_contingency
Küçük tablo	Beklenen < 5	fisher_exact
Normallik yok, iki grup	Sıra temelli	mannwhitneyu
Normallik yok, eşleştirilmiş	Sıra temelli	wilcoxon
Normallik yok, ≥ 3 grup	Sıra temelli	kruskal
İki değişken ilişkili mi?	Doğrusal	pearsonr
İki değişken ilişkili mi?	Monoton	spearmanr
Varsayım yapmak istemiyorum	Her durum	permutation_test
İki oran farklı mı?	A/B testi	proportions_ztest
Dağılım normal mi?	Uyum iyiliği	shapiro, anderson

66. scikit-learn Sözlüğü

Nesne	İş
<code>train_test_split</code>	Eğitim/test ayrımı (stratify kullan).
<code>StratifiedKFold / KFold / GroupKFold</code>	Çapraz doğrulama stratejileri.
<code>TimeSeriesSplit</code>	Zaman serisinde ileriye doğru doğrulama.
<code>Pipeline / make_pipeline</code>	Ön işleme + model zinciri; sızıntıya karşı ana savunma.
<code>ColumnTransformer</code>	Sütun tipine göre farklı ön işleme.
<code>StandardScaler / RobustScaler</code>	Ölçekleme.
<code>OneHotEncoder / OrdinalEncoder</code>	Kategorik kodlama.
<code>SimpleImputer / KNNImputer</code>	Eksik değer doldurma.
<code>GridSearchCV / RandomizedSearchCV</code>	Hiperparametre araması.
<code>cross_val_score / cross_validate</code>	CV skorları (çoklu metrik).
<code>cross_val_predict</code>	CV tahminleri (kalibrasyon/karışıklık matrisi için).
<code>learning_curve / validation_curve</code>	Tanı grafikleri.
<code>permutation_importance</code>	Model bağımsız öznelik önemi.
<code>PartialDependenceDisplay</code>	PDP / ICE grafikleri.
<code>CalibratedClassifierCV</code>	Olasılık kalibrasyonu.
<code>classification_report</code>	Kesinlik/duyarlılık/F1 özeti.
<code>DummyClassifier / DummyRegressor</code>	Temel çizgi — her zaman kur.

67. Hızlı Tarif Kitabı

```

1 import numpy as np, pandas as pd, seaborn as sns
2 from scipy import stats as st
3 import statsmodels.formula.api as smf
4
5 # 1) Veriyi tani
6 df.info(); df.describe(); df.isna().sum(); df.nunique()
7
8 # 2) İki grup ortalaması farklı mı? (+ etki büyüklüğü)
9 st.ttest_ind(a, b, equal_var=False)
10
11 # 3) Kategorik ilişkisi
12 st.chi2_contingency(pd.crosstab(df.x, df.y))
13
14 # 4) Güven aralığı
15 st.t.interval(0.95, len(x)-1, x.mean(), st.sem(x))
16
17 # 5) Bootstrap (formülü olmayan istatistik için)
18 st.bootstrap((x,), np.median, confidence_level=0.95)
19
20 # 6) Regresyon + tam özet
21 smf.ols('y ~ x1 + x2 + C(grup)', data=df).fit().summary()
22
23 # 7) Lojistik regresyon + odds ratio
24 m = smf.logit('y ~ x1 + C(grup)', data=df).fit(); np.exp(m.params)

```

```

25
26 # 8) Sızıntısız ML hattı
27 from sklearn.pipeline import Pipeline
28 pipe = Pipeline([('prep', on_isleme), ('clf', model)])
29 cross_val_score(pipe, X, y, cv=StratifiedKFold(5, shuffle=True, random_state=42))
30
31 # 9) Hiperparametre ayarı
32 GridSearchCV(pipe, izgara, cv=cv, scoring='roc_auc', n_jobs=-1).fit(Xtr, ytr)
33
34 # 10) Model yorumlama
35 permutation_importance(model, Xte, yte, n_repeats=20, random_state=42)

```

68. Sık Yapılan Hatalar

Hata	Doğrusu
Ön işlemeyi ayırmadan önce yapmak	Her şeyi Pipeline içine koy.
Test setine bakarak karar vermek	Test seti yalnız en sonda, bir kez.
Doğruluğa güvenmek	Dengesiz veride PR-AUC, duyarlılık, maliyet.
$p < 0,05$ 'i keşif saymak	Etki büyüklüğü + güven aralığı raporla.
Çoklu test düzeltmesi yapmamak	Holm veya Benjamini–Hochberg.
Korelasyonu nedensellik sanmak	Karıştırıcıları düşün; mümkünse deney yap.
Aykırı değeri sessizce silmek	Önce neden orada olduğunu anla.
Zaman serisini rastgele bölmek	Kronolojik ayır, TimeSeriesSplit.
Gini önemine güvenmek	Permütasyon önemi veya SHAP.
Ölçeklemeyi unutmak	k -NN, SVM, PCA, sinir ağları için zorunlu.
Temel çizgi kurmamak	DummyClassifier ile karşılaştır.
random_state vermemek	Tekrarlanabilirlik için her yere ver.
Varyans için np.std	Örnekleme ddof=1 kullan.
Eşiği 0,5'te bırakmak	Eşik bir iş karardır; maliyete göre seç.

69. Genel Bakış: Üç Alan Nasıl Bağlanıyor?

Kavram	İstatistikte	Makine öğrenmesinde
Varyans ayrıştırma	ANOVA'da η^2	Karar ağacında bölme ölçütü
Olabilirlik	MLE ile parametre tahmini	Çapraz entropi kaybı
Yanlılık–varyans	Tahmin edicinin MSE'si	Eksik/aşırı öğrenme
Düzenleştirme	Büzülme (shrinkage) tahmini	Ridge, Lasso, dropout
Bayes teoremi	Sonsal çıkarım	Naive Bayes, kalibrasyon
Yeniden örnekleme	Bootstrap güven aralığı	Bagging, Random Forest
Çoklu karşılaştırma	FDR düzeltmesi	Hiperparametre araması yanlılığı
Karıştırıcı değişken	Kovaryans analizi	Veri sızıntısı, sahte öznitelik

İpucu

Çalışma yöntemi. (1) Her kavramı önce *simüle et*: gerçeği bildiğin veri üret, yöntemin onu bulup bulmadığına bak. (2) Sonra *gerçek veriye* uygula ve sonucun anlamlı olup olmadığını sorgula. (3) Sonucu *boz*: aykırı değer ekle, varsayımı ihlal et, örnekleme küçült — yöntemin nerede çöktüğünü gör. (4) Her bulguyu *iki farklı yöntemle* doğrula (örneğin *t*-testi ve permütasyon testi). Bir analistin değeri, doğru cevabı bulmasından çok, *yanlış cevabı fark edebilmesindedir*.